OLK — Open Library Kashmir

The Complete Technical Manual

An Engineering Handbook for Maintainers & Interns

Architecture ▪ Frontend ▪ Supabase & Docker ▪ Deployment ▪ Maintenance

First Edition ▪ July 2026

Team OLK

Preface — Read This First

How this manual is organised

- **Part I — Orientation.** What OLK is, who it is for, and the shape of the system at a glance. Read this even if you only have ten minutes.
- **Part II — Environment & Workflow.** How to get the project running on your machine — including the local Docker/Supabase stack — and the conventions the project expects you to follow.
- **Part III — The Frontend.** Next.js routing, every page, every shared component, and the styling system.
- **Part IV — The Backend.** Supabase as a concept, then the full database schema, every function and trigger, the row-level security model, and how server-side actions and notifications work.
- **Part V — Key Workflows End to End.** The same system traced through its three central journeys: exchanging a book, matching a wishlist, and moderating the community — reading top to bottom instead of file to file.
- **Part VI — Operating in Production.** Deploying to Vercel, the full environment variable reference, a maintenance playbook of "if you need to do X, start here," a candid list of known issues, and a troubleshooting guide.
- **Appendices.** A compact table reference, a command cheat-sheet, and a glossary.

Conventions used in this manual

Four recurring boxes appear throughout. They are not decoration — they are load-bearing:

> **NOTE**: Background context or a clarification that helps the surrounding text make sense. Safe to skim on a first read.

> **PRO TIP**: A shortcut, a command, or a habit that will save you time. Worth remembering.

> **WARNING**: A place where the obvious thing to do is the wrong thing to do. Read these before you touch the related code.

> **KNOWN ISSUE**: A documented, real, currently-existing bug, gap, or piece of technical debt in the codebase as of this edition — not hypothetical. Each one names the exact file and line. See also Chapter 22, *Known Issues & Technical Debt*, which aims to collect all of them in one place.

INTERN EXERCISE

A hands-on exercise. If you are an intern working through this manual for the first time, do these — reading about a codebase and changing it are different skills, and these are designed to be small enough to finish in under an hour each while touching a real, working part of the system.

Code listings are copied verbatim from the repository and captioned with their file path, so you can always locate the original with a simple search. Where a listing has been trimmed for length, an ellipsis comment (`// ...`) marks the cut.

A note on honesty and openness

This manual documents OLK as it actually exists today — including the unfinished edges, the occasional inconsistency, and the deliberate trade-offs made along the way. We believe that a guide describing only an idealised design, while the shipped code tells a different story, actively misleads the next person who trusts it. By being candid about where we stand, we hope to save you time and frustration, and to invite you into a culture of transparency.

From the very beginning, OLK has been conceived as an open-source project. It is not a closed product delivered from on high; it is a living foundation for experimentation and collective growth. We actively encourage students, developers, and technologists to fork the repository, try out emerging tools, propose novel architectures, and push the platform in directions we have not yet imagined. Whether you are refining a database query, prototyping a new feature, or rethinking the entire UI layer, your contributions are welcome here. This manual is your starting point.

First edition — compiled July 2026
Owais Saleem

Contents

Preface — Read This First **i**

How this manual is organised i

Conventions used in this manual i

A note on honesty and openness ii

I Orientation **1**

1 Welcome & Vision **2**

1.1 What OLK is 2

1.2 Core features 2

1.3 Who this manual is for 3

1.4 The bigger picture 3

1.5 An infrastructure audit, and what we did with it 4

1.6 A tour of the rest of this manual, in one paragraph 5

2 Tech Stack at a Glance **6**

2.1 The stack, top to bottom 7

2.2 Full dependency manifest 7

2.3 Why this stack, in brief 9

3 System Architecture **10**

3.1 The system, end to end 10

3.2 Server Components, Client Components, and where the boundary actually falls . . 11

3.3 Where the Proxy sits 11

3.4 Local development vs. production, side by side 12

II Environment & Workflow **13**

4 Local Environment Setup: Docker & Supabase CLI **14**

4.1 Prerequisites 14

4.2 Why Docker, specifically . . . 14
4.3 Step by step . . . 15
4.3.1 1. Install the Supabase CLI . . . 15
4.3.2 2. Initialise the project config (already done — skip if `supabase/config.toml` exists) . . . 15
4.3.3 3. Start the stack . . . 15
4.3.4 4. Apply every migration from a clean slate . . . 16
4.3.5 5. Point the app at the local stack . . . 16
4.3.6 6. Run the app . . . 17
4.4 Day-to-day commands . . . 18
4.5 The intended workflow: develop locally, push when it works . . . 18
4.6 Verifying local and cloud are actually in sync . . . 19
4.6.1 Schema: do both sides agree on which migrations have run? . . . 19
4.6.2 Data: do both sides hold the same rows? . . . 19
4.6.3 When "cloud" becomes "our own server" . . . 20
4.7 Pulling the hosted project's data into local . . . 20
4.8 The daily-safe protocol: resetting without losing local data . . . 21
4.8.1 The tempting wrong answer: a committed `supabase/seed.sql` . . . 21
4.8.2 The actual answer: chain the two commands that already exist . . . 22
4.9 Useful local endpoints . . . 23
4.10 Why this matters beyond convenience . . . 23

5 Project Structure & File-by-File Reference 24
5.1 Top-level layout . . . 24
5.2 `src/app` — every route in the application . . . 24
5.3 `src/components` — shared UI . . . 26
5.4 `src/hooks` — small reusable client hooks . . . 27
5.5 `src/lib` — server logic and small utilities . . . 27
5.6 Configuration files at the repository root . . . 28

6 Development Workflow & Conventions 29
6.1 Migrations: the one hard rule . . . 29
6.2 Keeping `schema.sql` honest . . . 29
6.3 Linting and CI . . . 30
6.4 Automated tests: a minimal but real foundation . . . 31
6.5 Git conventions . . . 31

III The Frontend **33**

7 Next.js App Router, Routing & the Proxy Layer **34**

7.1 App Router conventions used in this project . . . 34
7.2 The Proxy layer — the rename, applied . . . 34
7.3 What the Proxy actually does, line by line . . . 34
7.4 The `matcher` config . . . 38

8 Pages Walkthrough **39**

8.1 The landing page — `app/page.tsx` . . . 39
8.2 Browse — `app/(dashboard)/browse/page.tsx` . . . 39
8.3 My Books — `app/(dashboard)/my-books/page.tsx` . . . 40
8.4 Requests — `app/(dashboard)/requests/page.tsx` . . . 41
8.5 Messages — list and detail . . . 42
8.6 Clubs & Events — directory, creation, and detail . . . 42
8.7 The admin section . . . 43

9 Shared Components Library **45**

9.1 The ISBN scanner — `isbn-scanner.tsx` . . . 45
9.2 The realtime notification bell — `notification-bell.tsx` . . . 46
9.3 The request lifecycle stepper — `request-stepper.tsx` . . . 47
9.4 Modals: a consistent portal pattern . . . 47
9.5 Toasts: a global notice, without a Context Provider . . . 48

10 The Styling System **50**

10.1 Tailwind CSS v4 — a CSS-first configuration . . . 50
10.2 No component library, no shared design primitives . . . 51
10.3 Fonts . . . 53
10.4 Dark mode . . . 53

IV The Backend **54**

11 Supabase Fundamentals **55**

11.1 What Supabase actually is . . . 55
11.2 Two ways application code talks to Postgres . . . 55
11.3 `SECURITY DEFINER`: the escape hatch from RLS, used deliberately . . . 56
11.4 `auth.users` vs. `public.profiles` . . . 57

12 Database Schema Reference **58**

12.1 Identity . . . 58
12.2 Books & the Exchange Lifecycle . . . 59
12.3 Community Features . . . 60
12.4 Notifications & Content . . . 61
12.5 Admin & Moderation . . . 62

13 Functions, Triggers & RPCs **64**

13.1 Identity & role helpers . . . 64
13.2 Discovery: geography . . . 64
13.3 Public aggregates . . . 66
13.4 The exchange lifecycle: completion & read-count . . . 66
13.5 Club membership and event attendance counting . . . 67
13.6 Club-creation eligibility and book-notes limits, enforced at the database layer . . . 67
13.7 Wishlist fuzzy matching . . . 68
13.8 Rate limiting . . . 69
13.9 Admin analytics . . . 70

14 Row-Level Security & the Role Model **71**

14.1 What RLS actually does . . . 71
14.2 The role hierarchy . . . 71
14.3 Policy summary, table by table . . . 72
14.4 Six RLS looseness gotchas — since tightened . . . 73

15 Server Actions, Email & Notifications **76**

15.1 What a Server Action is, and how OLK uses them . . . 76
15.2 The admin action pattern: guard, act, log, revalidate . . . 76
15.3 Notifications: in-app row first, email best-effort second . . . 78
15.4 Sending the actual email: a service-role client, on purpose . . . 79
15.5 The weekly digest: a genuine Route Handler . . . 80

V Key Workflows End to End **82**

16 The Book Exchange Lifecycle, End to End **83**

16.1 The full state machine . . . 83
16.2 Step 1 — Listing . . . 83
16.3 Step 2 — Discovery . . . 83

16.4 Step 3 — Requesting . . . 84
16.5 Step 4 — Accept, decline, or expire into noise . . . 84
16.6 Step 5 — Handover . . . 84
16.7 Step 6a — Lending: the book comes back . . . 85
16.8 Step 6b — Donating: the book changes hands permanently . . . 85

17 Wishlists, Matching, Clubs & Events **86**
17.1 Wishlists: asking for a book that doesn't exist yet . . . 86
17.2 Clubs: geography plus a trust gate . . . 86
17.3 Events: club-scoped, but publicly discoverable . . . 88

18 The Admin & Moderation Subsystem, End to End **90**
18.1 A report's journey . . . 90
18.2 The audit log is the whole point . . . 91
18.3 Beyond reports: content and settings . . . 91

VI Operating in Production **93**

19 Deploying to Vercel **94**
19.1 The mechanical part . . . 94
19.2 Running the database migrations against production . . . 94
19.3 The digest cron: wired up . . . 95
19.4 What Vercel does *not* give you here . . . 95

20 Environment Variables Reference **97**
20.1 Local vs. production, side by side . . . 98

21 The Maintenance Playbook **99**
21.1 "I need to add a new page" . . . 99
21.2 "I need to change the database schema" . . . 99
21.3 "I need to add a new admin action" . . . 100
21.4 "I need to add a new notification type" . . . 100
21.5 "I need to change a rate limit or feature flag" . . . 100
21.6 "I need to test something as a banned user / admin / moderator" . . . 100
21.7 "I need to regenerate the schema dump" . . . 100
21.8 "I need to run a security & code-quality audit" . . . 101
21.9 "Something changed in this codebase and this manual is now wrong" . . . 101

22 Known Issues & Technical Debt **103**

22.1 Still open . . . 103

23 Troubleshooting Guide **104**

VII Appendices **107**

A Full Table Reference **108**

B Command Cheat-Sheet **110**

B.1 Node / Next.js . . . 110

B.2 Supabase CLI (via npx — nothing installed globally) . . . 110

B.3 Docker (rarely needed directly — the Supabase CLI drives this for you) . . . 110

B.4 Verifying the environment (Chapter 4) . . . 111

B.5 Compiling this manual . . . 111

C Glossary **112**

PART I

Orientation

CHAPTER 1

Welcome & Vision

1.1 What OLK is

Open Library Kashmir (OLK) was conceived as a community-driven book-sharing initiative to serve readers across the Kashmir Valley. The original operating model relied on local volunteers to collect donated books and redistribute them to students in need. However, the organization rapidly encountered significant operational constraints—including a persistent shortage of volunteers and donors, as well as a limited distribution network that prevented us from reaching our target student demographic. These challenges necessitated a comprehensive strategic redesign of the platform.

The current model introduces a highly efficient, decentralized approach by eliminating nearly all intermediary layers. It establishes a direct, peer-to-peer connection between individuals looking to share their literature and those actively seeking it. Users with surplus books can catalog their collections on the platform, while prospective readers can browse the available inventory and submit a request. The involved parties then coordinate a mutually convenient handover or an agreed-upon shipping arrangement. By removing centralized warehousing and logistical intermediaries, the entire transaction takes place through direct, transparent user-to-user communication.

At the core of this architectural shift is a commitment to fostering meaningful human connections through the shared love of reading. This direct engagement extends beyond the simple exchange of books, enabling a richer, more substantive interaction between users. The Clubs feature is a natural extension of this philosophy—it empowers passionate individuals to create interest-based communities, invite like-minded members, and organize events, all within a single, unified digital ecosystem.

1.2 Core features

- **Community book sharing** — list a book you own as either a *loan* (you get it back) or a *donation* (you don't), with condition, genre, and now (as of the two most recent migrations) a description and publication year enriched automatically from an ISBN scan.
- **Location-aware discovery** — browsing sorts by distance from the viewer using the `get_books_nearby` RPC (Chapter 13), not just recency.
- **Requesting & the exchange lifecycle** — a full state machine from `pending` through `accepted`, `handed_over`, and `returned`, tracked per-request with reading-progress and due-date awareness (Chapter 16).
- **Messaging** — a realtime 1:1 chat scoped to each book request, so conversation history is naturally organised by exchange rather than by contact.

- **Wishlists with fuzzy matching** — ask for a title you can't find; the moment anyone lists something close to it, a `pg_trgm`-powered match fires a notification.
- **Reading clubs** — geographically-scoped community groups with posts, membership, and an eligibility gate tied to a user's exchange history.
- **Ratings & reputation** — a lightweight 1–5 star system that feeds a "Trusted Sharer" badge on public profiles.
- **Reporting & moderation** — every user and book can be reported; a full admin/moderator subsystem triages, bans, warns, and audits (Chapter 18).
- **Notifications & email digests** — in-app notifications with a Resend-powered weekly email digest for subscribed users.

1.3 Who this manual is for

Primarily, the two or three interns who are about to inherit this codebase. You are not expected to have touched Next.js's App Router before, or Supabase, or even necessarily written Postgres row-level security policies — this manual explains all three from the ground up in Parts III and IV. What it does assume is that you can read TypeScript and SQL.

> **WARNING**: This exact version of Next.js is **not** the Next.js in your training data or muscle memory. The project's own `AGENTS.md` says so explicitly: *"This version has breaking changes — APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in `node_modules/next/dist/docs/` before writing any code."* A concrete example: Next.js 16 renamed *Middleware* to *Proxy*. This project uses the current `src/proxy.ts` convention. See Chapter 7 for the full story.

1.4 The bigger picture

OLK wants two things, in this order. First: get the platform live, publicly, most likely on Vercel, serving real readers in Kashmir. Second, and this is the part worth stating explicitly because it will shape architectural decisions for years: **eventually run the platform's own infrastructure — including Supabase itself, since it is open source — on a local server physically located in Kashmir**, rather than depending indefinitely on a remote, foreign-hosted database.

That second goal is why Chapter 4 is not a throwaway "how to run this on your laptop" section. Learning to run Supabase's full stack under Docker locally — Postgres, GoTrue auth, PostgREST, Realtime, Storage, Studio, all of it — is a rehearsal for the day that same stack runs on a physical machine in-region instead of on a developer's laptop. Every command in that chapter (`supabase start`, `supabase stop`, `supabase db reset`) is a command that will eventually run against real hardware, not a metaphor for it.

> **NOTE**: As of this edition, the project already keeps a working local Docker/Supabase environment side-by-side with the hosted one, switched with a single environment variable (see Chapter 4). This exists specifically so that day-to-day development does not have to touch the production database at all.

1.5 An infrastructure audit, and what we did with it

In July 2026, a colleague of the founder reviewed the project and wrote up a full self-hosting migration plan: VPS + PM2 + Caddy in place of Vercel, a self-hosted Supabase Docker stack (Postgres, GoTrue, PostgREST, Realtime, Storage) in place of the managed cloud project, PgBouncer for connection pooling, a self-hosted mail server (Postfix/Mailu) in place of Resend, and system cron in place of Vercel Cron — all sized for a projected **100,000** users.

> **NOTE**: The plan's individual pieces were technically sound — PM2+Caddy, PgBouncer, cron, `pg_dump` backups are all conventional, correct choices. The problems were in the framing, not the components:
>
> - **The 100k figure had no basis.** At the time of the audit this project was pre-launch — no `/messages` page yet, no real users. Designing connection pooling and disaster recovery around a user count you don't have yet is solving a problem before it exists.
> - **"Zero code changes" for the Supabase swap is true narrowly, misleading broadly.** `src/lib/supabase/server.ts` and `client.ts` do just read environment variables, so the client code claim holds. But self-hosting Supabase means *you* now own Postgres backups, GoTrue config, the Kong gateway, Realtime, and Storage as running services with their own upgrade cadence and CVEs — a real ongoing burden for a single-developer project, not a config change.
> - **Self-hosted email is the highest-risk item and the report understated it.** A fresh VPS IP gets throttled or spam-foldered by Gmail/Outlook regardless of correct SPF/DKIM/DMARC — sending reputation is built over months, not configured. For a notifications/digest feature, landing in spam defeats the point. Resend's free tier is cheap insurance against this failure mode; self-hosting email is a false economy at this stage.

Decision (11-07-2026): none of this ships now. The project stays on Vercel + managed Supabase + Resend until there are more developers and evidence of real load. The first-year planning target is **10,000** users, not 100k — reasonable for a regional community platform and not a number that stresses the current managed stack.

> **WARNING**: A VPS is **not** automatically a step toward the data-sovereignty goal stated earlier in this chapter. Renting a generic VPS (Hetzner, DigitalOcean, AWS, ...) just swaps one third party's control (Vercel/Supabase Inc.) for another's — the data still lives on hardware you don't own, in a jurisdiction you didn't choose. It only becomes a genuine step *toward* sovereignty if it's treated as a rehearsal: running the same self-hosted Docker stack already exercised locally (Chapter 4) on rented infrastructure first, to prove the operational playbook — backups, upgrades, monitoring, TLS — before ever committing to physical hardware in-region. Stopping at "we rent a VPS instead of Vercel" and calling it done would be a lateral move, not progress on the actual goal.

What's parked for later, revisited once there's more than one developer and/or real signs of load (not before):

- VPS-hosted compute (PM2 + Caddy) — as a rehearsal step per the note above, not a cost-cutting move.
- Self-hosted Supabase Docker stack, including PgBouncer once connection limits are actually observed rather than assumed.

- Self-hosted mail — only if Resend's pricing or limits actually become a problem; deliverability risk needs a real plan first, not just DNS records.
- Automated off-site backups (`pg_dump` to self-hosted MinIO or similar) — worth doing earlier than the rest of this list, independent of the hosting question, once there's real user data worth losing.

What shipped immediately instead: indexes on the foreign-key and status columns the audit correctly flagged as missing — `supabase/migrations/20260711093615_index_fk_and_status_columns.sql` adds eleven `btree` indexes across `book_requests` (`book_id`, `requester_id`, `status`), `ratings` (`request_id`, `rater_id`, `rated_user_id`), and `reports` (`reporter_id`, `reported_user_id`, `reported_book_id`, `status`, `assigned_to` — Chapter 12). A cheap, reversible fix with no architectural bet attached, unlike everything else on this list.

1.6 A tour of the rest of this manual, in one paragraph

You will set up the project locally with Docker (Part II), learn how the Next.js frontend is organised page by page and component by component (Part III), learn the Supabase/Postgres backend table by table, function by function, and policy by policy (Part IV), then watch those two halves work together across three real user journeys — exchanging a book, matching a wishlist, moderating a report — traced start to finish (Part V). Part VI is what to do once this is live: deploying, the environment variables that matter, a maintenance playbook organised as "if you need to do X, start here," and a candid accounting of what is still broken or unfinished. Let's begin.

CHAPTER 2

Tech Stack at a Glance

2.1 The stack, top to bottom

Layer	Technology	Role in OLK
Hosting	Vercel	Builds and serves the Next.js app; will run the weekly digest via a scheduled call to `/api/digest` (Chapter 19).
Framework	Next.js 16.2.6 (App Router)	Routing, Server Components, Server Actions, the Route Handler under `src/app/api`. **Not** the Next.js of most tutorials — see Chapter 1's warning.
UI runtime	React 19.2.4	Server/Client Component model; no separate state-management library is used anywhere in the codebase.
Language	TypeScript 5 (strict mode)	All application code; `tsconfig.json` has `"strict":true`.
Styling	Tailwind CSS v4 (`@tailwindcss/postcss`)	Utility classes throughout; no component library (no MUI/Chakra/shadcn) — every button, modal, and card is hand-built.
Backend platform	Supabase (self-hostable)	Postgres, Auth (GoTrue), PostgREST, Realtime, Storage — all consumed through `@supabase/supabase-js` and `@supabase/ssr`.
Local backend runtime	Docker (via the Supabase CLI)	Runs the entire Supabase stack as containers on a developer's machine — the same images that would run self-hosted in production one day.
Transactional email	Resend (`resend` npm package)	Notification emails and the weekly digest, both sent from server-only code.

2.2 Full dependency manifest

`package.json` is intentionally small. There is no state management library, no data-fetching cache library (no React Query / SWR), no UI kit, no charting library, no form library, and only a minimal Vitest setup (Chapter 6) rather than full test coverage. Every one of those things that exists in the app was hand-rolled — the admin dashboard's growth charts are CSS bars, the

ISBN scanner drives the browser's native `BarcodeDetector` API directly, and every form is plain controlled React state with a raw Supabase call.

Listing 2.1: `package.json` — dependencies

```
"dependencies": {
  "@supabase/ssr": "^0.10.3",
  "@supabase/supabase-js": "^2.106.2",
  "next": "16.2.6",
  "react": "19.2.4",
  "react-dom": "19.2.4",
  "resend": "^6.14.0",
  "server-only": "^0.0.1"
},
"devDependencies": {
  "@tailwindcss/postcss": "^4",
  "@types/node": "^20",
  "@types/react": "^19",
  "@types/react-dom": "^19",
  "eslint": "^9",
  "eslint-config-next": "16.2.6",
  "tailwindcss": "^4",
  "typescript": "^5",
  "vitest": "^4.1.9"
}
```

- `@supabase/ssr` — the cookie-aware Supabase client factory used in both `src/lib/supabase/client.ts` (browser) and `src/lib/supabase/server.ts` (Server Components/Actions). This is the package that makes auth sessions work correctly across Next.js's server/client boundary; do not replace calls to it with the plain `@supabase/supabase-js` client inside anything that runs on the server and needs the logged-in user's session.
- `@supabase/supabase-js` — the underlying client. Used directly (without `ssr`) in two specific places that intentionally act with elevated, non-session privileges: `src/lib/send-notification-email.ts` and `src/app/api/digest/route.ts`, both of which construct a client with the **service role key** to look up any user's email address regardless of who is logged in.
- `resend` — a thin wrapper around Resend's transactional email API. Both the per-notification email and the digest cron degrade gracefully (silently no-op) if `RESEND_API_KEY` is unset, which is exactly what happens by default in local development.
- `server-only` — a zero-runtime marker package; importing it into a file makes Next.js throw a build error if that file is ever accidentally imported into client-side code. Used in `send-notification-email.ts` to guarantee the service-role key can never end up in a browser bundle.

> **PRO TIP**: There is no ORM (no Prisma, no Drizzle). Every database interaction goes through the Supabase JS client's query builder (`.from('table').select(...)`) or a raw Postgres function call (`.rpc('function_name', {...})`). If you are used to an ORM's autocompletion, lean on the schema reference in Chapter 12 instead — table and column names are not statically checked anywhere in this codebase.

2.3 Why this stack, in brief

Next.js + Supabase is a deliberately low-operational-overhead pairing for a small team: Supabase gives you a production Postgres database, authentication, file storage, and realtime subscriptions without writing a backend service, and Next.js's App Router lets server-side code (Server Components, Server Actions) talk to that database directly without a separate API layer for most operations — the one exception being `src/app/api/digest/route.ts`, a genuine Route Handler, because it needs to be triggered by an external cron caller rather than a page render.

The absence of an ORM and of a component library is a scope choice, not an oversight: for a project this size, hand-writing queries and UI keeps the dependency surface small enough for two or three interns to hold the whole thing in their heads — which is precisely the audience this manual is for.

INTERN EXERCISE

Pick one of the hand-rolled things named above — the admin dashboard's CSS growth bars (`admin/page.tsx`, Chapter 18), the ISBN scanner's `BarcodeDetector` integration (`isbn-scanner.tsx`, Chapter 9), or any plain-controlled-state form — and read it end to end. Then ask honestly: what npm package would a typical tutorial reach for here (a charting library, a form library), and what would adopting it actually buy this specific project at its current size? This isn't rhetorical. Being able to answer it with a real file open, not just agree with the paragraph above, is what "the dependency surface stays small enough to hold in your head" means in practice.

CHAPTER 3

System Architecture

3.1 The system, end to end

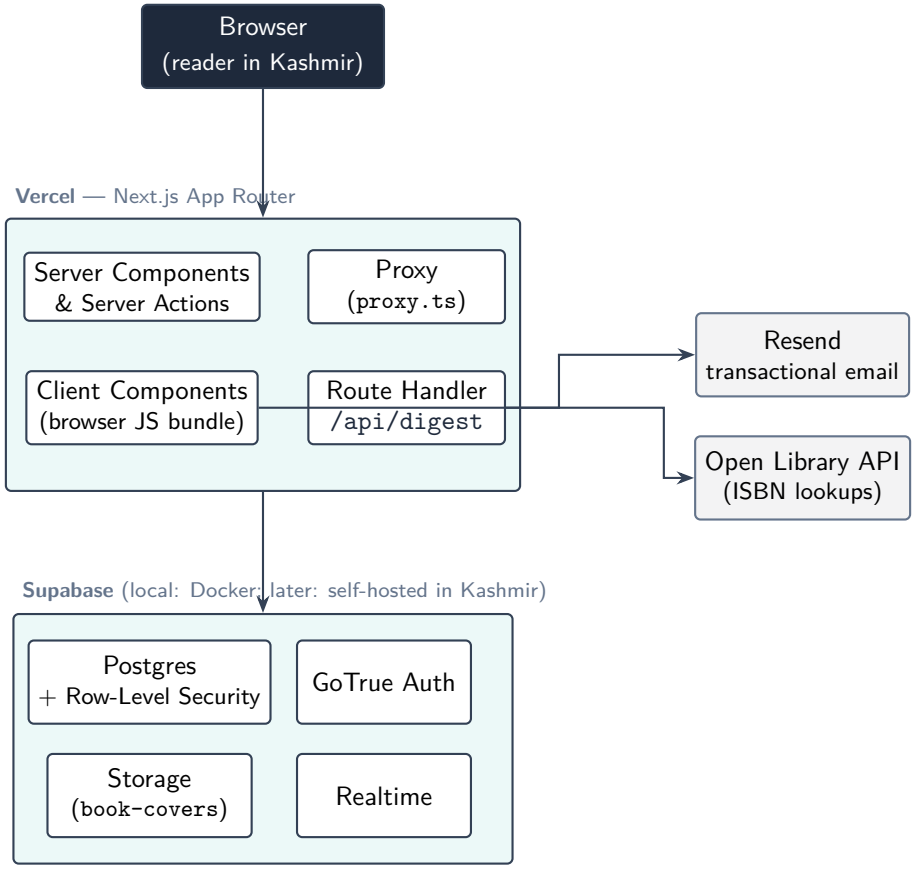

Figure 3.1: OLK's runtime shape. The Client-Component box is the only part of Vercel's box that actually executes in the reader's browser; everything else in that box runs on Vercel's servers. The Supabase box is a single Docker Compose stack in local development and, today, Supabase's hosted cloud in production — the long-term goal (Chapter 1) is for that same box to be a physical server in Kashmir.

3.2 Server Components, Client Components, and where the boundary actually falls

Every file under `src/app` is a Server Component by default. A file becomes a Client Component only if its first line is the string `"use client"`. This is not a stylistic choice per file — it changes what APIs are available and where the code physically runs:

- **Server Components** (no directive) render on Vercel's servers, can call `await createClient()` from `src/lib/supabase/server.ts` directly, and never ship their data-fetching code to the browser. `src/app/page.tsx` (the landing page) and every `layout.tsx` in this project are Server Components.
- **Client Components** (`"use client"`) render once on the server for the initial HTML, then hydrate and re-render in the browser. They use `src/lib/supabase/client.ts`'s `createClient()`, which talks to Supabase directly over HTTPS from the reader's browser — meaning every query a Client Component makes is subject to Row-Level Security as the logged-in user, never as an elevated role. Nearly every interactive page in the dashboard (`browse`, `my-books`, `requests`, `messages/[id]`, all of `admin/*`) is a Client Component, because they need `useState`/`useEffect` for interactivity.

> **NOTE**: This project's most complex pages are almost entirely Client Components, even though a "fetch on the server, render once" Server Component would often be more efficient. That is a real, current characteristic of the codebase, not a recommendation — see Chapter 8 for page-by-page detail, and treat "should this be a Server Component instead" as a legitimate refactor to propose, not a rule that's already been decided against.

3.3 Where the Proxy sits

Every request to the app — before any Server Component runs, before any HTML is generated — passes through `src/proxy.ts`. It refreshes the Supabase auth session cookies (required because `@supabase/ssr` needs a chance to rotate tokens on every request), checks `platform_settings` feature flags and maintenance mode, redirects unauthenticated visitors away from a fixed list of protected route prefixes, and redirects banned users or non-admins away from routes they should not reach. Chapter 7 covers this file line by line.

3.4 Local development vs. production, side by side

	Local development	Production (today)
Frontend	`next dev` on `localhost:3000`	Vercel build of the same repository
Database & Auth	Docker containers started by `supabase start`, at `127.0.0.1:54321`/`:54322`	Supabase's hosted cloud project
Config source	`.env.local` pointed at `127.0.0.1`	Environment variables set in the Vercel dashboard
Email	Silently disabled (no `RESEND_API_KEY` set)	Live, via Resend
Migrations	`supabase db reset` replays every file in `supabase/migrations`	Applied once, in order, via `supabase db push` or the linked project

The two columns are deliberately structured to be as close to identical as possible — the same migrations, the same RLS policies, the same functions — so that "it works locally" is a meaningful, trustworthy signal before you ever touch the hosted database. Chapter 4 is the practical walkthrough of the left column.

INTERN EXERCISE

Pick one concrete action — requesting a book from Browse is a good one — and trace it through Figure 3.1 by name, using only this chapter and the cross-references it points to, *before* reading the chapters that cover each hop in detail. Which box first receives the click (Client Components — `browse/page.tsx` is `"use client"`, Chapter 8)? What talks to Postgres, and as which role (Chapter 14)? What inside the Supabase box might reject the request even though your code is correct (Chapter 13)? Does the Proxy box get involved for this specific action, or only for the page load that preceded it (Chapter 7)? Write the chain of boxes down, left to right, then go verify it as you read those chapters. Getting this wrong on the first attempt and correcting it teaches you the system's shape far faster than reading the right answer first would.

PART II

Environment & Workflow

CHAPTER 4

Local Environment Setup: Docker & Supabase CLI

This chapter gets you from a fresh clone to a fully working local copy of OLK — frontend and database both — running entirely on your own machine, with zero requests ever touching the hosted production database. Follow it in order the first time; after that, Appendix B is a faster daily reference.

4.1 Prerequisites

- Node.js 18.18 or later, and `npm`.
- **Docker** — this is the piece that runs Supabase locally. Docker Desktop on macOS/Windows, or the Docker Engine + CLI on Linux. Verify with:

Listing 4.1: Verifying Docker is installed and the daemon is reachable

```
docker --version
docker info    # should print server info, not a connection error
```

> **WARNING:** On Linux, your user account needs to be in the `docker` group (`groups` should list `docker`) or every command in this chapter will need `sudo`. If it's missing: `sudo usermod -aG docker $USER`, then log out and back in.

4.2 Why Docker, specifically

Supabase is not one server — it is roughly a dozen cooperating services: Postgres itself, GoTrue (auth), PostgREST (the REST API that turns table/RLS definitions into HTTP endpoints), Realtime, Storage, a Postgres-metadata API, Kong (API gateway), Mailpit (a fake SMTP inbox for local auth emails), Studio (the admin UI), and a couple of supporting services for logs and analytics. Docker is what lets the Supabase CLI hand you all of that with two commands instead of installing each service by hand. You will never run a raw `docker` or `docker compose` command yourself in this project — the Supabase CLI drives Docker for you.

4.3 Step by step

4.3.1 1. Install the Supabase CLI

There is nothing to install globally — the project uses it via `npx`, which downloads and caches it on first use:

Listing 4.2: Confirming the CLI is reachable

```
npx supabase --version
```

4.3.2 2. Initialise the project config (already done — skip if `supabase/config.toml` exists)

Listing 4.3: One-time: generates supabase/config.toml

```
npx supabase init
```

This writes `supabase/config.toml`, which fixes the project's identity to `project_id = "olk"` — this is what makes every container the CLI creates be named `supabase_<service>_olk`, and it's how `supabase start` recognises "the OLK stack" specifically versus any other Supabase project you might also have running locally.

> **KNOWN ISSUE**: This exact situation happened during this manual's own research: a full set of `supabase_*_olk` Docker containers were already running on the development machine, healthy, with **no `supabase/config.toml` in the repository at all** and no record of it in git history. The Supabase CLI had matched containers to the working directory name (`olk`) even without a committed config. Worse, that stack's schema was stale — it only had the first 7 tables from the very first migration, missing 8 subsequent migrations' worth of columns and tables (`bio`, geolocation, clubs, bans, ratings, and more). **Lesson:** never assume a running `_olk` container set is up to date. Always check (`docker exec supabase_db_olk psql -U postgres -d postgres -c "\dt public.*"`) against `supabase/migrations` before trusting it, and when in doubt, run `supabase db reset`.

4.3.3 3. Start the stack

Listing 4.4: Pulls Supabase's Docker images (first run only) and starts every service

```
npx supabase start
```

On success, this prints a block of credentials — save this output, or just re-run `npx supabase status` any time you need it again:

Listing 4.5: Typical 'supabase start' / 'supabase status' output

```
{
  "ANON_KEY": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
```

```
  "API_URL": "http://127.0.0.1:54321",
  "DB_URL": "postgresql://postgres:postgres@127.0.0.1:54322/postgres",
  "GRAPHQL_URL": "http://127.0.0.1:54321/graphql/v1",
  "INBUCKET_URL": "http://127.0.0.1:54324",
  "MAILPIT_URL": "http://127.0.0.1:54324",
  "SERVICE_ROLE_KEY": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "STUDIO_URL": "http://127.0.0.1:54323"
}
```

PRO TIP: If `supabase start` reports *"Starting database from backup"* instead of applying migrations fresh, it has restored a previous Docker volume rather than replaying `supabase/migrations`. That volume could be stale (see the KNOWN ISSUE box above). When you are not certain the volume matches the current migrations directory, immediately follow up with `supabase db reset` — it is cheap, local-only, and guarantees correctness.

4.3.4 4. Apply every migration from a clean slate

Listing 4.6: Wipes the local database and replays every file in supabase/migrations in order

```
npx supabase db reset
```

You should see all 15 (at the time of writing) migrations apply in chronological order, each printed by filename. This is the command to reach for whenever local data drifts, whenever you pull a teammate's new migration, or whenever something in the local database looks wrong and you'd rather start over than debug it.

4.3.5 5. Point the app at the local stack

Two gitignored files hold a full credential set each — `.env.docker` (local Docker) and `.env.remote` (the hosted/production project) — and a pair of npm scripts copy whichever one you want over `.env.local`, which is the only file Next.js actually reads:

Listing 4.7: Switching which backend the dev server talks to

```
npm run env:local    # copies .env.docker -> .env.local (local Docker stack)
npm run env:remote   # copies .env.remote -> .env.local (hosted project)
```

`.env.docker` is generated once (and regenerated any time the local stack's keys change, e.g. after `supabase stop`/`start` on a fresh volume) with:

Listing 4.8: Regenerating .env.docker from the running local stack

```
npx supabase status -o env \
  --override-name api.url=NEXT_PUBLIC_SUPABASE_URL \
  --override-name auth.anon_key=NEXT_PUBLIC_SUPABASE_ANON_KEY \
  --override-name auth.service_role_key=SUPABASE_SERVICE_ROLE_KEY \
  | grep -E '^(NEXT_PUBLIC_SUPABASE_URL|NEXT_PUBLIC_SUPABASE_ANON_KEY|
    SUPABASE_SERVICE_ROLE_KEY)=' \
  > .env.docker
```

NOTE: `.env.docker` carries `SUPABASE_SERVICE_ROLE_KEY` alongside the two `NEXT_PUBLIC_*` vars — `src/lib/notifications.ts`'s `createNotification` (Chapter 15) requires it to write cross-user notifications, so local dev is incomplete without it. `.env.remote` does *not* carry one by default — the hosted project's service-role key bypasses RLS entirely (Chapter 14) and is never something to fetch or store automatically; pull it yourself from the hosted project's dashboard (Project Settings → API) if you need it there.

4.3.6 6. Run the app

Listing 4.9: Standard Next.js dev server

```
npm run dev
```

WARNING: Next.js does **not** hot-reload `NEXT_PUBLIC_*` environment variables into an already-running dev server process — they are baked in at server start. If you edit `.env.local` while `npm run dev` is already running, kill and restart it, or you will silently keep talking to whichever backend it started with.

4.4 Day-to-day commands

Command	What it does
`npx supabase start`	Start the local Docker stack (containers persist between restarts via a Docker volume).
`npx supabase stop`	Stop all containers. Data is preserved in the volume — a subsequent `start` restores it, it does not wipe it.
`npx supabase db reset` (or `npm run db:reset`)	Drop and recreate the local database, then replay every migration from scratch. Schema-only — wipes local data; use when iterating quickly on a migration and you don't need real data yet.
`npm run db:reset-full`	The same reset, followed automatically by `db:pull-data` (§4.8). The one to reach for by default — schema *and* real data, in one command, never stale.
`npx supabase status`	Reprint the local credentials block without restarting anything.
`npx supabase migration new <name>`	Scaffold a new, correctly-timestamped empty migration file (see Chapter 6 — never hand-create migration files).
`npx supabase migration list`	Compare local migration files against the linked hosted project's applied history — the first check to run when confirming local and cloud are in sync.
`npx supabase db query "<sql>" -linked`	Run a read-only SQL query against the linked hosted project (e.g. row counts) without needing its database password.
`npm run env:local` / `npm run env:remote`	Point `.env.local` at the local Docker stack or the hosted project, respectively (restart `npm run dev` afterward).
`npm run db:push`	`supabase db push` — applies any migration that exists locally but not yet on the linked hosted project.
`npm run db:pull-data`	Copies *data* (not schema) from the hosted project into the local database — see below.

4.5 The intended workflow: develop locally, push when it works

The point of all of the above is a specific loop: write and test migrations and features entirely against the local Docker stack, where mistakes are free (`db reset` throws it all away and replays from scratch in seconds) — then, once satisfied, run `npm run db:push` to apply the same migrations to the hosted project. Never develop directly against hosted data; never run `db reset` against a linked remote project (it has no `-linked` form for exactly this reason — `reset` is a local-only concept).

KNOWN ISSUE: This chapter's own example: four migrations were written and fully tested against local Docker in one sitting, but the person testing them ran `supabase db reset` to apply them — without registering that the *local* database also held real, hand-entered test data (an account, some books) that `db reset` silently discards, since a reset's entire contract is "recreate the database, then replay migrations," never "preserve what was there." The migrations themselves were correct and the hosted project was never touched, but local testing data was gone. **Lesson: `db reset` is safe for schema, never for data you haven't already pushed or don't have a copy of** — see the data-pull workflow immediately below for how this was recovered.

4.6 Verifying local and cloud are actually in sync

Pushing migrations is the easy half; knowing for certain that local and the hosted project agree — both in *schema* (every migration applied on both sides) and in *data* (same rows, not just the same table shapes) — is the half people skip until something breaks. Two checks, run together, cover both.

4.6.1 Schema: do both sides agree on which migrations have run?

Listing 4.10: Compares local migration files against the hosted project's applied history

```
npx supabase migration list
```

This prints every migration timestamp the CLI knows about in two columns, `local` and `remote`, side by side. Every row should show the same timestamp in both columns. A row with a `local` value and a blank `remote` (or vice versa) means exactly one side has applied a migration the other hasn't — normally a not-yet-pushed local migration, occasionally a migration that was applied by hand against the hosted project outside the normal `db push` flow (don't do this).

4.6.2 Data: do both sides hold the same rows?

A clean `migration list` only proves the two schemas are structurally identical — it says nothing about the actual rows in either database. `npx supabase db query "<sql>" -linked` runs a read-only query against the hosted project without ever needing its database password yourself (the CLI authenticates using your existing `supabase login` session); pair it with the same query against the local Docker Postgres directly to compare:

Listing 4.11: Row-count comparison — repeat per table, or script over all of them

```
# Local
docker exec supabase_db_olk psql -U postgres -d postgres -t \
  -c "select count(*) from profiles;"

# Hosted
npx supabase db query "select count(*) from profiles;" --linked
```

PRO TIP: For a full sweep instead of one table at a time, build a single `UNION ALL` query over every table in `pg_tables where schemaname = 'public'` and run it both places — one side-by-side diff instead of 24 separate commands. Don't forget `auth.users` itself; it lives outside `public` and the two sides can drift there even when every `public` table matches (e.g. a signup made against local that was never pushed anywhere, per the `db reset` gotcha above).

NOTE: This is a read-only sanity check, not a repair tool. If it finds drift, the fix is `npm run db:push` for schema gaps, or `scripts/pull-remote-data.sh` (next section) for data gaps — never hand-editing either database to force agreement.

4.6.3 When "cloud" becomes "our own server"

Nothing about either check is Supabase-Cloud-specific — both `supabase migration list` and `supabase db query -linked` work against any Postgres the project is linked to via `supabase link`, hosted or self-hosted. When OLK eventually moves off Supabase Cloud onto a self-hosted stack (Chapter 1), the same two commands, pointed at the new project ref, are how you'll confirm a migration tested locally landed correctly there too — this verification workflow is not something to relearn later.

4.7 Pulling the hosted project's data into local

A fresh `db reset` gives you correct *schema* with zero rows — no test account, no books, nothing to click through. `npm run db:pull-data` (`scripts/pull-remote-data.sh`) closes that gap by copying real data down from the linked hosted project:

Listing 4.12: scripts/pull-remote-data.sh — the whole script

```
npx supabase db dump --data-only --linked --schema public,auth \
  -f supabase/.temp/remote_data.sql
docker exec -i supabase_db_olk psql -U postgres -d postgres \
  < supabase/.temp/remote_data.sql
```

KNOWN ISSUE: The obvious version of that second line, `npx supabase db query -local -f <file>`, fails on this exact dump with `cannot insert multiple commands into a prepared statement` — that command wraps the whole file in a single prepared statement, and `pg_dump`'s output is dozens of separate statements. Piping the file straight into `psql` inside the `supabase_db_olk` container (Postgres's own client, with no such restriction) is what actually works; this is what the script has done since.

Two things worth understanding about what this does and doesn't do:

- **It really does restore logins.** The `auth` schema (users, identities, hashed passwords, sessions) is included in the dump, so accounts that exist on the hosted project become usable locally with the same email/password — GoTrue's password hashing is identical in both environments. Old session/refresh-token rows copied along with it won't actually validate

locally (they were signed with the hosted project's JWT secret, not the local stack's), which just means those specific rows are inert — logging in fresh works normally and issues new, valid local tokens.

- **Expect, and ignore, "duplicate key" errors on `areas`, `genres`, `notification_templates`, and `platform_settings`.** These tables are seeded by the migrations themselves (Chapter 12), so a fresh `db reset` already populated them before the data pull runs. Since `pg_dump` emits one multi-row `INSERT` per table, a single colliding row aborts that whole statement — but if the migration-seeded rows are identical to the hosted ones (the normal case, since the same migration created both), nothing was actually missing; verify with a row count if in doubt, don't assume from the error alone.

> **WARNING**: Storage *objects* (uploaded cover images) are not part of this data pull — only `public`/`auth` schema rows are copied, not files sitting in Supabase Storage's blob backend. In practice this rarely matters for viewing existing content: `cover_url` values are stored as full `https://*.supabase.co/storage/v1/object/public/...` URLs (Chapter 12), so a browser fetches them directly from the hosted project regardless of which database the app is talking to. It only becomes a real gap for *new* uploads made against the local stack, which land in local Storage and would need a separate, manual copy to appear elsewhere.

4.8 The daily-safe protocol: resetting without losing local data

The `db reset` gotcha earlier in this chapter (§4.5, three sections up) is not a one-time story — it is the single most common way a productive local session ends with an empty homepage: you reset to test a migration, the reset does exactly what it promises (wipe, then replay migrations), and the test account and books you'd been clicking through all afternoon are gone. Recovering them meant remembering a second, separate command afterward. This section turns that two-step, easy-to-forget manual recovery into one command that cannot be forgotten because there is nothing left to remember.

4.8.1 The tempting wrong answer: a committed `supabase/seed.sql`

The Supabase CLI has a built-in answer to "give me data back after every reset": drop a `supabase/seed.sql` file in the project, and `db reset` runs it automatically, every time, after replaying migrations — no second command at all. For most Supabase projects this is exactly the right tool.

> **WARNING**: It is the *wrong* tool here, specifically because of what §4.7's data-pull step above actually copies. `scripts/pull-remote-data.sh` dumps `-schema public,auth` — not just `public.profiles`, but `auth.users` itself: real email addresses and real bcrypt password hashes for every account that has ever registered. A `supabase/seed.sql` generated from that dump and committed to git would put live users' credentials into version-control history permanently — recoverable by anyone with repo access, on every clone, forever, even if the file is later deleted (git does not forget). `supabase/.temp/` — where today's dump actually lands — is git-ignored for exactly this reason (check `.gitignore` yourself if in doubt). Never move a real data dump out of `.temp/` and into a committed path.

The other reason a static snapshot is wrong for OLK specifically: the hosted project gets written

to from more than one machine, so any committed snapshot is stale the moment a teammate adds a book from a different laptop. A file that is both a security liability *and* perpetually out of date is worth ruling out explicitly, once, rather than reinventing later.

4.8.2 The actual answer: chain the two commands that already exist

Nothing new needs to be built — `db reset` and `db:pull-data` both already exist and both already work; the fix is making the second one impossible to forget by making it not a separate step:

→ HOW TO CHANGE THIS

`package.json`'s `scripts` block gains one line, chaining the two commands you would otherwise have to remember to run in sequence:

Listing 4.13: package.json — the new script, alongside the two it reuses

```
"db:reset": "npx supabase db reset",
"db:reset-full": "npx supabase db reset && npm run db:pull-data",
"db:push": "npx supabase db push",
```

Then run the one command instead of two:

Listing 4.14: The whole daily-safe reset, start to finish

```
npm run db:reset-full
```

What happens, in order, and why it's safe to walk away from mid-run:

1. `npx supabase db reset` — wipes the local Postgres volume and replays every file in `supabase/migrations` from scratch (Step 4 above). This is the part that discards local data; it is unavoidable and correct, since verifying a migration applies cleanly *from zero* is the entire point of testing locally before `db:push`.
2. `&&` — the second command only runs if the first exited successfully. A migration that fails to apply leaves the database in a known-bad state; silently loading real user data on top of that would hide the actual failure. No data pull happens until the schema is confirmed clean.
3. `npm run db:pull-data` — dumps `public`/`auth` from the linked hosted project and loads it into the now-freshly-migrated local database (the section immediately above this one). The four "duplicate key" errors on `areas`/`genres`/`notification_templates`/`platform_settings` are expected here too, for the same reason: those tables were already seeded by step 1.

The net effect: one command, and local ends up with correct schema *and* current real data, pulled live from whichever machine most recently touched the hosted project — never stale, never committed anywhere, because it only ever exists transiently in the git-ignored `supabase/.temp/` directory and inside your local Docker volume.

PRO TIP: Reach for plain `npm run db:reset` instead, without the data pull, when you're mid-loop rapidly iterating on a single migration file and resetting many times in a row — the data pull needs a live connection to the hosted project and takes several seconds each

time, which adds up across a dozen quick resets. Switch to `db:reset-full` once for the final check before you move on, or any time you actually want something to click through in the browser.

WARNING: `db:reset-full` is exactly as local-only as the plain `db reset` it wraps (§4.6.3's "when cloud becomes our own server" note applies here word for word). The day OLK's own machine is what `supabase link` points at instead of Supabase Cloud, that machine *is* production — running `db:reset-full`, or even plain `db reset`, against it would wipe every real user's account and every real book listing with no undo. At that point, schema changes only ever move forward via `migration new` + `db push` (§4.5's workflow, unchanged), and this whole section stops applying to that machine entirely — it remains exactly what it is today: a *local* developer-machine command, run against a *local* Docker volume, never against whatever `supabase link` currently points at in production.

4.9 Useful local endpoints

Service	URL
App (Next.js)	`http://localhost:3000`
Supabase API gateway	`http://127.0.0.1:54321`
Studio (visual table/SQL editor)	`http://127.0.0.1:54323`
Mailpit (catches every auth/verification email sent locally)	`http://127.0.0.1:54324`
Postgres (for any SQL client)	`postgresql://postgres:postgres@127.0.0.1:54322/postgres`

INTERN EXERCISE

Start the stack from a clean slate (`supabase start` then `supabase db reset`), then open Studio at `http://127.0.0.1:54323` and confirm you can see all 24 tables listed in Chapter 12. Then register a brand-new account through the running app at `localhost:3000/register`, open Mailpit at `:54324`, and find the confirmation email that was "sent" — this is how you will test any auth-related change without needing a real inbox.

4.10 Why this matters beyond convenience

As covered in Chapter 1, this is not just a developer-experience nicety. Supabase is open source and self-hostable; the exact same Docker images this chapter runs on a laptop are what would eventually run on a physical server in Kashmir. Every hour spent comfortable with `supabase start` / `stop` / `db reset` is time spent rehearsing the operational muscle memory that self-hosting will require later — this chapter is deliberately not "throwaway."

CHAPTER 5

Project Structure & File-by-File Reference

5.1 Top-level layout

Listing 5.1: Repository root (trimmed of build artefacts, node_modules, and dotfiles)

```
olk/
|-- src/
|   |-- app/                  Next.js App Router: every route lives here
|   |-- components/           Shared React components, used across routes
|   |-- hooks/                Small reusable client hooks (see below)
|   |-- lib/                  Server actions, Supabase clients, small utilities
|   `-- proxy.ts               The Proxy layer (see Chapter 7)
|-- supabase/
|   |-- migrations/           Chronological, hand-numbered SQL migration files
|   |-- schema.sql            A point-in-time full schema dump (can go stale -- see Ch.12)
|   `-- config.toml           Local Supabase CLI project config (project_id = "olk")
|-- .github/workflows/ci.yml  Lint + test on push/PR (see Ch.6)
|-- public/                   Static assets served as-is
|-- Guide/                    This manual
|-- vercel.json               Cron schedule for the weekly digest (see Ch.19)
|-- vitest.config.ts          Test runner config (see Ch.6)
|-- package.json / tsconfig.json / next.config.ts / eslint.config.mjs / postcss.config.mjs
|-- .env.local                Active environment (gitignored)
`-- README.md / AGENTS.md / CLAUDE.md
```

> **NOTE**: `AGENTS.md` (pulled in by `CLAUDE.md`) is read by AI coding assistants working in this repository and carries one instruction: this Next.js version has real breaking changes from what most training data assumes, so check `node_modules/next/dist/docs/` before writing framework-adjacent code. It is short, but do not skip it — Chapter 7's entire subject exists because of exactly this kind of drift.

5.2 `src/app` — every route in the application

Route groups in parentheses, e.g. `(dashboard)`, do not appear in the URL — they exist purely to share a layout without adding a path segment.

File	Kind	Purpose
app/layout.tsx	Server	Root HTML shell, Geist fonts (now actually the default body font), real "OLK - Open Library Kashmir" metadata.
app/page.tsx	Server	Public landing page: hero, live community stats, Book of the Month, recent activity, nearby clubs preview.
app/login/page.tsx	Client	Email/password sign-in via supabase.auth.signInWithPassword.
app/register/page.tsx	Client	Sign-up via supabase.auth.signUp; profile row is created by a database trigger, not this page.
app/maintenance/page.tsx	Server	Static page shown to non-admins when maintenance_mode is on (Chapter 7).
app/api/digest/route.ts	Route Handler	GET/POST endpoint for the weekly email digest cron (Chapter 19).
app/(dashboard)/layout.tsx	Server	Branches entirely on auth state: guests get a minimal navbar, signed-in users get the full DashboardShell.
app/(dashboard)/browse/page.tsx	Client	Book discovery: search, filters, distance radius, request/bookmark actions.
app/(dashboard)/my-books/page.tsx	Client	Add/edit/delete listings, ISBN scanner, reading-progress, "Pass It On."
app/(dashboard)/requests/page.tsx	Client	Incoming/outgoing request management, handover, return, ratings.
app/(dashboard)/messages/page.tsx	Client	Conversation list, derived from accepted requests (no dedicated conversations table).
app/(dashboard)/messages/[id]/page.tsx	Client	1:1 realtime chat scoped to one book_requests row.
app/(dashboard)/profile/page.tsx	Client	Edit display name, area, bio, geolocation, email-digest preference.
app/(dashboard)/notifications/page.tsx	Client	Full notification history with mark-as-read.
app/(dashboard)/saved/page.tsx	Client	Bookmarked books grid.
app/(dashboard)/wishlist/page.tsx	Client	Wishlist CRUD; shows matches found by the fuzzy-match RPC.
app/(dashboard)/clubs/page.tsx	Client	Club directory: search, interest filter, geo radius.
app/(dashboard)/clubs/create/page.tsx	Client	Gated club-creation form (5+ completed exchanges, no reports).
app/(dashboard)/clubs/[id]/page.tsx	Client	Club detail: posts, membership, creator-only edit/soft-delete.
app/(dashboard)/user/[id]/page.tsx	Client	Public profile view: listings, rating average, "Trusted Sharer" badge.
app/(dashboard)/admin/layout.tsx	Server	Wraps all /admin/* routes with requireAdmin('viewer').
app/(dashboard)/admin/page.tsx	Client	Overview dashboard: stats, growth charts, top contributors.
app/(dashboard)/admin/users/page.tsx	Client	User search, ban/unban/warn/reset-field/set-role.
app/(dashboard)/admin/books/page.tsx	Client	Paginated book table with hide/unhide/edit/bulk-hide.

File	Kind	Purpose
app/(dashboard)/admin/requests/page.tsx	Client	All-requests table plus an "Overdue" tab.
app/(dashboard)/admin/reports/page.tsx	Client	Report triage, quick actions, internal notes.
app/(dashboard)/admin/clubs/page.tsx	Client	Club moderation: deactivate/reactivate, remove member, transfer ownership.
app/(dashboard)/admin/content/page.tsx	Client	Tabbed CMS: announcements, genres, areas, Book of the Month.
app/(dashboard)/admin/notifications/page.tsx	Client	Broadcast / direct notifications, templates.
app/(dashboard)/admin/settings/page.tsx	Client	Feature-flag toggles, platform settings, audit log.

5.3 src/components — shared UI

File	Purpose
dashboard-shell.tsx	Responsive app shell: mobile drawer vs. desktop persistent sidebar.
dashboard-sidebar.tsx	Main nav; live active-link highlighting; sign-out.
admin/admin-nav.tsx	Horizontal admin section nav, filtered by role hierarchy.
about-modal.tsx	Static About/Vision/Team modal.
animated-counter.tsx	Count-up-on-scroll number, used for landing-page stats.
announcement-banner.tsx	Renders active banner announcements.
book-notes-modal.tsx	Per-user community notes on a book.
book-of-month.tsx	Landing-page featured book card + detail modal.
isbn-scanner.tsx	Camera barcode scan (or manual entry) + Open Library lookup + genre guessing.
notification-bell.tsx	Header bell with unread badge, dropdown, realtime updates.
rating-modal.tsx	Post-exchange 1–5 star rating.
report-modal.tsx	Report a user or book.
request-card.tsx	Shared card layout for incoming/outgoing requests.
request-stepper.tsx	Visual lifecycle stepper (lend vs. donate, declined short-circuit).
genre-select.tsx	The genre <optgroup> <select> markup, shared by My Books' add/edit forms and Browse's genre filter (Chapter 8) — promoted here from a my-books-only component once a second page needed it.
toaster.tsx	Renders the active toast queue (Chapter 9); mounted once in the dashboard layout.

> **NOTE**: `src/components/browse/`, `my-books/`, and `requests/` are three newer subdirectories not listed in the table above — they hold page-specific components extracted out of what used to be three very large single-file pages (`browse`, `my-books`, `requests/page.tsx`), not components meant to be reused across unrelated routes the way everything above is. Chapter 8 covers each of those splits where it covers the page it came from, which is the more useful place to read about them than a flat file listing here.

5.4 `src/hooks` — small reusable client hooks

A newer, small directory — before it existed, the codebase's one reusable hook (`useFeatureFlags`) lived in `src/lib` alongside plain server utilities, which is where you'll still find it (below). Everything genuinely hook-shaped added since goes here instead.

File	Purpose
`use-async-effect.ts`	`useAsyncEffect(fn, deps)` — the shared `useEffect(() => { queueMicrotask(() => fn()) }, deps)` shape (Chapter 6), used across most dashboard pages' fetch-on-mount effects.
`use-toast.ts`	`toast.error(...)` / `toast.success(...)` plus the `useToasts()` subscription hook behind `toaster.tsx` (Chapter 9).
`use-requests.ts`	All data fetching and mutations for the Requests page (Chapter 8) — `fetchRequests` plus six `handle*` mutation functions, extracted from `requests/page.tsx` into a single hook.

5.5 `src/lib` — server logic and small utilities

File	Purpose
`supabase/client.ts`	Browser Supabase client factory (`createBrowserClient`).
`supabase/server.ts`	Server Supabase client factory (`createServerClient`), cookie-aware.
`admin.ts`	`requireAdmin` / `checkAdminAPI` — the role-hierarchy gate every admin page and action goes through.
`admin-actions.ts`	Every admin/moderator Server Action: bans, warnings, book hiding, report resolution, club moderation, content management, broadcasts, settings.
`notifications.ts`	`createNotification` — writes a notification row via the service-role client (after confirming the caller is authenticated), then best-effort sends an email.
`send-notification-email.ts`	Resend integration; builds a service-role client to resolve any user's email.

File	Purpose
email-brand.ts	Shared hex-color constants for transactional email HTML (can't use CSS custom properties).
platform-settings.ts	FeatureFlags type, defaults, and parseFeatureFlags — the single source of truth proxy.ts, the dashboard layout, and client components all read platform_settings flags through.
use-feature-flags.ts	Client-side useFeatureFlags() hook, for components that need a flag (e.g. hiding the Rate/Message buttons on the requests page).
notification-icons.ts	NOTIFICATION_ICONS — the one icon map shared by notification-bell.tsx and notifications/page.tsx.
text-limits.ts	wordCount — shared by book-notes-modal.tsx and its unit tests.
geo.ts	formatDistance — turns a km float into a human string.
html-escape.ts	escapeHtml — used before interpolating user content into email HTML.
image-utils.ts	Client-side image compression to WebP before upload; remote URL validation.

5.6 Configuration files at the repository root

File	Purpose
next.config.ts	Whitelists *.supabase.co/storage/v1/object/public/** as a remote image source for next/image.
tsconfig.json	Strict TypeScript, @/* path alias to src/*.
eslint.config.mjs	Flat ESLint config built on eslint-config-next.
postcss.config.mjs	Tailwind v4 via @tailwindcss/postcss.
supabase/config.toml	Local Supabase CLI project identity and per-service port configuration.
vercel.json	Cron schedule (crons) calling /api/digest weekly (Chapter 19).
vitest.config.ts	Vitest test runner config; @/* alias mirrors tsconfig.json.
.github/workflows/ci.yml	Runs npm run lint and npm test on every push/PR (Chapter 6).
.env.local	Active Supabase URL/anon key (gitignored).

INTERN EXERCISE

Without opening any page file, predict from this table alone which pages are Server Components and which are Client Components, based only on whether they plausibly need interactivity (forms, buttons, live state). Then open five of them and check your predictions against the "use client" directive. Where you were wrong, ask yourself why — it is a fast way to build intuition for the split described in Chapter 3.

CHAPTER 6

Development Workflow & Conventions

6.1 Migrations: the one hard rule

Never hand-create a file inside `supabase/migrations`. Always use the CLI:

Listing 6.1: Correct way to start a new migration

```
npx supabase migration new add_some_feature
```

This produces a correctly-timestamped file (`YYYYMMDDHHMMSS_add_some_feature.sql`) in the exact chronological ordering the CLI expects. Migrations in this project are applied strictly in filename order — the timestamp *is* the dependency graph. A hand-created file with a wrong or duplicate timestamp can silently apply out of order, or collide with a timestamp another migration already used, and either failure mode is much harder to debug than the two seconds it costs to run the command above.

> **NOTE:** Look at the existing migration history in Chapter 12 for the house style: every non-trivial migration in this project opens with a plain-English comment block explaining *why*, not just what — see `20260701182526_rate_limit_messages_and_requests.sql` or `20260701182527_wishlist_fuzzy_match_and_unique.sql` for examples worth imitating. A migration titled only with what it does (*"add column bio"*) is worth a fraction of one that also records why it was needed.

6.2 Keeping `schema.sql` honest

`supabase/schema.sql` is a full point-in-time dump of the database, intended as a single-file, greppable reference. It is **not** automatically regenerated — it once drifted five migrations stale before being caught and regenerated (Chapter 12 tells that story). Nothing about the tooling prevents it happening again: regenerate it deliberately after a batch of migrations, with:

Listing 6.2: Regenerating the schema dump against the local database

```
npx supabase db dump --local --schema public > supabase/schema.sql
```

> **WARNING:** Do not treat `schema.sql` as the source of truth if it disagrees with `supabase/migrations` — the migrations directory always wins, because it is what actu-

ally gets applied. Chapter 12's reference tables are built from the migrations, not from the dump, for exactly this reason. CI (below) does not check whether the dump is current — regenerating it after a batch of migrations is still a manual habit, not an enforced one.

6.3 Linting and CI

Listing 6.3: Running the linter

```
npm run lint
```

The config (`eslint.config.mjs`) is Next.js's flat-config defaults (`core-web-vitals` + `typescript`), plus one project-specific override: `react-hooks/exhaustive-deps` is configured with `additionalHooks: "useAsyncEffect"` so the custom hook described in the note below gets the same missing-dependency checking as a native `useEffect` call, rather than going unchecked because its name doesn't match what the rule looks for by default. `.github/workflows/ci.yml` runs `npm run lint` and `npm test` on every push and pull request against `main`, and both pass cleanly.

NOTE: `eslint-config-next`'s React-Compiler-era rules (`react-hooks/immutability`, `react-hooks/purity`, `react-hooks/set-state-in-effect`, among others) are stricter than what most of this codebase's pages were originally written against — a large number of pages used the ordinary `useEffect(() => { loadX() }, [deps])` pattern with `loadX` declared below the effect, which several of these newer rules flag. Wiring up CI surfaced this pre-existing lint debt (roughly 70 errors across some twenty files); it has since been fixed, mechanically, file by file:

- **"accessed before declared"** (`react-hooks/immutability`) — the data-fetching function is now declared *before* the `useEffect` that calls it, everywhere. The function's own hoisting doesn't matter to this rule; only textual order does.
- **"setState synchronously within an effect"** (`react-hooks/set-state-in-effect`) — once a function is reachable from the effect (per the reordering above), the linter traces into it and flags any setState call reachable without an intervening callback boundary — including the extremely common `setLoading(true)` as an async function's first line. Wrapping the effect's call site in `queueMicrotask(() => fn())` (or, where one already existed, a `setTimeout`) breaks that trace without changing observable behavior — the callback still runs effectively immediately, before the next paint. Genuine "derive state from a changed prop" cases (`dashboard-shell.tsx`'s sidebar-close-on-navigation) were instead rewritten to adjust state directly during render, comparing against a tracked previous value — the pattern react.dev itself recommends over an effect for that specific shape.
- **"impure function during render"** (`react-hooks/purity`) — `Date.now()`/`Math.random()` calls are flagged wherever they appear *lexically inside* a component or hook body, regardless of whether that code path actually runs during render (e.g. inside an upload handler) — but *not* inside a plain function declared outside the component and merely called from within it. The fix already used elsewhere in this codebase (`requests/page.tsx`'s top-level `dueDaysLeft` helper,

now shared via `src/lib/date-utils.ts`) generalizes: extract the impure call into a small top-level helper.

- **A false-positive** `react-hooks/rules-of-hooks` fired on `admin/notifications/page.tsx`'s `useTemplate` function, because its name merely *started with* "use" — it was an ordinary function, not a hook. Renamed to `applyTemplate`.

The remaining lint output — 37 warnings, `react-hooks/exhaustive-deps` and `@next/next/no-img-element` — has since been closed out too; `npm run lint` is now silent (Chapter 22).

NOTE: The `queueMicrotask(() => fn())` fix above was applied file by file, by hand, in roughly twenty places — leaving `useEffect(() => { queueMicrotask(() => fn()) }, deps)` duplicated verbatim across every one of them. A later code-quality pass consolidated the identical seventeen of those call sites into a shared hook, `useAsyncEffect(fn, deps)` (`src/hooks/use-async-effect.ts`), that wraps exactly that pattern once. The remaining handful of call sites were deliberately *not* folded in: `browse/page.tsx` and `clubs/page.tsx` each pair a mount-only effect (now using `useAsyncEffect`) with a second, differently-shaped effect that skips its first invocation via a `mounted` ref before refetching on a specific dependency change — a distinct pattern, not the same one, so it was left as a direct `useEffect`/`queueMicrotask` call rather than forced into an abstraction that didn't quite fit. `messages/[id]/page.tsx`'s effect mixes the fetch with a realtime subscription setup in the same effect body, for the same reason.

6.4 Automated tests: a minimal but real foundation

`vitest.config.ts` configures Vitest with the same `@/*` alias as `tsconfig.json`; `npm test` runs every `src/**/*.test.ts` file. Coverage today is intentionally narrow — pure, isolable logic only (`html-escape.test.ts`, `text-limits.test.ts`, `geo.test.ts`) — not component tests, integration tests, or anything that touches Supabase. There is no Playwright or similar end-to-end runner. Until that changes, the practical discipline for anything not covered by a unit test is unchanged:

- Run `npm run dev` against the *local* Supabase stack, never the hosted one, while iterating.
- After any schema change, run `supabase db reset` and re-exercise the affected page end to end before considering the change done.
- After any RLS or admin-role change, test as more than one role — a regular user *and* at least a `moderator` account — since RLS bugs are invisible if you only ever test as a `super_admin`.
- When you add a new pure function (a formatter, a validator, a small parser), add a test file next to it rather than only exercising it by hand — that's how the three files above got there, and it's the cheapest way to grow real coverage incrementally.

6.5 Git conventions

Recent commit history is the best guide to house style — short, imperative, present-tense summaries of user-visible change (*"Redesign the requests page as clear per-exchange cards," "Add distance*

radius filter to Browse"). Match that tone: describe the outcome, not the mechanism. There is no enforced branch-naming or PR-template convention as of this edition.

INTERN EXERCISE

Create a migration with `npx supabase migration new my_test_migration` that adds a throwaway column to a table you don't care about (e.g. a `test_note text` column on `genres`), run `supabase db reset`, confirm the column exists via Studio, then write and run a second migration that drops it again. This is the full round-trip you'll use for every real schema change from here on.

PART III

The Frontend

CHAPTER 7

Next.js App Router, Routing & the Proxy Layer

7.1 App Router conventions used in this project

Every folder under `src/app` is a URL segment; a folder becomes a real page only when it contains a `page.tsx`. Three conventions are used throughout OLK:

- **Route groups** — `(dashboard)` wraps most of the app in a shared `layout.tsx` without adding `/dashboard` to any URL. `/browse` is served by `src/app/(dashboard)/browse/page.tsx`.
- **Dynamic segments** — `[id]` folders (`messages/[id]`, `clubs/[id]`, `user/[id]`) receive the segment value as a route param.
- **Nested layouts** — `admin/layout.tsx` wraps every `admin/*` page a second time, inside the outer `(dashboard)/layout.tsx`, so an admin page is rendered inside two layouts stacked: dashboard shell, then admin nav.

7.2 The Proxy layer — the rename, applied

> **NOTE:** **Next.js 16 renamed Middleware to Proxy.** From the framework's own documentation (`node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/proxy.md`):
>
> *"The `middleware` file convention is deprecated and has been renamed to `proxy`."*
>
> The convention is a `proxy.ts` file exporting a function named `proxy` (or a default export), instead of `middleware.ts` exporting `middleware`. OLK now uses `src/proxy.ts` — the body of the function is otherwise unchanged from the old `middleware.ts`, per the official migration guidance (`npx @next/codemod@canary middleware-to-proxy .`): only the filename and the exported function's name moved.

7.3 What the Proxy actually does, line by line

Listing 7.1: `src/proxy.ts` — full file

```
import { createServerClient } from '@supabase/ssr'
import type { SupabaseClient } from '@supabase/supabase-js'
import { NextResponse, type NextRequest } from 'next/server'
```

```
import { FEATURE_FLAG_KEYS, parseFeatureFlags, type FeatureFlags } from '@/lib/platform-settings'

const PROTECTED_ROUTES = ['/my-books', '/messages', '/requests', '/profile', '/user', '/saved', '/wishlist', '/clubs/create', '/admin']

let cachedFlags: { flags: FeatureFlags; expiresAt: number } | null = null
const FLAGS_TTL_MS = 30_000

async function getFeatureFlags(supabase: SupabaseClient): Promise<FeatureFlags> {
  const now = Date.now()
  if (cachedFlags && cachedFlags.expiresAt > now) return cachedFlags.flags
  const { data } = await supabase.from('platform_settings').select('key, value').in('key', FEATURE_FLAG_KEYS)
  const flags = parseFeatureFlags(data)
  cachedFlags = { flags, expiresAt: now + FLAGS_TTL_MS }
  return flags
}

export async function proxy(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() { return request.cookies.getAll() },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value))
          supabaseResponse = NextResponse.next({ request })
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    }
  )

  const { data: { user } } = await supabase.auth.getUser()
  const { pathname } = request.nextUrl
  const isProtected = PROTECTED_ROUTES.some(route => pathname.startsWith(route))
  const flags = await getFeatureFlags(supabase)

  let profile: { is_banned: boolean; ban_expires_at: string | null; is_admin: boolean; admin_role: string | null } | null = null
  if (user && (isProtected || flags.maintenance_mode)) {
    const { data } = await supabase
      .from('profiles')
      .select('is_banned, ban_expires_at, is_admin, admin_role')
      .eq('id', user.id)
      .single()
    profile = data
  }

  if (flags.maintenance_mode && pathname !== '/maintenance' && pathname !== '/login' && !profile?.is_admin) {
    const maintenanceUrl = request.nextUrl.clone()
    maintenanceUrl.pathname = '/maintenance'
    return NextResponse.redirect(maintenanceUrl)
  }

```

```
  if (!flags.feature_clubs && pathname.startsWith('/clubs')) {
    const browseUrl = request.nextUrl.clone(); browseUrl.pathname = '/browse'; return
    NextResponse.redirect(browseUrl)
  }
  if (!flags.feature_wishlists && pathname.startsWith('/wishlist')) {
    const browseUrl = request.nextUrl.clone(); browseUrl.pathname = '/browse'; return
    NextResponse.redirect(browseUrl)
  }
  if (!flags.feature_messages && pathname.startsWith('/messages')) {
    const browseUrl = request.nextUrl.clone(); browseUrl.pathname = '/browse'; return
    NextResponse.redirect(browseUrl)
  }
  // Event *creation* is nested under /clubs/[id]/events/..., not under /events,
  // so this one check needs both prefixes -- the others above only need one.
  if (!flags.feature_events && (pathname.startsWith('/events') || /^\/clubs\/[^/]+\/events\//.
    test(pathname))) {
    const browseUrl = request.nextUrl.clone(); browseUrl.pathname = '/browse'; return
    NextResponse.redirect(browseUrl)
  }

  if (!user && isProtected) {
    const loginUrl = request.nextUrl.clone()
    loginUrl.pathname = '/login'
    return NextResponse.redirect(loginUrl)
  }

  if (user && isProtected) {
    if (profile?.is_banned) {
      const expired = profile.ban_expires_at && new Date(profile.ban_expires_at) < new Date()
      if (!expired && !pathname.startsWith('/profile')) {
        const bannedUrl = request.nextUrl.clone()
        bannedUrl.pathname = '/profile'
        return NextResponse.redirect(bannedUrl)
      }
    }

    if (pathname.startsWith('/admin') && !profile?.is_admin) {
      const browseUrl = request.nextUrl.clone()
      browseUrl.pathname = '/browse'
      return NextResponse.redirect(browseUrl)
    }
  }

  return supabaseResponse
}

export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$)
    .*)'],
}
```

Five responsibilities, in the order they execute:

1. **Session refresh** — the `cookies.getAll`/`setAll` dance exists purely so `@supabase/ssr` can rotate the auth token cookies on every request. This runs unconditionally, for *every* request matched by `config.matcher` (i.e. everything except static assets), whether or not the route is protected.
2. **Feature flags** — `platform_settings` (Chapter 18) is read once per (cached, 30-second-TTL) window via `src/lib/platform-settings.ts`, shared with the `(dashboard)/layout.tsx`

server component that hides nav links for disabled features.

3. **Maintenance mode** — if `maintenance_mode` is on, every non-admin visitor (whether logged in or not) is redirected to `/maintenance`, except `/login` itself (so an admin can still sign in) and `/maintenance`.
4. **Per-feature route gates** — `feature_clubs`/`feature_wishlists`/`feature_messages`/`feature_events` being off redirects their entire route tree (`/clubs`, `/wishlist`, `/messages`, `/events` plus `/clubs/[id]/events/*`) to `/browse`, independent of whether the route is otherwise "protected."
5. **Unauthenticated, banned-user, and admin gates** — unchanged from before: redirect a session-less visitor away from `PROTECTED_ROUTES` to `/login`; for a logged-in user on a protected route, bounce a currently-banned user to `/profile`, and bounce any non-admin away from `/admin`.

NOTE: Look again at `PROTECTED_ROUTES`: it does *not* include `/browse` or `/notifications` or `/clubs` (only `/clubs/create`). This is deliberate — those pages are meant to be at least partially visible to guests — but it means any new page added under `(dashboard)` that assumes a logged-in `user` object without checking for `null` itself will crash for anonymous visitors unless it is also added to this list, or unless the page guards `auth.getUser()` being null on its own. There is no automatic enforcement tying new routes to this list — it is a manually maintained array. The feature-flag route gates above are a separate, independent set of checks — a route can be flag-gated without being in `PROTECTED_ROUTES`, and vice versa.

KNOWN ISSUE: The `feature_events` check above was missing entirely when the Events feature first shipped — `feature_clubs`, `feature_wishlists`, and `feature_messages` all had their route-gate check, but the fourth flag added alongside Events did not get the equivalent line in this file. The flag still existed, was still toggleable from `/admin/settings`, and still hid the nav link (Chapter 8) — but toggling it off left every `/events` page (and `/clubs/[id]/events/create`) fully reachable by URL, unlike every sibling feature flag. Caught by toggling the flag off in Studio and hitting `/events` directly rather than only checking the nav link disappeared. **Lesson:** a feature flag touches three independent places — the seed row, the sidebar's `NAV_ITEMS` filter, and this file's route gates — and nothing ties them together; adding one without the other two silently ships a flag that only half-works.

→ HOW TO CHANGE THIS

Adding a new page that should require login (say `/my-loans`) means touching three places, none of which reference each other:

1. `src/app/(dashboard)/my-loans/page.tsx` — the page itself, inside the `(dashboard)` route group so it inherits the shared shell.
2. `PROTECTED_ROUTES` in `src/proxy.ts` — add `'/my-loans'`, or an anonymous visitor reaches the page and your code has to handle a null `user` entirely on its own.
3. `NAV_ITEMS` in `src/components/dashboard-sidebar.tsx` — add `{ href: '/my-loans', label: '...' }`, or the page exists but nothing in the app links to it.

None of these three will error or warn if you forget one — the page will just be unprotected,

or unprotected-but-unlisted, or listed-but-crashing for guests. The habit from Chapter 17 applies here too: visit your new URL directly, logged out, before considering the page done.

7.4 The `matcher` config

The exported `config.matcher` is a single regex excluding `_next/static`, `_next/image`, `favicon.ico`, and common image extensions — meaning the Proxy runs on literally every other request, including API routes and RSC payload fetches, not just page navigations. This is why the session-refresh logic at the top has to be cheap and unconditional: it runs far more often than the redirect logic below it.

CHAPTER 8

Pages Walkthrough

Chapter 5 gave you the one-line-per-file index. This chapter goes deeper on the pages worth understanding in detail — the ones with real logic, real gotchas, or patterns you'll reuse elsewhere. Purely presentational pages (`saved`, `report-modal`-style CRUD) are not repeated here.

8.1 The landing page — `app/page.tsx`

A Server Component, and one of the few pages in the whole app that is. It fetches, in parallel, community stats via the `get_community_stats()` RPC, the active Book of the Month, a handful of recent public activity rows from `books`, and nearby active clubs — all before any HTML reaches the browser, so the landing page has real content for search engines and for users before any JavaScript runs.

Listing 8.1: Disambiguating a foreign-key embed — app/page.tsx

```
const { data: recentActivity } = await supabase
  .from('books')
  .select('id, title, cover_url, created_at, profiles!books_owner_id_fkey(display_name,
    area_name)')
  .in('status', ['available', 'given'])
  .order('created_at', { ascending: false })
  .limit(6)
```

The `profiles!books_owner_id_fkey(...)` syntax is PostgREST's way of naming *which* foreign key to embed through, when a table could otherwise be reached by more than one path. You'll see this pattern again — and its absence causing real bugs — in Chapter 16.

8.2 Browse — `app/(dashboard)/browse/page.tsx`

The main discovery surface: search, genre/type/condition/area filters, a distance radius slider, request/bookmark actions, and entry points into the notes and report modals. It calls the `get_books_nearby` RPC (Chapter 13) when the viewer has a location set, and falls back to a plain ordered query otherwise.

> **NOTE:** This page used to be a single 683-line file mixing data fetching, the search/filter bar, and the per-book grid card all in one component. A code-quality pass split the presentational pieces out: `src/components/browse/search-filters.tsx` (the search bar, filter toggle, and

all four filter controls) and `src/components/browse/book-card.tsx` (one grid item: cover, badges, owner info, and the request/bookmark/notes/report action row). `page.tsx` itself is now a 336-line orchestrator that keeps the location-aware fetching logic below and hands rendering to those two components — the sample below is unchanged and still lives directly in `page.tsx`, since search sanitisation is a data-fetching concern, not a presentational one.

Listing 8.2: Sanitising free-text search before building a PostgREST .or() filter

```
function sanitizeSearchQuery(query: string): string {
  return query.replace(/[%_\\,().]/g, '\\$&')
}
// ...
const { data } = await supabase
  .from('books')
  .select('*')
  .or(`title.ilike.%${sanitizeSearchQuery(term)}%,author.ilike.%${sanitizeSearchQuery(term)}%`
    )
```

NOTE: This is a genuinely good, easy-to-miss defensive pattern worth understanding rather than copy-pasting blindly. Supabase's `.or()` takes a small filter-expression DSL as a *string* — commas separate conditions, parentheses can nest, and `%`/`_` are `ilike` wildcards. If you interpolate raw user input into that string, a search for `"a,is_admin.eq.true"` could inject an unintended additional filter condition. Escaping the DSL's special characters before interpolating is the same category of defence as parameterising SQL — it just isn't automatic here the way it is for column value bindings, because the whole point of `.or()` is that its argument *is* a tiny query language.

8.3 My Books — `app/(dashboard)/my-books/page.tsx`

Add/edit/delete your own listings, the ISBN scanner integration (Chapter 9), and — for books currently out on loan or donation — a "books in your possession" panel with a reading-progress slider and a "Pass It On" flow that calls the `complete_donated_book_reading` RPC (Chapter 13).

NOTE: This page used to guess the id of a just-inserted book by re-querying `books` filtered by `owner_id` and the just-typed `title`, ordered by `created_at desc limit 1` — a rapid double-submission of a book with the same title could have matched the wrong row. It now uses `.insert(...).select('id').single()` and passes the returned id directly to `match_wishlists_for_book`. That code now lives in `src/components/my-books/add-book-form.tsx` rather than `page.tsx` itself — see the note below.

NOTE: This was the single largest page in the frontend before a code-quality pass split it: 905 lines covering the add-book form, the listed-books table with inline editing, and the received-books/progress panel, all in one component. It's now five files under `src/components/my-books/`: `AddBookForm`, `MyBooksList` (with its own inline-edit state), `ReceivedBooksList` (progress + "Pass It On"), and two small shared pieces, `CoverInput`

and GenreSelect — the latter promoted to src/components/genre-select.tsx once Browse's filter bar needed the same genre <select> markup that used to be duplicated verbatim between this page's add and edit forms. page.tsx itself is now a 75-line orchestrator: it owns the two top-level fetches (fetchMyBooks, fetchReceivedBooks) and passes data and refetch callbacks down.

8.4 Requests — app/(dashboard)/requests/page.tsx

Incoming and outgoing request management: accept/decline, mark handed-over, mark returned, reading-progress, "Complete Reading" for donations, and triggering the rating modal on completion. This file carries the single best piece of institutional memory in the whole frontend, left as an inline comment:

Listing 8.3: app/(dashboard)/requests/page.tsx — a bug fix explained in-place

```
if (outgoingData) {
  setOutgoingRequests(outgoingData as any)

  // profiles(display_name, area_name) above resolves via requester_id, i.e. ME --
  // useless for outgoing cards. books.owner_id has no FK to profiles (only to
  // auth.users), so PostgREST can't embed it; batch-fetch owner profiles instead.
  const ownerIds = [...new Set((outgoingData as any[]).map(r => r.books?.owner_id).filter(
    Boolean))]
  if (ownerIds.length > 0) {
    const { data: ownerProfileData } = await supabase
      .from('profiles')
      .select('id, display_name, area_name')
      .in('id', ownerIds)
    // ... merged into outgoingRequests by id
  }
}
```

The underlying fact worth internalising: books.owner_id is a foreign key to auth.users(id), *not* to public.profiles(id) (Chapter 12). PostgREST can only auto-embed a related table across an actual foreign-key edge that its schema cache knows about, and it does not reach across auth.users into public.profiles for you — even though every profiles.id *is* an auth.users.id by construction (Chapter 13's handle_new_user trigger guarantees the pairing exists). Whenever you need "the book owner's profile" rather than "the requester's profile" (whose FK to profiles does exist and embeds fine), you need a second, manual, batched query like the one above — never a per-row query in a loop.

NOTE: That comment and the fetch logic around it now live in src/hooks/use-requests.ts, not page.tsx — a code-quality pass extracted every data fetch and mutation (fetchRequests plus the six handle* functions for accept/decline/handover/return/complete-reading/progress) into a useRequests() hook, and the near-duplicated incoming/outgoing action-button JSX into src/components/requests/request-actions.tsx (one component, taking a kind: 'incoming' | 'outgoing' prop, since the two share every button except the incoming-only Accept/Decline pair) and reading-progress-footer.tsx (the outgoing-only progress slider and "Complete Reading" flow). page.tsx dropped from 522 lines to 183 and is now purely: call the hook, map two arrays through RequestCard. The

> institutional-memory comment above moved with its code, unchanged — extraction is not an excuse to lose a comment like that.

8.5 Messages — list and detail

The conversation list (`messages/page.tsx`) has no dedicated "conversations" table — it derives conversations from accepted `book_requests` rows and, for each, looks up the latest `messages` row. It does this as one batched query (`.in('request_id', allRequestIds)`) rather than one query per conversation, a deliberate N+1 avoidance worth imitating any time you're about to write a loop containing a query.

The detail page (`messages/[id]/page.tsx`) is realtime:

Listing 8.4: /page.tsx — realtime subscription with de-duplication]app/(dashboard)/messages/[id]/page.tsx — realtime subscription with de-duplication

```
const channel = supabase
  .channel(`chat:${requestId}`)
  .on(
    'postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'messages', filter: `request_id=eq.${
    requestId}` },
    (payload) => {
      const incoming = payload.new as Message
      setMessages(prev =>
        prev.some(m => m.id === incoming.id) ? prev : [...prev, incoming]
      )
    }
  )
  .subscribe()
```

The `prev.some(m => m.id === incoming.id)` guard matters because the sender's own optimistic local insert and the realtime echo of that same row arriving back over the wire are two separate code paths that both want to add the message to state — without the guard, the sender would see their own message twice.

> **NOTE**: This page also throttles "new message" notification emails to one per conversation per 15 minutes, by querying existing `notifications` rows before inserting a new one, rather than notifying on every single message. Combined with the database-level rate limit on the `messages` table itself (`trg_enforce_message_rate_limit`, Chapter 13), a chat conversation is protected against spam at two independent layers: the client throttles notification noise, the database throttles the underlying inserts.

8.6 Clubs & Events — directory, creation, and detail

Club creation (`clubs/create/page.tsx`) gates on the creator having 5 or more completed exchanges and no reports against them — computed live, client-side, from two count queries.

> **NOTE**: This eligibility check used to be enforced only in this page's React code — nothing stopped a signed-in user from calling `supabase.from('clubs').insert(...)` directly, bypassing the UI. A `BEFORE INSERT` trigger on `clubs` (`check_club_creation_eligibility`, Chapter 13) now mirrors the same 5-completed-exchanges-and-no-reports rule at the database layer, so the frontend check is now a UX nicety rather than the only gate.

Club detail (`clubs/[id]/page.tsx`) fans out a notification to every member when a new post is created, routed one at a time through the `createNotification` server action (rather than a bulk client-side `.insert(notifications[])`, now that direct cross-user notification inserts are blocked by RLS — Chapter 14). Its "delete club" is a soft-delete (`active: false`), confirmed through the shared `ConfirmModal` component like the rest of the app's destructive actions. It also lists the club's upcoming events (Chapter 17) and, for the creator, a "Create Event" link into `clubs/[id]/events/create/page.tsx`, which fans out its own `event_created` notification the same way.

The cross-club `events/page.tsx` directory and `events/[id]/page.tsx` detail page follow the same shape as their club equivalents — geo-sorted list with a plain-query fallback, RSVP as a toggleable insert/delete on `event_rsvps` — with one addition: an "Add to calendar" link straight to the `.ics` Route Handler (Chapter 15), no client-side calendar library involved.

8.7 The admin section

Chapter 18 is a full end-to-end treatment of the admin/moderation subsystem; this section is a quick map of the ten admin pages so you know where to look.

Page	What it's for
Overview	Ten queries fired in a single `Promise.all`: counts, 30-day growth (hand-rolled CSS bar charts, no charting library), exchange health, rating distribution, top areas, top contributors.
Users	Server-side search (name/area/exact id) across all profiles, not just the most recent 200, ban/unban/warn/reset-field/set-role.
Books	Paginated (50/page) table; server-side search across the full `books` table (title/author/owner name), not just the current page. hide/unhide/edit/bulk-hide.
Requests	All requests, paginated and filterable, plus a dedicated Overdue tab powered by `admin_get_overdue_books`.
Reports	Triage queue: filter, categorise, resolve/dismiss, internal notes, and "quick actions" that ban/warn/hide in one click.
Clubs	Deactivate/reactivate, remove a member, transfer ownership (via a dropdown of current club members, rather than a raw pasted user id).
Events	Deactivate/reactivate an event; view its attendee list (name + RSVP date) without needing direct database access.
Content	Tabbed CMS for announcements, genres, areas, and Book of the Month.
Notifications	Broadcast (optionally area-filtered), direct message to one user, and reusable templates.
Settings	Feature-flag toggles, platform settings key/value editor, paginated audit log.

NOTE: `admin-nav.tsx` used to treat `pathname === '/admin/overview'` as an alias for `/admin`, but `src/app/(dashboard)/admin/overview/` was an empty directory with no `page.tsx` — visiting it 404'd. The dead alias-check and the empty directory have both been removed; `/admin` is the only URL for the Overview page shown in the table above.

CHAPTER 9

Shared Components Library

Every shared file directly in `src/components` is listed in Chapter 5, along with the newer `browse/`, `my-books/`, and `requests/` subdirectories of page-specific extractions that same chapter deliberately doesn't enumerate flatly. This chapter covers the handful with logic worth understanding, not just using.

9.1 The ISBN scanner — `isbn-scanner.tsx`

Used from `my-books` to fill in title/author/cover/genre from a barcode instead of typing. It talks to no Supabase table at all — it calls the external Open Library REST API directly from the browser — and it has to cope with real-world camera API support gaps:

Listing 9.1: Feature-detecting the camera barcode API, with a graceful fallback

```
const startCamera = async () => {
  setMode('camera')
  setError('')

  if (!('BarcodeDetector' in window)) {
    setError('Barcode scanning not supported in this browser. Try entering the ISBN manually.')
    setMode('manual')
    return
  }

  try {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: { facingMode: 'environment' },
    })
    streamRef.current = stream
    if (videoRef.current) {
      videoRef.current.srcObject = stream
      await videoRef.current.play()
    }
    const detector = new (window as any).BarcodeDetector({ formats: ['ean_13', 'ean_8'] })
    // ...
```

`BarcodeDetector` is a Chrome/Edge-only Web API — it does not exist in Firefox or Safari. The component checks for it before ever requesting camera access, and drops straight into a manual-entry mode with an explanatory message if it's missing, rather than showing a broken camera view or a cryptic error.

Once an ISBN resolves via Open Library, the component has to guess a genre from Open Library's

free-text subject tags, which are noisy and inconsistent:

Listing 9.2: A "clear lead over runner-up" heuristic, not first-match

```
function guessGenre(subjects: string[]): string | null {
  const scores = GENRE_KEYWORDS.map(([genre, keywords]) => ({
    genre,
    count: subjects.filter(s => {
      const lower = s.toLowerCase()
      return keywords.some(kw => lower.includes(kw))
    }).length,
  })).sort((a, b) => b.count - a.count)

  const [top, runnerUp] = scores
  if (!top || top.count < 2 || top.count <= (runnerUp?.count ?? 0)) return null
  return top.genre
}
```

It requires at least two matching keywords *and* a strictly clear lead over the second-place genre before committing to a guess — otherwise it returns `null` and leaves the genre field for the human to fill in. This is a deliberately conservative pattern: a wrong confident guess is worse than no guess, because a wrong guess looks like data and a blank field looks like a prompt.

→ HOW TO CHANGE THIS

Adding scanner support for a genre it currently misses (say "Poetry") is one tuple in `GENRE_KEYWORDS` in `src/components/isbn-scanner.tsx` — `['Poetry', ['poetry', 'poems', 'verse']]`. Nothing else needs to change: `guessGenre` iterates the array generically, and the two-keyword-and-clear-lead rule above applies automatically to the new entry. This list is unrelated to the `genres` table (Chapter 12, admin-editable from `admin/content/page.tsx`) — that one supplies Browse's genre filter options; this one only controls what the scanner *guesses* for a newly-scanned book, and the two lists can silently drift out of sync without either one failing.

9.2 The realtime notification bell — `notification-bell.tsx`

Lives in the dashboard header on every authenticated page. It maintains its own Supabase Realtime channel independent of whatever page is currently mounted:

Listing 9.3: Instance-unique channel naming to survive Fast Refresh / multiple mounts

```
const instanceId = useRef(Math.random().toString(36).slice(2)).current
// ...
const channel = supabase
  .channel(`notif:${instanceId}:${Date.now()}`)
  .on('postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'notifications', filter: `user_id=eq.${user.
    id}` },
    (payload) => { /* prepend to local list, bump unread count */ }
  )
  .subscribe()
```

The unique channel name per mount matters in development specifically: React's Fast Refresh

can re-run an effect without a full page reload, and if two subscriptions ever shared one channel name, Supabase's realtime client would treat the second `.subscribe()` as a conflicting duplicate rather than an independent listener. An `aborted` flag guards the async setup against the classic React unmount race (component unmounts while `await`ing something, then tries to call `setState` on a component that no longer exists).

9.3 The request lifecycle stepper — `request-stepper.tsx`

A small, purely presentational component, but it encodes a real branch in the product's domain model: a *lend* and a *donate* listing have different lifecycles.

Listing 9.4: Two different step lists, and a short-circuit for the third outcome

```
const LEND_STEPS = [
  { key: 'pending', label: 'Requested' },
  { key: 'accepted', label: 'Accepted' },
  { key: 'handed_over', label: 'Handed Over' },
  { key: 'returned', label: 'Returned' },
]

const DONATE_STEPS = [
  { key: 'pending', label: 'Requested' },
  { key: 'accepted', label: 'Accepted' },
  { key: 'handed_over', label: 'Donated' },
]

export default function RequestStepper({ status, listingType }: { status: string; listingType:
    string }) {
  if (status === 'declined') {
    // renders a 2-dot "Requested -> Declined" view instead of the normal stepper
  }
  const steps = listingType === 'donate' ? DONATE_STEPS : LEND_STEPS
  const currentIndex = steps.findIndex(s => s.key === status)
  // ...
```

A *lend* has four stages because the book comes back (`returned` is a real, distinct state); a *donate* has three, because "handed over" *is* the terminal state — the book was given away, there's nothing further to track on this stepper (reading progress and eventual "Pass It On" for donations is tracked separately, see Chapter 16).

9.4 Modals: a consistent portal pattern

`about-modal.tsx`, `book-of-month.tsx`, `report-modal.tsx`, `book-notes-modal.tsx`, and `confirm-modal.tsx` all render via `createPortal(..., document.body)`, the standard React pattern for content that needs to visually escape whatever scrolling/overflow container it's logically nested inside. `rating-modal.tsx` used to use a raw `alert()` for its error states and `clubs/[id]/page.tsx`'s delete used a raw `confirm()` for its one destructive action; the former now shows an inline styled error message and the latter now goes through the shared `ConfirmModal` component.

→ HOW TO CHANGE THIS

Start a new modal from `confirm-modal.tsx` — at 46 lines, the smallest of the five — rather than from a blank file. Its whole shape is worth copying exactly: `createPortal(<jsx>, document.body)`, a fixed, full-screen, semi-transparent backdrop `<div>` whose own `onClick` closes the modal, and the card itself with `onClick=e => e.stopPropagation()` so a click *inside* the card doesn't bubble up and trigger that same close handler. Skipping `createPortal` still looks fine in most places you'd try it, but breaks the moment the modal is triggered from inside a card or list row that has its own `overflow-hidden` — the modal gets visually clipped by its logical parent instead of escaping to the top of the page.

NOTE: `rating-modal.tsx`'s and the clubs delete confirmation's `alert()`/`confirm()` instances were fixed first, in isolation, leaving the broader pattern — five more pages still using raw `alert()` for one-off error messages — flagged as separate, open debt (Chapter 22). That debt has since been closed out too, by the toast pattern below, rather than by converting each site to its own inline error message.

9.5 Toasts: a global notice, without a Context Provider

`browse/page.tsx`, `requests/page.tsx`, `my-books/page.tsx`, `messages/[id]/page.tsx`, and `clubs/create/page.tsx` used to call the browser's native `alert()` for eleven different failed-action messages between them — a rate limit hit, a duplicate request, a failed save. `alert()` blocks the entire tab on a native OS dialog until dismissed, which is jarring compared to every other error surface in the app, and it's the one UX pattern in this codebase that a screenshot or design review would never catch, because it doesn't render as part of the page.

Listing 9.5: src/hooks/use-toast.ts — module-level state, no Provider

```
type Toast = { id: number; message: string; variant: 'error' | 'success' }

let toasts: Toast[] = []
let nextId = 0
const listeners = new Set<() => void>()

function show(message: string, variant: Toast['variant']) {
  const id = nextId++
  toasts = [...toasts, { id, message, variant }]
  listeners.forEach((l) => l())
  setTimeout(() => {
    toasts = toasts.filter((t) => t.id !== id)
    listeners.forEach((l) => l())
  }, 5000)
}

export const toast = {
  error: (message: string) => show(message, 'error'),
  success: (message: string) => show(message, 'success'),
}

export function useToasts() {
  return useSyncExternalStore(
    (onChange) => { listeners.add(onChange); return () => listeners.delete(onChange) },
    () => toasts,
```

```
    () => toasts
  )
}
```

The state lives in a plain module-level variable, not `useState` or React Context — `toast.error(...)` is a function you can call from anywhere (an event handler, a `.catch()`, deep inside a nested component) without that call site needing to be a descendant of any Provider, and without prop-drilling a `showToast` callback down through component trees the way the pre-extraction pages would otherwise have needed. `useSyncExternalStore` is what makes this safe to read from a React component despite the mutation happening entirely outside React's own state system: it's the hook React itself recommends specifically for subscribing a component to external, mutable state (browser APIs, module-level variables like this one) without risking tearing under concurrent rendering — the alternative, `useState` plus a manual `useEffect` subscription, is the same idea with more boilerplate and without that safety guarantee.

A single `<Toaster />` (`src/components/toaster.tsx`) is mounted once, in the authenticated branch of `app/(dashboard)/layout.tsx`, fixed to the bottom-right corner; it subscribes via `useToasts()` and renders nothing when the list is empty. Every call site that used to say `alert('Some message')` now says `toast.error('Some message')` — a mechanical, one-line-per-callsite change, styled to match the app's existing red-for-error/teal-for-success convention rather than introducing a new color scheme.

→ HOW TO CHANGE THIS

Need a third toast variant (say `'info'`)? Add it to the `Toast['variant']` union in `use-toast.ts`, add a matching `toast.info(...)` export next to `error`/`success`, and add its background/border/text classes to the ternary in `toaster.tsx`'s `className`. Nothing else needs to change — the store, the 5-second auto-dismiss timer, and the `useSyncExternalStore` plumbing are all variant-agnostic.

CHAPTER 10

The Styling System

10.1 Tailwind CSS v4 — a CSS-first configuration

OLK uses Tailwind v4, which is configured very differently from v3: there is no `tailwind.config.js` anywhere in this repository. Instead, Tailwind is invoked as a PostCSS plugin and configured directly in CSS:

Listing 10.1: postcss.config.mjs — the entire build wiring

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};
export default config;
```

Listing 10.2: src/app/globals.css — in full

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);

  --color-brand-slate: #0f172a;
  --color-brand-slate-light: #1e293b;
  --color-brand-slate-muted: #334155;
  --color-brand-teal: #14b8a6;
  --color-brand-teal-dark: #0d9488;
  --color-brand-teal-light: #2dd4bf;
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}
```

```
28
29 body {
30   background: var(--background);
31   color: var(--foreground);
32   font-family: var(--font-sans);
33 }
```

The `@theme inline` block is Tailwind v4's mechanism for exposing CSS custom properties as Tailwind design tokens (so `bg-background` and `text-foreground` utility classes become available, backed by the `-color-background`/`-color-foreground` variables).

NOTE: The app's dark slate-and-teal identity used to exist only as repeated, ad hoc Tailwind utility classes (`bg-slate-900`, `text-teal-400`, and so on, hand-typed independently in dozens of files) and as literal hex values inside `src/lib/send-notification-email.ts`'s inline email HTML. `globals.css` now defines named `-color-brand-slate`/`-color-brand-teal` (plus `-light`/`-dark` variants) in the `@theme` block, which Tailwind v4 automatically turns into `bg-brand-teal`/`text-brand-slate`-style utilities — matching the exact hex values Tailwind's stock `slate-900`/`teal-400/500/600` already used, so nothing visually changed. The email HTML (which can't reference CSS custom properties) now pulls the same hex values from a single shared `src/lib/email-brand.ts` instead of duplicating them across `send-notification-email.ts` and `api/digest/route.ts`. Every stock `slate-700`/`slate-800`/`slate-900`/`teal-400`/`teal-500`/`teal-600` call site elsewhere in the app (563 occurrences across 39 files) has since been mechanically migrated to the matching `brand-*` utility too — a plain token substitution, since the hex values were already identical, verified by diffing the compiled CSS before and after. Any other shade (`slate-300`, `teal-700`, etc.) has no `brand-*` equivalent and was intentionally left alone.

10.2 No component library, no shared design primitives

There is no button component, no card component, no shared `<Modal>` wrapper. Every card, button, badge, and modal is written inline with Tailwind utility classes at its call site, independently, in whichever file needed it. This has a real, visible consequence covered already in Chapters 8 and 9: modal styling is *almost* entirely consistent (portal-based, dark card, teal accent), with a couple of drift points (`alert()`/`confirm()` instead of the shared visual style) that stand out precisely because the convention is otherwise so consistent.

PRO TIP: Before writing a new UI element from scratch, search the codebase for something visually similar first (a card, a badge, a status pill) and copy its Tailwind class list rather than inventing a new visual style. It is the only mechanism currently holding visual consistency together, since there is no shared component to import instead.

→ HOW TO CHANGE THIS

Swapping the brand teal (`#14b8a6`) for a different accent touches two files, because email HTML can't read CSS custom properties:

1. `src/app/globals.css` — the `-color-brand-teal`/`-dark`/`-light` lines inside `@theme inline`. Every `bg-brand-teal`/`text-brand-teal`-style utility across the app picks

this up automatically; no per-component edits needed.

2. `src/lib/email-brand.ts` — the `teal`/`tealGradientFrom`/`tealGradientTo` hex strings, consumed by `send-notification-email.ts` and `api/digest/route.ts` for transactional email, which is rendered outside Tailwind entirely.

Change only #1 and every screen in the app matches, but every notification email keeps shipping the old color.

Unlike the teal accent, the homepage's dark base tone is *not* theme-token driven — it started life as `#020817` typed directly into a handful of Tailwind arbitrary-value classes and was later lightened to `#0f172a` (Tailwind's stock `slate-900`, which happens to already be `-color-brand-slate` in `globals.css`, though the homepage still references the literal hex rather than the token).

→ HOW TO CHANGE THIS

Four call sites share the same hardcoded hex, across two files, because two of them need opacity suffixes (`/80`, `/96`) and one sits inside a `from-[...]` gradient stop — none of which read a `-color-brand-*` custom property. Find every occurrence before touching anything:

Listing 10.3: Locate every homepage background call site

```
grep -rn "0f172a\]" src/app/page.tsx src/components/about-modal.tsx
```

That returns the page wrapper, the sticky navbar's backdrop, and the final-CTA glow gradient in `src/app/page.tsx`, plus the About modal's backdrop in `src/components/about-modal.tsx`. Since all four share the identical hex, a scoped find-and-replace across just those two files is safe and sufficient:

Listing 10.4: Swap the base tone in one shot

```
sed -i 's/0f172a/1c1917/g' src/app/page.tsx src/components/about-modal.tsx
```

Then restart `npm run dev` and reload the homepage — Tailwind's JIT compiler resolves new arbitrary-value classes like `bg-[#1c1917]` on the fly, so no other build step is needed. `Guide/main.tex`'s own `olkSlate` colour (§10's preamble) is unrelated and does not need to change — it drives this PDF manual, not the website.
A few stock Tailwind shades worth trying as the base, in place of `#0f172a`:

Theme	Hex	Feel
Slate Navy (current)	`#0f172a`	Cool, neutral, current default — Tailwind `slate-900`.
Slate Black	`#020617`	Closest stock match to the original near-black — Tailwind `slate-950`.
Warm Charcoal	`#1c1917`	Neutral-warm, pairs well with the amber ambient glow added alongside the teal one — Tailwind `stone-900`.
Zinc Dark	`#18181b`	True neutral gray-black, no blue or warm cast — Tailwind `zinc-900`.
Midnight Indigo	`#1e1b4b`	Moodier, purple-leaning dark — Tailwind `indigo-950`.

For a full light theme instead of a lighter dark one, swap `bg-[#0f172a] text-white` for `bg-slate-50 text-slate-900` on the page wrapper and re-derive every secondary-text

class in the opposite direction — a lighter background needs *darker* secondary text (e.g. `text-slate-600` instead of `text-slate-300`). That is a bigger, more invasive change than the swaps above: not a one-line `sed`, since every text shade in `page.tsx`, `about-modal.tsx`, `book-of-month.tsx`, and `announcement-banner.tsx` would need to flip direction together or contrast breaks.

10.3 Fonts

`src/app/layout.tsx` loads Geist Sans and Geist Mono via `next/font/google`, exposing them as the `-font-geist-sans`/`-font-geist-mono` CSS variables consumed by the `@theme inline` block above. `globals.css`'s `body` rule references `font-family: var(-font-sans)` — which resolves to Geist — rather than hard-coding a fallback stack, so the loaded Geist font is now actually the default body font, not just an opt-in Tailwind utility.

10.4 Dark mode

The only dark-mode logic in the entire codebase is the `prefers-color-scheme: dark` media query in `globals.css`, which swaps the two CSS variables above. There is no user-facing toggle, no `class="dark"` strategy, and no persisted preference — the app simply follows the OS/browser setting. Given that almost the entire dashboard UI is hand-built with explicit dark-slate Tailwind classes (`bg-slate-900` etc.) regardless of `prefers-color-scheme`, the practical reality is that the authenticated dashboard *always* looks dark, and only the landing/login/register pages' backgrounds actually respond to the media query.

INTERN EXERCISE

Grep the codebase for `bg-slate-900` (e.g. `grep -rn "bg-slate-900" src/`) — it should return nothing, since the `brand-*` migration above covers exactly that shade. Now grep for `text-teal-300`: it *will* turn up several files. Explain why one search comes back empty and the other doesn't — the answer is in `globals.css`'s `@theme` block above, not in any special-casing in the migrated files themselves.

PART IV

The Backend

CHAPTER 11

Supabase Fundamentals

11.1 What Supabase actually is

"Supabase" is not one server — it is a bundle of independent, cooperating open-source services in front of a stock Postgres database. Chapter 4 listed them as Docker containers; conceptually, the ones OLK actually uses are:

- **Postgres** — the real database. Every table in Chapter 12 is a plain Postgres table; nothing about them is Supabase-proprietary.
- **PostgREST** — introspects the Postgres schema and turns it into a REST API automatically. When the frontend calls `supabase.from('books').select('*')`, that becomes an HTTP `GET` to PostgREST, which translates it into SQL. This is why table/column names are not statically checked anywhere (Chapter 2) — there is no generated client, just a REST convention.
- **GoTrue (Auth)** — issues and verifies JWTs, manages `auth.users` (a schema *outside* `public`, which application tables reference but never own).
- **Storage** — an S3-like object store with its own RLS-like policy system, layered on a Postgres table (`storage.objects`) under the hood. OLK's one bucket, `book-covers`, is configured this way (Chapter 12).
- **Realtime** — listens to Postgres's logical replication stream and forwards row changes to subscribed clients over WebSockets. This is what powers the chat page and the notification bell (Chapters 8, 9).

> **NOTE**: Every one of these is open source and self-hostable, which is precisely why Chapter 1 frames "get comfortable with the local Docker stack" as more than a developer-experience nicety — the same containers can run anywhere, including on a physical server in Kashmir.

11.2 Two ways application code talks to Postgres

1. **The query builder**, e.g. `supabase.from('book_requests').select('*, books(*)').eq('status', 'pending')`. Every such call is subject to Row-Level Security (Chapter 14) as whichever user's JWT is attached to the request — an anonymous visitor, a logged-in reader, or (in two specific server-only files) the service role.
2. **RPC calls to Postgres functions**, e.g. `supabase.rpc('get_books_nearby', { user_lat, user_lng })`. This is how OLK does anything the query builder can't express directly: geographic distance sorting, cross-user aggregate stats, and — critically —

anything that needs to see *other users'* rows despite RLS, via `SECURITY DEFINER` (next section). Chapter 13 is the full reference.

11.3 `SECURITY DEFINER`: the escape hatch from RLS, used deliberately

By default, a Postgres function runs with the privileges of whoever calls it (`SECURITY INVOKER`) — so a function called by an ordinary logged-in user is just as constrained by RLS as a raw query would be. Marking a function `SECURITY DEFINER` makes it run with the privileges of the function's *owner* (here, the `postgres` superuser role that created it) — meaning it can read or write rows that RLS would otherwise hide from the calling user, *but only through whatever that function's own SQL body chooses to expose.*

Nearly every function in this codebase is `SECURITY DEFINER`, and that is the correct choice for what they do: `get_community_stats()` needs to count every row in `books` and `profiles` to produce a public aggregate, even though a plain user cannot `SELECT *` across other people's data; `match_wishlists_for_book` needs to search across *every* user's wishlist rows to find a fuzzy match, even though the `wishlists` RLS policy restricts a normal query to your own rows only.

WARNING: `SECURITY DEFINER` is exactly as dangerous as it is useful: a bug in a `SECURITY DEFINER` function's SQL body is a bug that runs with elevated privilege, bypassing the RLS policies that would otherwise contain it. Every such function in this codebase is deliberately narrow — it returns only specific, chosen columns (never `SELECT *` on a sensitive table), and several (`match_wishlists_for_book`, the `admin_get_*` analytics functions) explicitly `REVOKE`/`GRANT` execute permission rather than relying on the default. If you add a new `SECURITY DEFINER` function, that same discipline — narrow return columns, explicit grants — is not optional.

→ HOW TO CHANGE THIS

Building a new feature and unsure whether it needs a query-builder call or a new RPC? Ask, in order:

1. Does a plain `.from(...).select(...)` under the *current user's own* RLS policies return everything you need? If yes, use the query builder — it's simpler, and every future reader sees exactly what's being fetched without also opening a separate SQL file.
2. If not: is the gap "I need to see rows RLS would normally hide from this user" (a cross-user aggregate, a fuzzy match across everyone's data)? That's a `SECURITY DEFINER` RPC — but keep its return columns as narrow as the feature actually needs, per the warning above, never `SELECT *`.
3. If the gap is instead "this needs to happen atomically, or enforce a rule no client should be able to bypass" (a rate limit, an eligibility check) — that's a trigger the database enforces on every write, not an RPC the client has to remember to call. Chapter 13 covers both patterns side by side.

11.4 `auth.users` vs. `public.profiles`

Supabase Auth owns a schema called `auth`, and `auth.users` is its user table — email, hashed password, confirmation status, and so on. Application code is not supposed to touch `auth.users` directly (and mostly can't, through the query builder — RLS on `auth` tables is Supabase's own concern, not this project's). Instead, OLK's own `public.profiles` table holds everything the app actually needs to display or query (display name, area, bio, admin role, ban state), with `profiles.id` equal to `auth.users.id`.

The two are kept in lockstep by a single trigger, fired the moment GoTrue creates a new `auth.users` row:

Listing 11.1: The trigger that makes signup "just work" (see supabase/migrations/20260621123649...)

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
  INSERT INTO public.profiles (id, display_name)
  VALUES (NEW.id, NEW.email);
  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

This is why `app/register/page.tsx` (Chapter 8) never explicitly creates a `profiles` row itself — by the time `supabase.auth.signUp()` resolves, this trigger has already run, and a matching profile (with `display_name` defaulted to the user's email) already exists.

> **WARNING**: Almost every foreign key in this schema points at `public.profiles(id)` — with one deliberate, consequential exception: `books.owner_id` points at `auth.users(id)` directly. Chapter 8's "Requests" section and Chapter 12 both cover the concrete consequence: PostgREST cannot auto-embed `profiles` through `books.owner_id`, because there is no foreign-key edge from `books` to `profiles` for it to walk — only `books` → `auth.users`, a schema the query builder cannot embed into at all. Any code that needs "the book owner's profile" has to fetch it as a second, separate, batched query.

CHAPTER 12

Database Schema Reference

> **NOTE:** `supabase/schema.sql` is a point-in-time dump, not a live view — it used to be five migrations stale (missing rate limiting, wishlist fuzzy matching, book description/year, and the profile `bio` column) before being regenerated with `npx supabase db dump -local -schema public`. **This chapter's tables reflect the true current schema**, built from `supabase/migrations` in full, per the house rule in Chapter 6: the migrations directory is always the authoritative source, never the dump — treat `schema.sql` as a convenience snapshot that can drift again the moment a migration lands without a follow-up regeneration.

All 26 tables live in the `public` schema and have Row-Level Security enabled; policies are covered separately in Chapter 14 so this chapter can focus purely on shape. Every primary key is a `uuid` defaulting to `gen_random_uuid()` unless noted otherwise.

12.1 Identity

`profiles`

One row per `auth.users` row, created automatically by the `handle_new_user` trigger (Chapter 11). This is the table almost everything else in the schema references.

Column	Type	Notes
`id`	uuid, PK	= `auth.users(id)`, `ONDELETECASCADE`
`display_name`	varchar(50)	
`area_name`	varchar(100)	Free-text area label shown alongside `latitude`/`longitude`
`latitude`, `longitude`	double precision	Feed `get_books_nearby`/`get_clubs_nearby`
`bio`	text	CHECK length ≤ 300
`email_digest`	boolean, default true	Opt-out flag for the weekly digest (Chapter 19)
`is_admin`	boolean, default false	Must be true *and* `admin_role` set for any admin RLS policy to apply
`admin_role`	`admin_role` enum	`super_admin` / `moderator` / `viewer` — see Chapter 14
`is_banned`, `ban_reason`, `ban_expires_at`	boolean / text / timestamptz	Checked by the Proxy layer (Chapter 7) on every protected-route request

Column	Type	Notes
warning_count	integer, default 0	Kept in sync by warnUser (Chapter 18)
last_active_at, created_at, updated_at	timestamptz	

12.2 Books & the Exchange Lifecycle

books

Column	Type	Notes
owner_id	uuid	FK → auth.users(id) — **not** profiles (see the warning in Chapter 11)
title, author	varchar(500)	
condition	varchar(20)	CHECK IN excellent/good/fair/poor
listing_type	varchar(10), NOT NULL	CHECK IN donate/lend
status	varchar(20), default available	CHECK IN available/unavailable/given
genre	varchar(100)	
cover_url	text	Points into the book-covers Storage bucket
lending_duration_months	smallint	CHECK IN (1,2,3) — only meaningful when listing_type='lend'
acquired_via_donation	boolean, default false	Set true by complete_donated_book_reading — locks the book into perpetual donation circulation
read_count	integer, default 0	Auto-incremented by trg_increment_read_count
description, publication_year	text, smallint	*Added in the two most recent-but-one migrations, populated by ISBN-scan enrichment; CHECK 1000–2200 on the year.*
hidden_by_admin, admin_hide_reason, hidden_at	boolean / text / timestamptz	Set by hideBook (Chapter 18)

→ HOW TO CHANGE THIS

Adding a fifth condition value (say 'like_new') means editing the same fact in three places, none of which derive from the others:

1. The books_condition_check CHECK constraint — ALTER TABLE books DROP CONSTRAINT books_condition_check, ADD CONSTRAINT books_condition_check CHECK (condition IN (...)) inside a new migration (Chapter 6).

2. src/app/(dashboard)/my-books/page.tsx — the <option value="excellent"> list in the listing form.
3. src/app/(dashboard)/browse/page.tsx — the identical, independently-typed <option> list in the filter dropdown.

Miss #1 and both frontend forms happily offer a value the database will reject with a constraint violation on submit; miss #2 or #3 and the new value is a valid row you can only ever create by hand in Studio. listing_type (books_listing_type_check, donate/lend) is the same three-location shape if you ever need to extend that instead.

book_requests

The spine of the exchange lifecycle (Chapter 16).

Column	Type	Notes
book_id	uuid	FK → books(id)
requester_id	uuid	FK → profiles(id)
status	varchar(20), default pending	CHECK IN pending/accepted/declined/handed_over/returned
created_at, handed_over_at, completed_at	timestamptz	

A partial unique index, idx_unique_active_request(book_id, requester_id) WHERE status IN (pending, accepted, handed_over), is the only thing stopping the same user from opening a second concurrent request for a book they already have an active request on — note it does *not* stop two different users from having simultaneous pending requests on the same book (the owner is expected to pick one and decline the rest).

book_progress

One row per request, tracking a reader's percentage through a book. book_id → books(id); request_id → book_requests(id), UNIQUE; reader_id → profiles(id); progress_pct smallint CHECK 0–100. Publicly viewable (community visibility of "who's reading what"), writable only by the reader themself.

12.3 Community Features

messages

request_id → book_requests(id); sender_id → profiles(id); content text. Member of the supabase_realtime publication — this is the table the chat page (Chapter 8) subscribes to. Guarded by trg_enforce_message_rate_limit (Chapter 13).

bookmarks

user_id + book_id, UNIQUE pair — a simple save-for-later join table, fully self-service.

ratings

request_id → book_requests(id); rater_id, rated_user_id → profiles(id); score smallint CHECK 1–5; comment text. UNIQUE(request_id, rater_id) is the constraint rating-modal.tsx (Chapter 9) relies on to detect "already rated" via Postgres error code 23505 rather than a pre-check query.

wishlists

user_id → profiles(id); title, author, genre; matched_book_id → books(id) ON DELETE SET NULL. As of the most recent-but-one migration: a unique index on (user_id, lower(btrim(title))) prevents duplicate active wishes for the same title, and a pg_trgm GIN index on title backs the fuzzy-match RPC (Chapter 13).

book_notes

book_id → books(id); user_id → profiles(id); note text; UNIQUE(book_id, user_id) — one editable note per user per book, publicly readable.

clubs, club_members, club_posts

clubs: name, description, interest, area_name, latitude/longitude, cover_url, creator_id → profiles(id), member_count (int, kept in sync by trg_club_member_count — Chapter 13), active boolean (used for soft-delete). club_members: club_id + user_id, UNIQUE pair, joined_at. club_posts: club_id, author_id, content, created_at — creator-only to post, author-only to delete, no UPDATE policy at all (posts are immutable once made).

club_events, event_rsvps

club_events: club_id → clubs(id), creator_id → profiles(id), title, description, cover_url, starts_at (NOT NULL) / ends_at, is_online boolean, location_name / meeting_url, latitude/longitude, visibility (CHECK IN public/members_only — gates RSVP, not visibility of the event itself, Chapter 17), capacity, attendee_count (kept in sync by trg_event_attendee_count, the same shape as trg_club_member_count), active boolean. event_rsvps: event_id + user_id, UNIQUE pair, created_at — no status column; an RSVP is a row, cancelling one is a DELETE.

12.4 Notifications & Content

notifications

user_id → profiles(id); type, title, body, link text; read boolean. Indexed (user_id, created_at DESC) for the bell/list views. See Chapter 14 for a real RLS looseness on this table's

INSERT policy.

notification_templates

Admin-only reusable message templates (name UNIQUE, title, body, type). Seeded with 7 defaults (welcome, warnings, bans, unban, book drive) by the admin-comprehensive migration.

announcements

title, body, type (CHECK info/warning/success/event), is_banner boolean, active, starts_at/ends_at. The "one active banner" invariant is enforced in application code (createAnnouncement unsets any existing banner first, Chapter 18), not by a database constraint.

genres, areas

Simple admin-managed lookup tables (name UNIQUE, active boolean). Seeded with 19 genres and 29 real Kashmir-region areas across Srinagar, Anantnag, Baramulla, Kupwara, Budgam, Pulwama, Shopian, Kulgam, Ganderbal, and Bandipora — a concrete reflection of the project's regional scope (Chapter 1).

book_of_month

title, author, description, cover_url, month_label, active. saveBotm (Chapter 18) deactivates any existing entry before inserting a new one, the same single-active-row pattern as announcements.

platform_settings

key text **PK** (not a uuid — this table is a key/value store), value jsonb, description, updated_at/updated_by. Seeded keys: max_books_per_user (50), max_requests_per_day (10), max_messages_per_hour (30), overdue_days_threshold (30), maintenance_mode, and five feature_* flags (clubs/wishlists/ratings/messages/events). Publicly readable (anyone can SELECT), writable only by super_admin.

12.5 Admin & Moderation

reports

reporter_id → profiles(id); reported_user_id/reported_book_id (either may be null, ON DELETE SET NULL); reason, details, status (default pending), category (default other), assigned_to, resolved_at/resolved_by.

user_bans, user_warnings

Append-only history tables behind the profiles.is_banned/warning_count summary fields. user_bans: is_permanent boolean, expires_at, unbanned_at/unbanned_by — a ban is never

deleted, only marked unbanned.

admin_audit_log

admin_id, action text, target_type text, target_id uuid (nullable), details jsonb. Every single Server Action in admin-actions.ts (Chapter 18) writes exactly one row here — this table is the full history of every moderation action ever taken, by whom. No UPDATE or DELETE policy exists for any role: the log is genuinely immutable once written.

admin_notes

report_id → reports(id); admin_id; content text — free-text internal notes moderators leave on a report, visible only to other admins/moderators.

INTERN EXERCISE

Using only this chapter (no source code), sketch the foreign-key graph for the exchange lifecycle: books → book_requests → book_progress, plus everything that references book_requests.id (there are three). Then open Studio's table view (Chapter 4) and check your sketch against the real foreign keys shown there.

CHAPTER 13

Functions, Triggers & RPCs

Every function below lives in the `public` schema and is called from the frontend via `supabase.rpc('function_name', {...})` unless noted as trigger-only. Recall from Chapter 11 that almost all of them are `SECURITY DEFINER` — that is called out per-function only where it is the whole reason the function exists.

13.1 Identity & role helpers

Listing 13.1: The three functions every admin RLS policy is built from (supabase/schema.sql)

```sql
CREATE OR REPLACE FUNCTION public.is_admin() RETURNS boolean
LANGUAGE sql STABLE SECURITY DEFINER AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.profiles
    WHERE id = auth.uid() AND is_admin = true
  );
$$;

CREATE OR REPLACE FUNCTION public.is_admin_or_mod() RETURNS boolean
LANGUAGE sql STABLE SECURITY DEFINER AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.profiles
    WHERE id = auth.uid() AND is_admin = true
      AND admin_role IN ('super_admin', 'moderator')
  );
$$;
-- is_super_admin() follows the same shape, requiring admin_role = 'super_admin' exactly.
```

These three are why RLS policies throughout Chapter 14 can be as short as `USING (public.is_admin_or_mod())` — the role hierarchy check lives in exactly one place, not duplicated across every policy that needs it.

`handle_new_user()` was already covered in full in Chapter 11 — it is the `AFTER INSERT ON auth.users` trigger that provisions a matching `profiles` row for every new signup.

13.2 Discovery: geography

Listing 13.2: get_books_nearby — haversine distance, nulls last

```sql
CREATE OR REPLACE FUNCTION public.get_books_nearby(user_lat double precision, user_lng double
    precision)
RETURNS TABLE(id uuid, title varchar, author varchar, condition varchar,
              listing_type varchar, status varchar, genre varchar, cover_url text,
              owner_id uuid, created_at timestamptz, distance_km double precision,
              owner_name varchar, owner_area varchar, read_count int,
              acquired_via_donation boolean)
LANGUAGE sql STABLE SECURITY DEFINER AS $$
  SELECT
    b.id, b.title, b.author, b.condition, b.listing_type, b.status,
    b.genre, b.cover_url, b.owner_id, b.created_at,
    CASE
      WHEN p.latitude IS NOT NULL AND p.longitude IS NOT NULL THEN
        6371 * acos(
          LEAST(1.0, GREATEST(-1.0,
            cos(radians(user_lat)) * cos(radians(p.latitude)) *
            cos(radians(p.longitude) - radians(user_lng)) +
            sin(radians(user_lat)) * sin(radians(p.latitude))
          ))
        )
      ELSE NULL
    END AS distance_km,
    p.display_name AS owner_name, p.area_name AS owner_area,
    b.read_count, b.acquired_via_donation
  FROM books b
  JOIN profiles p ON p.id = b.owner_id
  WHERE b.status IN ('available', 'given', 'unavailable')
  ORDER BY distance_km ASC NULLS LAST, b.created_at DESC;
$$;
```

NOTE: The `6371 * acos(...)` expression is the haversine great-circle-distance formula (6371 = Earth's radius in km). `LEAST(1.0, GREATEST(-1.0, ...))` clamps the argument to `acos` into $[-1, 1]$ — without it, floating-point rounding on two nearly-identical coordinates (e.g. a user browsing their own listing) can push the argument fractionally outside that range and make `acos` return `NaN`, since `acos` is only defined on $[-1, 1]$.

`get_clubs_nearby(user_lat, user_lng)` is the same haversine pattern applied to `clubs` (only `active = true`), joined to the creator's profile, ordered by distance then by `member_count DESC`. `get_events_nearby(user_lat, user_lng)` is the same pattern again, one table further out: `club_events` (only `active = true`) joined to both its parent `clubs` row and the creator's profile, ordered by distance then by `starts_at ASC` — soonest first, rather than most-popular first, since "what's happening near me next" is the more useful default for events than it is for clubs.

KNOWN ISSUE: Both nearby functions have an explicit column list in `RETURNS TABLE(...)`. When `books.description` and `books.publication_year` were added, an earlier version of this function briefly shipped *without* them in its return type — silently dropping those two columns from every geo-sorted browse result for any user with a location set, even though the columns existed in the table. A follow-up migration (`20260702184016_...`) had to `DROP` and re-`CREATE` the function to add them. **Lesson for the next schema change that touches `books` or `clubs`:** adding a column to the underlying table does *not* automatically surface it through either nearby function — you must also update its `RETURNS TABLE` list and redeploy the function.

13.3 Public aggregates

`get_community_stats()` returns `(total_books, total_users, completed_exchanges)` as a single-row aggregate for the landing page. It is `SECURITY DEFINER` specifically so it can `COUNT(*)` across every user's rows despite RLS, while the function's return type guarantees it can never leak anything beyond those three numbers — a clean example of the "narrow, deliberate escape hatch" pattern from Chapter 11.

13.4 The exchange lifecycle: completion & read-count

Listing 13.3: complete_donated_book_reading — ownership transfer with an authorization check inline

```
CREATE OR REPLACE FUNCTION public.complete_donated_book_reading(p_request_id uuid)
RETURNS void LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE v_book_id uuid; v_owner uuid; v_listing_type varchar;
BEGIN
  SELECT br.book_id, b.owner_id, b.listing_type
    INTO v_book_id, v_owner, v_listing_type
  FROM book_requests br JOIN books b ON b.id = br.book_id
  WHERE br.id = p_request_id AND br.status = 'handed_over';

  IF v_book_id IS NULL THEN
    RAISE EXCEPTION 'Request not found or not in handed_over state';
  END IF;
  IF v_listing_type <> 'donate' THEN
    RAISE EXCEPTION 'Only donated books can be completed this way';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM book_requests
                 WHERE id = p_request_id AND requester_id = auth.uid()) THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  UPDATE books SET owner_id = auth.uid(), status = 'available',
                   listing_type = 'donate', acquired_via_donation = true
  WHERE id = v_book_id;

  UPDATE book_requests SET status = 'returned', completed_at = now()
  WHERE id = p_request_id;

  DELETE FROM book_progress WHERE request_id = p_request_id;
END;
$$;
```

This function is the entire "Pass It On" feature (Chapter 16): finishing a donated book transfers ownership to the reader, who now becomes its new owner and can re-list it, while forcing `acquired_via_donation = true` so the book is permanently locked into the donation circuit — it can never quietly become someone's personal lending copy. The authorization check (`requester_id = auth.uid()`) matters precisely *because* the function is `SECURITY DEFINER`: without that explicit check, any authenticated user could call this RPC with any request id and reassign someone else's book to themselves.

`increment_read_count_on_return()` is a small trigger function, fired `AFTER UPDATE OF status ON book_requests FOR EACH ROW`: whenever a request's status transitions *into* `returned`, it increments the associated book's `read_count` by one. It fires for both an ordinary lending return

and the UPDATE inside complete_donated_book_reading above — one trigger, two call sites, no duplicated increment logic.

13.5 Club membership and event attendance counting

update_club_member_count(), trigger trg_club_member_count (AFTER INSERT OR DELETE ON club_members), increments or decrements clubs.member_count atomically in the database. This trigger *replaced* an earlier client-side "read count, then write count + 1" approach specifically because two people joining the same club at nearly the same moment could both read the same starting count and each write the same (wrong, too-low) result — a classic read-modify-write race. Doing the arithmetic inside the trigger, on the actual row being inserted/deleted, removes the race entirely.

> **NOTE:** src/lib/admin-actions.ts's removeClubMember (Chapter 18) used to not trust this trigger — after deleting a membership, it *also* manually re-counted club_members and wrote count + 1 back onto clubs.member_count, double-counting on top of the trigger's already-correct decrement. That manual re-count/write has been removed; the trigger is now the sole writer of member_count for join/leave.

> **KNOWN ISSUE:** clubs.member_count's column default was 1 — an assumption, dating from before trg_club_member_count existed, that the creator was counted "for free" without needing a club_members row. But clubs/create/page.tsx has always *also* inserted an explicit club_members row for the creator, and once the atomic trigger above was added, that insert fired the trigger too — incrementing a count that already assumed the creator was included. Every club's member_count was exactly one higher than its real club_members row count, permanently, for every club created after that trigger shipped: caught by creating a fresh club with only its creator as a member and finding member_count = 2, not 1. Fixed by changing the default to 0 and backfilling existing rows from the true count — the trigger was already correct; the default underneath it was not.

update_event_attendee_count(), trigger trg_event_attendee_count (AFTER INSERT OR DELETE ON event_rsvps), is the identical shape applied to club_events.attendee_count — written correctly from the start, having learned from the gotcha immediately above it: club_events.attendee_count defaults to 0, and only the trigger ever increments it.

13.6 Club-creation eligibility and book-notes limits, enforced at the database layer

check_club_creation_eligibility(), trigger trg_check_club_creation_eligibility (BEFORE INSERT ON clubs), re-derives the same "5+ completed exchanges, no reports" rule clubs/create/page.tsx (Chapter 8) already computes client-side, and raises an exception if NEW.creator_id doesn't qualify — so the check now holds even for a request that bypasses the UI entirely.

check_book_notes_limits(), trigger trg_check_book_notes_limits (BEFORE INSERT OR UPDATE ON book_notes), mirrors book-notes-modal.tsx's 100-word note limit and its max-10-community-notes-per-book cap (Chapter 9), rejecting the write at the database layer rather than

trusting the client to have enforced it.

→ HOW TO CHANGE THIS

Raising the per-note word cap to, say, 150 is the same "UI mirrors DB" shape as the club-creation example in Chapter 17, just with two files instead of one:

1. `check_book_notes_limits()` in `supabase/schema.sql` — `IF word_count > 100 THEN`, changed inside a new migration.
2. `src/components/book-notes-modal.tsx` — `const overLimit = words > 100` and the `{words}/100 words` counter label just below it.

The max-10-notes-per-book cap is the identical pattern one level up: `IF other_notes_count >= 10` in the same trigger function, mirrored by `otherNotesCount >= 10` in the same component.

13.7 Wishlist fuzzy matching

Listing 13.4: match_wishlists_for_book — pg_trgm similarity search across all users

```sql
CREATE OR REPLACE FUNCTION public.match_wishlists_for_book(
  p_title text, p_owner_id uuid, p_threshold real DEFAULT 0.35
) RETURNS TABLE (id uuid, user_id uuid, title text)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT w.id, w.user_id, w.title
  FROM public.wishlists w
  WHERE w.active = true AND w.matched_book_id IS NULL
    AND w.user_id <> p_owner_id
    AND similarity(w.title, p_title) > p_threshold
  ORDER BY similarity(w.title, p_title) DESC;
$$;

REVOKE ALL ON FUNCTION public.match_wishlists_for_book(text, uuid, real) FROM public;
GRANT EXECUTE ON FUNCTION public.match_wishlists_for_book(text, uuid, real) TO authenticated;
```

This function exists to fix a bug, not just add a feature: the original client-side approach ran an `ilike '%title%'` query under the *book owner's own session*, but the `wishlists` RLS policy only allows a user to `SELECT` their own rows — so cross-user matching silently returned nothing, ever, for anyone. `SECURITY DEFINER` plus an explicit, narrow return (`id`/`user_id`/`title` only, never a full row) lets it search everyone's wishlists safely. `pg_trgm`'s `similarity()` also fixes a second, independent problem: plain `ilike` matches substrings anywhere (*"The Book"* inside *"The Bookshelf is Empty"*), which trigram similarity does not.

→ HOW TO CHANGE THIS

Matches feeling too loose (*"Old Man River"* pairing with *"The Old Man and the Sea"*)? Raise `p_threshold real DEFAULT 0.35` in the `CREATE OR REPLACE FUNCTION` signature above — say to `0.5` — inside a new migration (`npx supabase migration new tighten_wishlist_match`). One file, one number: unlike the club-creation example above, nothing on the frontend duplicates this threshold, since `my-books/page.tsx` just calls the RPC and trusts whatever it returns.

13.8 Rate limiting

Two BEFORE INSERT triggers enforce limits that, before this migration existed, were configured in platform_settings but never actually checked:

Listing 13.5: enforce_message_rate_limit — reads its own limit from platform_settings

```
CREATE OR REPLACE FUNCTION public.enforce_message_rate_limit()
RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE v_limit int; v_count int;
BEGIN
  v_limit := public.get_platform_setting_int('max_messages_per_hour', 30);
  SELECT count(*) INTO v_count FROM public.messages
  WHERE sender_id = NEW.sender_id AND created_at > now() - interval '1 hour';
  IF v_count >= v_limit THEN
    RAISE EXCEPTION 'RATE_LIMIT_EXCEEDED: max % messages per hour reached', v_limit
      USING ERRCODE = 'P0001';
  END IF;
  RETURN NEW;
END;
$$;
-- trg_enforce_message_rate_limit: BEFORE INSERT ON messages, FOR EACH ROW
-- enforce_request_rate_limit / trg_enforce_request_rate_limit mirrors this
-- exactly, against book_requests and the max_requests_per_day setting.
```

get_platform_setting_int(p_key, p_default) is the small helper both triggers call: it reads platform_settings.value (a jsonb column) and unwraps it with #>> '{}' rather than a plain ::text cast, specifically because the admin settings UI always writes values as JSON strings (e.g. the JSON string "30", not the JSON number 30) — a plain ::text cast on a JSON string keeps the surrounding quote characters and breaks a numeric cast, while #>> '{}' correctly unwraps either encoding to plain unquoted text first.

> **NOTE**: Enforcing these limits as BEFORE INSERT triggers — rather than only checking in the frontend before calling .insert() — means the limit holds even against a direct API call or a malicious client that skips the UI entirely. This is the same category of defence as RLS itself: the database is the last line, not the frontend.

→ HOW TO CHANGE THIS

This is the one limit in the whole manual you should *not* fix with a migration: max_messages_per_hour lives as a row in platform_settings, not a literal in the function body, specifically so it can be tuned without a deploy. As a super_admin, go to /admin/settings in the app and edit the value there (backed by src/app/(dashboard)/admin/settings/page.tsx); the next call to get_platform_setting_int picks it up immediately. Compare this to the club-creation threshold (Chapter 17) or the similarity threshold above — both are baked into code and need a migration. If a value *might* need retuning by a non-engineer, platform_settings is where it belongs.

→ HOW TO CHANGE THIS

Unlike the two limits above, `max_books_per_user` (also seeded in `platform_settings`, default `50`) is currently decorative — grep the codebase and you will not find a single trigger, RPC, or frontend check that ever reads it. Wiring it up means writing a third rate-limit trigger, copying `enforce_message_rate_limit`'s shape almost exactly: a `BEFORE INSERT ON books` trigger that calls `get_platform_setting_int('max_books_per_user', 50)`, counts the caller's existing `books` rows, and `RAISE EXCEPTION`s past the limit — then a migration, per Chapter 6. A good first real ticket if you want practice with this exact pattern before touching a limit that already matters in production.

13.9 Admin analytics

Six `SECURITY DEFINER` functions back the admin Overview dashboard (Chapter 18), each doing one aggregate query so the frontend never has to compute statistics client-side over unfiltered data it isn't allowed to see in full:

Function	What it returns
`admin_get_daily_stats(days_back=30)`	One row per calendar day (via `generate_series`): new users, new books, new requests, completed exchanges — the growth chart's data source.
`admin_get_top_contributors(lim=10)`	Per user: books listed (total/donated/lent, via `FILTER`) and average rating received, ordered by books listed.
`admin_get_area_stats()`	Per non-empty area: distinct user count and distinct book count.
`admin_get_exchange_stats()`	Request counts by status plus a computed `success_rate` (handed-over + returned, over the completed-or-declined total).
`admin_get_overdue_books(threshold_days=30)`	Lend-type requests still `handed_over` past the threshold, with owner and borrower names joined in.
`admin_get_rating_distribution()`	Exactly 5 rows (scores 1–5) via `generate_series`, left-joined to actual counts — guarantees a complete histogram even for scores nobody has ever given.

■ INTERN EXERCISE

Using Studio's SQL editor against your local stack (Chapter 4), run `SELECT * FROM public.admin_get_exchange_stats();` and `SELECT * FROM public.get_books_nearby(34.0837, 74.7973);` (Srinagar's approximate coordinates) directly. Confirm the shapes match this chapter, then find where in the frontend each is called from and match the RPC arguments to what you just typed by hand.

CHAPTER 14

Row-Level Security & the Role Model

14.1 What RLS actually does

Row-Level Security is a Postgres feature, not a Supabase invention: once `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` is run on a table (true for all 26 tables in this schema), *every* query against it — from any role, through any route — is filtered by whichever `CREATE POLICY` statements apply to the current role and command (`SELECT`/`INSERT`/`UPDATE`/`DELETE`). No policy matching means no rows: RLS defaults closed, not open.

The Supabase JS client attaches the calling user's JWT to every request, and Postgres evaluates policies against `auth.uid()` (the JWT's subject claim) and `auth.role()` (`anon` or `authenticated`). This is the entire security model for every Client Component in this app (Chapter 3) — there is no separate authorization layer in application code for ordinary data access, only for admin Server Actions (Chapter 18), which add an *additional* check on top of RLS, not instead of it.

14.2 The role hierarchy

Listing 14.1: The only enum in the schema

```
CREATE TYPE public.admin_role AS ENUM ('super_admin', 'moderator', 'viewer');
```

Role	Can do
`viewer`	Pass `requireAdmin('viewer')` (Chapter 18) — can view every `/admin/*` page but is below every RLS check that specifically requires `is_admin_or_mod()`, so most write actions still fail for a viewer at the database layer even if the frontend let them click a button.
`moderator`	Everything `is_admin_or_mod()` gates: ban/unban/warn users, hide/edit books, manage reports, moderate clubs, broadcast notifications, manage content (Chapter 18).
`super_admin`	Everything above, plus everything `is_super_admin()` gates specifically: deleting books/clubs outright (not just hiding/deactivating), setting other users' admin roles, and writing to `platform_settings`.

A profile only counts as an admin at all if *both* `is_admin = true` **and** `admin_role` is non-null — the two-field check appears in `is_admin()`, `is_admin_or_mod()`, `is_super_admin()`, `requireAdmin()`, and `checkAdminAPI()` identically (Chapter 13, Chapter 18). Setting only one of the two fields (e.g. `admin_role = 'moderator'` without `is_admin = true`) grants nothing.

→ HOW TO CHANGE THIS

Adding a fourth role (say `'support'`, between `viewer` and `moderator`) is more than a one-line enum edit:

1. `ALTER TYPE public.admin_role ADD VALUE 'support';` in a new migration (`npx supabase migration new add_support_role`) — Postgres enums can only be extended, never edited in place.
2. Decide which existing gate the new role should pass — probably widening `is_admin_or_mod()`'s `admin_role IN ('super_admin', 'moderator')` check (Chapter 13) to include `'support'`, in the same migration.
3. `requireAdmin()` and `checkAdminAPI()` (Chapter 18) — wherever they whitelist role strings for a given `/admin/*` page or API route.

Skip step 2 and the new role is assignable in `/admin/settings` but grants nothing at the database layer — every RLS policy still only recognises the three original roles.

14.3 Policy summary, table by table

Table	Who can do what
`profiles`	Any authenticated user SELECTs all profiles (community-visible by design); a user INSERTs/UPDATEs only their own row; admins SELECT all and UPDATE any (for bans/moderation) — but see the trigger below the table, which restricts *which columns* an UPDATE may touch regardless of whose row it is. No DELETE policy — profiles cascade-delete only via `auth.users`.
`books`	Authenticated users SELECT all; anon/public SELECT only `available`/`given`; owner INSERT/UPDATE/DELETE their own; admins SELECT all and UPDATE any; only `super_admin` can DELETE any book outright.
`book_requests`	A user SELECTs requests where they are the requester *or* own the requested book; requester INSERTs their own; the book's owner UPDATEs it; admins SELECT/UPDATE all. No DELETE policy — a request is never removed, only status-transitioned.
`book_progress`	Any authenticated user SELECTs all (community visibility of reading activity); only the reader themself INSERT/UPDATE/DELETEs their own row.
`messages`	A user SELECTs/INSERTs only for requests where they are the requester or the book owner, and only ever as themself; admins SELECT all (moderation). No UPDATE/DELETE policy at all.
`bookmarks`, `wishlists`	Fully self-service: SELECT/INSERT/UPDATE/DELETE restricted to `user_id=auth.uid()`, no exceptions, no admin override.
`ratings`	Authenticated users SELECT all; a user INSERTs only as `rater_id=auth.uid()`, and only if `rated_user_id` is the verified counterparty on a `handed_over`/`returned` `book_requests` row (see below the table); admins SELECT all. No UPDATE/DELETE — ratings are immutable once given.

Table	Who can do what
book_notes	Any authenticated user SELECTs all; a user INSERT/UPDATE/DELETEs only their own note.
clubs	Anyone (incl. anon) SELECTs active clubs; any authenticated user INSERTs as creator; the creator UPDATE/DELETEs their own; admins/mods UPDATE any, only super_admin DELETEs any.
club_members	Any authenticated user SELECTs; a user INSERTs themself (join); a user DELETEs their own membership, the club's creator can remove anyone, admins/mods can too.
club_posts	Anyone SELECTs; only the club's creator INSERTs (posts); only the post's author DELETEs it. No UPDATE at all.
club_events	Anyone (incl. anon) SELECTs active events — visible regardless of the viewer's club membership; only the club's creator INSERTs/UPDATEs/DELETEs.
event_rsvps	Any authenticated user SELECTs (mirrors club_members); a user INSERTs themself, but only if the event is public *or* they're already a club member, *and* it isn't already at capacity; a user DELETEs only their own RSVP.
notifications	A user SELECT/UPDATEs only their own; admins/mods INSERT for anyone explicitly — **but see the gotcha below**.
reports	A reporter INSERTs as themself and SELECTs their own; admins/mods SELECT all and UPDATE any. No DELETE.
admin_audit_log, admin_notes, user_bans, user_warnings	Admin/mod-only SELECT and INSERT (as themself where an admin id is recorded). No UPDATE/DELETE on the audit log or notes; user_bans allows admin UPDATE (for unbanning).
announcements, genres, areas, book_of_month	Public/anon SELECT of active rows; admins/mods have full ALL-command management.
notification_templates	Admin/mod-only SELECT and full management — not readable by regular users at all.
platform_settings	Anyone (incl. anon) SELECTs all settings; only super_admin can INSERT/UPDATE/DELETE.
storage.objects (book-covers)	Anyone (incl. anon) SELECTs — cover images are public by design; a user may only INSERT/UPDATE/DELETE objects whose path is prefixed with their own auth.uid() ({userId}/{timestamp}.{ext}, matching the client's own upload convention). This is a different schema (storage, not public) but the same RLS mechanism applies.

14.4 Six RLS looseness gotchas — since tightened

> **NOTE**: notifications **INSERT policy.** There used to be a separate, broader policy (alongside the explicit "admins/mods can INSERT for anyone" one) allowing *any* authenticated user to INSERT a notification row, checked only against auth.uid() IS NOT NULL — with no restriction on whose user_id the notification targets. That policy has been replaced with "Users insert own notifications", requiring auth.uid() = user_id. Since most

of the app's real notification flows are cross-user by design (a request being accepted notifies the *other* party, not the actor), `createNotification` (Chapter 15) now writes through the service-role client instead of the calling user's session — it remains the one trusted, server-side gate for cross-user notification creation, after first confirming the caller is authenticated. A direct, unauthenticated-of-relationship REST call to `/notifications` can no longer insert a row addressed to an arbitrary other user.

NOTE: **Duplicate legacy/new policy pairs — removed, except once, which mattered.** `profiles`, `reports`, *and* `books` each used to carry two separate, overlapping sets of admin policies — one written against the older inline pattern (`EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND is_admin = true)`) from before the `admin_role` hierarchy existed, and a newer set written against `is_admin_or_mod()`/`is_super_admin()`. For `profiles` and `reports` this was harmless (Postgres OR-combines multiple permissive policies for the same command, and both old and new policies granted the same effective access) — dead weight to keep in sync, nothing more. It was *not* harmless for `books`: the legacy `"Admins delete books"` policy granted DELETE to any `is_admin = true` user, while the newer `"Admins can delete any book"` policy correctly restricted DELETE to `is_super_admin()` — so a plain `moderator` could delete any book outright, silently bypassing the higher bar the newer policy was written to enforce. A dedup pass had already dropped the legacy `profiles`/`reports` policies; `books` was missed because nothing about the OR-combination behavior makes an over-broad legacy policy *visible* — it just quietly wins whenever it's the more permissive of the two. The legacy `"Admins delete books"` policy has now been dropped too.

NOTE: **Self-escalation via unrestricted profile UPDATE — closed with a trigger, not a policy.** The `"Users can update own profile"` policy (`auth.uid() = id`) and the admin equivalent both grant UPDATE access based on *who* is updating a row, never *which columns* the update touches. Nothing stopped an ordinary authenticated user from calling PostgREST's `PATCH /profiles` directly, scoped to their own row, with a body of `{"is_admin": true, "admin_role": "super_admin"}` — and granting themselves full admin. A genuine, exploitable privilege-escalation bug, not a theoretical one. A second, narrower version of the same gap let an existing `moderator` promote themselves to `super_admin`, since the DB-level admin UPDATE policy only checked `is_admin_or_mod()` while the app-level `guardRole('super_admin')` check in `setAdminRole` (Chapter 15) could simply be bypassed by calling PostgREST directly. RLS policies can't express "these specific columns, conditionally" — so the fix is a `BEFORE UPDATE` trigger, `guard_profile_privileged_columns()`, layered on top of the existing policies: it compares `OLD`/`NEW` and raises an exception if `is_admin`/`admin_role` changed without the caller already being `is_super_admin()`, or if `is_banned`/`ban_reason`/`ban_expires_at`/`warning_count` changed without `is_admin_or_mod()`. The trigger explicitly no-ops when `auth.uid()` is `NULL` — a service-role or superuser context, never spoofable by an end-user JWT — so backend scripts and the very first admin bootstrap still work.

NOTE: `ratings` **INSERT — now verifies the counterparty, not just the rater.** `"Users can insert own ratings"` used to check only `auth.uid() = rater_id`, with no verification that `rated_user_id` was the real counterparty on `request_id`, nor that the exchange had actually completed. Anyone who knew or guessed a

`request_id` UUID could insert a rating against an arbitrary `rated_user_id`, skewing `admin_get_top_contributors()`'s average (the existing `UNIQUE(request_id, rater_id)` constraint capped the damage to one bad rating per known request, but didn't prevent it). The policy's `WITH CHECK` now also requires an `EXISTS` match against `book_requests`/`books` confirming `request_id` is `handed_over` or `returned` *and* that `rater_id`/`rated_user_id` are exactly the request's requester and the book's owner, in either direction.

NOTE: `book-covers` **storage uploads — now scoped to the uploader's own path.** `"Authenticated users can upload book covers"` used to check only `bucket_id = 'book-covers'`, with no path restriction — and no UPDATE/DELETE policy existed on the bucket at all. Any authenticated user could upload to (and, once one existed, overwrite) any object path, including another user's. The client already namespaced uploads as `{userId}/{timestamp}.{ext}` (Chapter 8); the INSERT policy was replaced and UPDATE/DELETE policies added, all three requiring `(storage.foldername(name))[1] = auth.uid()::text` — enforcing server-side what was previously only a client-side naming convention.

NOTE: `event_rsvps` **INSERT —** `capacity` **was missing from the check entirely at first.** The policy shipped checking `visibility` (public event, or caller is a club member) but not `capacity` — so the "Event full" state was enforced only by `events/[id]/page.tsx` disabling its RSVP button once `attendee_count >= capacity`, exactly the "UI-only gate" pattern the club-creation eligibility check (Chapter 17) already taught this codebase to distrust. Confirmed exploitable by testing it directly: a raw `POST /rest/v1/event_rsvps`, as a real signed-in user, against a capacity-1 event that already had its one attendee, returned `201`. The policy's `WITH CHECK` now also requires `capacity IS NULL OR attendee_count < capacity`, evaluated inside the same `EXISTS` as the visibility check, alongside the caller's own `user_id`.

INTERN EXERCISE

Log into the local Studio SQL editor (Chapter 4) as the anon role would see it — run `SET ROLE anon;` then `SELECT * FROM public.profiles;` and `SELECT * FROM public.books;` in the same session, and compare row counts against running the same two queries as an authenticated test user. This is the fastest way to build real intuition for what RLS is actually restricting, versus what you'd assume from reading policy SQL alone.

CHAPTER 15

Server Actions, Email & Notifications

15.1 What a Server Action is, and how OLK uses them

A file starting with the string `'use server'` exports functions that Next.js turns into callable, server-only RPCs from Client Components — the function body only ever executes on Vercel's servers, but a Client Component can `await` it as if it were a local async function, with no hand-written `fetch` or API route needed. `src/lib/admin-actions.ts` and `src/lib/notifications.ts` are the only two `'use server'` files in this codebase; every admin page in Chapter 18 calls into the former directly.

15.2 The admin action pattern: guard, act, log, revalidate

Every one of the 24 functions in `admin-actions.ts` follows the identical four-step shape — but as of a code-quality pass across the whole codebase, that shape now lives in exactly one place, a higher-order function named `withAdminAction`, instead of being copy-pasted into every function body:

Listing 15.1: src/lib/admin-actions.ts — the shared wrapper, illustrated with banUser

```
function withAdminAction<Args extends unknown[]>(
  minRole: AdminRole,
  action: string,
  handler: (ctx: AdminActionCtx, ...args: Args) => Promise<AuditMeta>
): (...args: Args) => Promise<ActionResult> {
  return async (...args: Args) => {
    try {
      const admin = await guardRole(minRole)                    // 1. GUARD
      const supabase = await createClient()
      const { targetType, targetId, details } =
        await handler({ admin, supabase }, ...args)              // 2. ACT
      await auditLog(admin.id, action, targetType, targetId, details)  // 3. LOG
      revalidatePath('/admin')                                   // 4. REVALIDATE
      return { success: true }
    } catch (e) {
      return { success: false, error: (e as Error).message }
    }
  }
}

export const banUser = withAdminAction(
  'moderator', 'ban_user',
  async ({ admin, supabase }, userId: string, reason: string, isPermanent: boolean,
    durationDays?: number) => {
```

```
    const expiresAt = !isPermanent && durationDays
      ? new Date(Date.now() + durationDays * 86400000).toISOString() : null

    await supabase.from('user_bans').insert({ user_id: userId, admin_id: admin.id, reason,
    is_permanent: isPermanent, expires_at: expiresAt })
    await supabase.from('profiles').update({ is_banned: true, ban_reason: reason,
    ban_expires_at: expiresAt }).eq('id', userId)
    await supabase.from('notifications').insert({ user_id: userId, type: 'admin', title: /*
    ... */ '', link: '/profile' })

    return { targetType: 'user', targetId: userId, details: { reason, isPermanent,
    durationDays } }
  }
)
```

1. **Guard** — `guardRole(minRole)` calls `checkAdminAPI` (Chapter 14) and throws if the caller doesn't meet the role floor. This is a check made on top of RLS, not instead of it — the subsequent Supabase calls in the handler still go through RLS as whatever role the Server Action's own Supabase client authenticates as (the calling user's session, via `src/lib/supabase/server.ts`), so a bug in `guardRole` would still have to also defeat RLS to cause real damage.
2. **Act** — the handler passed to `withAdminAction` does the actual mutation(s), near-always inserting a `notifications` row telling the affected user what just happened to them, then returns *what to audit-log* rather than performing the log itself.
3. **Log** — the wrapper writes exactly one row to `admin_audit_log` (Chapter 12) per call, built from the handler's return value plus the wrapper's own `action` string. This is not optional or conditional anywhere — a handler cannot forget it, because it isn't the handler's job to call it.
4. **Revalidate** — `revalidatePath('/admin')` tells Next.js's cache that server-rendered data under `/admin` may be stale and should be refetched on next navigation.

PRO TIP: Every action still resolves to `{ success: boolean, error?: string }` rather than throwing across the server/client boundary — that contract is unchanged, only where the `try/catch` lives has moved. Calling code throughout the admin pages (Chapter 8) still uniformly checks `result.success` rather than using `try/catch` at the call site.

→ HOW TO CHANGE THIS

Adding action #25 to `admin-actions.ts`? Wrap it with `withAdminAction` rather than hand-writing the guard/log/revalidate scaffolding:

1. `export const yourAction = withAdminAction(minRole, 'your_action_name', async ({ admin, supabase }, ...yourArgs) => { ... })` — `minRole` is `'moderator'` for anything reversible, `'super_admin'` for anything destructive or platform-settings-adjacent (Chapter 18).
2. Inside the handler: mutate, then insert a `notifications` row for whichever user should be told what just happened.
3. End the handler with `return { targetType: 'your_target_type', targetId, details: { ...whatever's worth auditing } }` — this is what becomes the

`admin_audit_log` row; there is no way to skip it, since it's also the handler's only return value.

The old failure mode — an action that works but is invisible to `admin/settings/page.tsx`'s audit log because a developer forgot the `auditLog(...)` call — is now structurally harder to hit, since logging happens in the wrapper rather than being one more line a handler author has to remember.

15.3 Notifications: in-app row first, email best-effort second

Listing 15.2: src/lib/notifications.ts — in full

```
type NotificationContext =
  | { kind: 'request'; id: string }
  | { kind: 'club_join'; id: string }
  | { kind: 'club_announcement'; id: string }
  | { kind: 'event_created'; id: string }
  | { kind: 'wishlist_match'; id: string }

export async function createNotification({ userId, type, title, link, context }:
    NotificationPayload) {
  const supabase = await createServerClient()          // the calling user's session
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error('Unauthorized')

  const { data: allowed, error: guardError } = await supabase.rpc('can_notify', {
    p_target_user: userId,
    p_context_type: context.kind,
    p_context_id: context.id,
  })
  if (guardError || !allowed) {
    throw new Error('Forbidden: no valid relationship to notify this user')
  }

  const supabaseAdmin = createAdminClient(              // service-role client
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
  )
  const { error } = await supabaseAdmin.from('notifications').insert({
    user_id: userId, type, title, link: link || '/notifications',
  })
  if (error) throw error

  try {
    await sendNotificationEmail({ userId, type, title })
  } catch (err) {
    console.error('Email notification failed:', err)
  }
}
```

The in-app notification row is written first and its failure is a hard error (thrown, propagated to the caller); the email send is wrapped in its own `try/catch` and its failure is only logged. This ordering is deliberate: a user should always see the notification inside the app even if their email is undeliverable or Resend is down — email is a convenience layer on top of a system that must work without it.

NOTE: This function used to insert with the calling user's own session client, which worked only because the `notifications` INSERT policy was, at the time, wide open to any authenticated user (Chapter 14). Now that policy only allows `auth.uid() = user_id`, and most calls here are inherently cross-user (the *other* party in a request/message/club gets notified, not the caller) — so this action switched to the service-role client, after first confirming the caller has a valid session. It is now the trusted server-side gate for every cross-user notification the app creates outside the admin actions in Chapter 18 (which have their own, separate "admins/mods can insert for anyone" policy).

NOTE: **That gate used to check only "is the caller logged in," not "does the caller actually have a reason to notify this specific person."** Since every call here goes through the service-role client, an authenticated-only check meant any user could call this Server Action directly (it's a plain exported async function, reachable from any client) with an arbitrary `userId`, `title`, and `link` — spamming or phishing an unrelated user with a fake notification. The fix is the `context` parameter and the `can_notify(p_target_user, p_context_type, p_context_id)` RPC shown above: a `SECURITY DEFINER` SQL function (Chapter 13) that verifies the caller and `userId` are the two real parties on the referenced `book_requests`/`clubs`/`club_members`/`wishlists` row, before the insert is allowed to proceed. It needs to be `SECURITY DEFINER` for at least one case that RLS alone can't express: a book owner adding a new listing that matches someone else's wishlist has no RLS-visible relationship to that wishlist row (`"Users can view own wishlists"` would otherwise hide it from them entirely), so the verification has to run with elevated privilege and check the real underlying fact — that the wishlist's `matched_book_id` now points to a book the caller owns — rather than relying on what the caller's own session is allowed to `SELECT`. All six call sites (`browse`, `my-books`, `requests`, `messages/[id]`, `clubs/[id]` ×2) now pass a `context` identifying which real-world relationship justifies the notification.

15.4 Sending the actual email: a service-role client, on purpose

Listing 15.3: src/lib/send-notification-email.ts — the privileged lookup

```
export async function sendNotificationEmail({ userId, type, title }: {...}) {
  if (!resend) return   // silently no-op if RESEND_API_KEY is unset

  const supabaseAdmin = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,   // NOT the anon key
  )
  const { data: authUser } = await supabaseAdmin.auth.admin.getUserById(userId)
  const email = authUser?.user?.email
  if (!email) return
  // ... resend.emails.send({ ..., html: `... ${escapeHtml(title)} ...` })
}
```

Resolving *any* user's email address by id is an operation no ordinary RLS policy would ever grant a client — email addresses live in `auth.users`, not `public.profiles`, and are never exposed through the query builder. This file constructs a second, separate Supabase client using the **service role key** instead of the anon key specifically to call the privileged `auth.admin.getUserById` API, and the file begins with `import 'server-only'` (Chapter 2) so that a build error occurs if this

file is ever accidentally reachable from client-side code — the service role key must never reach a browser bundle.

WARNING: Notice `escapeHtml(title)` around the one piece of user-influenced content interpolated into the email's raw HTML string. A notification's `title` field can, depending on the notification type, contain text a regular user chose (e.g. a report reason echoed back, or in principle a maliciously crafted book title flowing through some future notification type). Escaping it before string-interpolating into HTML is the only thing standing between that and an HTML/script injection inside an email client. If you add a new notification type, keep any user-influenced text passed through `escapeHtml` before it reaches this file.

→ HOW TO CHANGE THIS

A brand-new notification `type` (say `'club_invite'`) touches more than the string you pass to `createNotification`:

1. There is no enum or TypeScript union constraining `type` — it's a plain string column (Chapter 12). A typo here doesn't fail at write time; it fails silently at read time, as step 2 below.
2. `src/components/notification-bell.tsx` and `src/app/(dashboard)/notifications/page.tsx` both index into an `ICONS` lookup object keyed by `n.type`, falling back to a generic bell emoji when the key is missing — add your new type as a key in *each* file's `ICONS` object, or it silently falls back to that generic icon rather than erroring.
3. If `title` ever includes user-typed text for this new type, route it through `escapeHtml` before it can reach `send-notification-email.ts` (the warning above) — easy to forget for a brand-new call site since nothing enforces it.
4. Decide which existing `context.kind` the new call site's relationship maps to (`'request'`, `'club_join'`, `'club_announcement'`, `'event_created'`, or `'wishlist_match'`) — or, if it's a genuinely new kind of relationship, add a new branch to `can_notify()` (Chapter 14) *before* wiring up the call site. `'event_created'` itself is the `'club_announcement'` shape one table over: the caller must be the event's own `creator_id`, and the target must be a real member of that event's club. `createNotification` throws `"Forbidden"` if `can_notify` doesn't recognise the `context.kind` or can't verify the relationship, so a new notification type with no matching branch fails loudly for every caller rather than silently succeeding.

15.5 The weekly digest: a genuine Route Handler

`src/app/api/digest/route.ts` is the one place in this codebase that is a real HTTP endpoint rather than a Server Action, because it needs to be invoked by something outside a page render — an external cron scheduler (Chapter 19).

Listing 15.4: app/api/digest/route.ts — auth gate and the per-subscriber loop

```
async function handleDigest(request: Request) {
  const authHeader = request.headers.get('authorization')
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return Response.json({ error: 'Unauthorized' }, { status: 401 })
  }
```

```
  if (!resend) return Response.json({ error: 'Email service not configured' }, { status: 503
    })

  const supabaseAdmin = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.
    SUPABASE_SERVICE_ROLE_KEY!)
  const subscribers = /* profiles where email_digest = true */
  const recentBooks = /* books from the last 7 days, status = available */

  let sent = 0
  const failures: { userId: string; error: string }[] = []
  for (const sub of subscribers) {
    try {
      const { data: authUser } = await supabaseAdmin.auth.admin.getUserById(sub.id)
      const email = authUser?.user?.email
      if (!email) continue
      await resend.emails.send({ /* ... */ })
      sent++
    } catch (e) {
      failures.push({ userId: sub.id, error: (e as Error).message })
    }
  }
  return Response.json({ sent, books: recentBooks.length, failures: failures.length,
    failureDetails: failures })
}

export async function GET(request: Request) { return handleDigest(request) }
export async function POST(request: Request) { return handleDigest(request) }
```

Authorization here is a plain shared-secret Bearer token comparison against `CRON_SECRET` (Chapter 19 covers where that value comes from) — appropriate for a server-to-server cron trigger with no user session involved.

> **NOTE**: The per-subscriber loop used to have no `try/catch` around `resend.emails.send(...)` — a malformed address or a transient Resend error would abort the entire run for every subscriber after that point in iteration order. Each send is now wrapped individually, with failures collected and returned in the response rather than left to abort the run. The route also used to export only `POST`, while Vercel's Cron Jobs feature calls with `GET` by default (Chapter 19); it now exports both, delegating to the same `handleDigest` logic.

INTERN EXERCISE

With `RESEND_API_KEY` unset (the default in local development, per Chapter 4), trigger the digest route locally with `curl -X POST localhost:3000/api/digest -H "Authorization: Bearer $CRON_SECRET"` and confirm you get the `503` "Email service not configured" response. Then set a (fake, non-functional) `RESEND_API_KEY` in `.env.local`, restart the dev server, and confirm the response changes to an attempted send that now fails differently — this is the fastest way to see the file's two independent failure modes without needing a real Resend account.

PART V

Key Workflows End to End

CHAPTER 16

The Book Exchange Lifecycle, End to End

Every previous chapter covered one layer — a page, a table, a function. This chapter traces the single most important journey in the product through all of them at once: a book going from someone's shelf to someone else's hands.

16.1 The full state machine

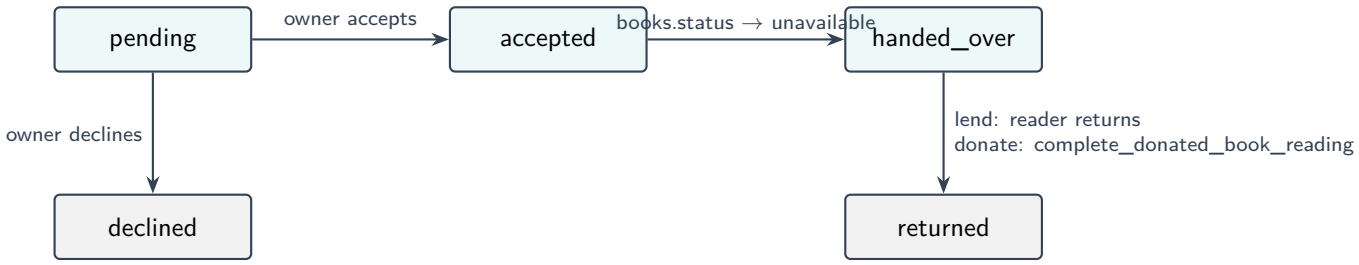

Figure 16.1: `book_requests.status`, the spine of every exchange. `trg_increment_read_count` fires the instant a row enters `returned`, regardless of which arrow got it there.

16.2 Step 1 — Listing

A book starts life in `my-books/page.tsx` (Chapter 8): title, author, condition, `listing_type` (`donate` or `lend`, chosen once, up front), optionally enriched via the ISBN scanner (Chapter 9). The moment it's inserted, `match_wishlists_for_book` (Chapter 13) runs against it — if anyone's wishlist fuzzy-matches the new title, that is the first notification this book will ever cause, before anyone has even requested it (Chapter 17).

16.3 Step 2 — Discovery

`browse/page.tsx` surfaces it, sorted by distance if the viewer has a location set (`get_books_nearby`, Chapter 13), filterable by genre/condition/type/area. Both `available` and `unavailable` books are shown (the latter marked as such) — Chapter 13 noted that `get_books_nearby`'s `WHERE` clause deliberately includes `unavailable`, so the community can see a book is out on loan rather than it vanishing from view entirely.

16.4 Step 3 — Requesting

A reader requests it: a `book_requests` row is inserted, `status = 'pending'`, guarded by `trg_enforce_request_rate_limit` (max 10/day, Chapter 13) and the partial unique index preventing a second concurrent request from the same user on the same book (Chapter 12). The owner sees it appear on their `requests` page as an *incoming* request; the requester sees it on theirs as *outgoing* — same table, two different query shapes (Chapter 8, §8.4), and the owner-profile-lookup gotcha documented there.

16.5 Step 4 — Accept, decline, or expire into noise

The owner accepts or declines from their incoming list. There is no automatic expiry — a `pending` request with no owner response simply sits there indefinitely; nothing in the schema or a scheduled job currently cleans these up (worth knowing if you're ever asked "why do old pending requests never disappear").

Accepting flips `status` to `accepted` and (in the UI layer) typically flips the book's own `status` to `unavailable`, signalling to every other browsing user that this copy is now spoken for.

16.6 Step 5 — Handover

When the two parties actually meet, the owner marks the request `handed_over`, stamping `handed_over_at`. From this point, `book_progress` (Chapter 12) exists for this request — the reader can update a 0–100 reading-progress percentage, visible to the whole community as a piece of ambient activity (nobody else can write it, everybody can see it).

> **NOTE:** `handed_over_at` is also the timestamp `admin_get_overdue_books` (Chapter 13) measures against, for `lend`-type books only, using the `overdue_days_threshold` platform setting (default 30). A donation is never "overdue" — there's nothing to give back.

→ HOW TO CHANGE THIS

This is the one "threshold" in the whole manual with *three* independent numbers, not two — worth tracing in full before you touch it:

1. `admin_get_overdue_books(threshold_days integer DEFAULT 30)` in `supabase/schema.sql` — the SQL function's own default.
2. The `overdue_days_threshold` row in `platform_settings` (also `30`), editable at `/admin/settings`.
3. `src/app/(dashboard)/admin/requests/page.tsx`, line 87 — `supabase.rpc('admin_get_overdue_books', { threshold_days: 14 })` — a hardcoded `14`.

Because the frontend always passes an explicit argument, #3 is the *only* value that actually reaches production — #1 never runs (there's always an argument), and editing #2 in Settings changes nothing, since nothing ever reads it back for this call. If you're asked to make the overdue threshold admin-configurable, the real fix is changing line 87 to first `select()` the `overdue_days_threshold` row from `platform_settings` (the same

plain-query approach `src/proxy.ts`'s feature flags use, Chapter 7 — not the SQL-only `get_platform_setting_int` helper, which only trigger functions can call) and pass *that* value as `threshold_days`, rather than trusting the Settings-page value to already be wired up.

16.7 Step 6a — Lending: the book comes back

The owner (or, per RLS, whoever can update the request — Chapter 14) marks it `returned`. `trg_increment_read_count` fires, `books.read_count` goes up by one, and both parties are prompted to rate each other via `rating-modal.tsx` (Chapter 9) — one rating per `(request_id, rater_id)` pair, enforced by a unique constraint rather than a pre-check.

16.8 Step 6b — Donating: the book changes hands permanently

For a `donate` listing, there is no "return" — instead, once the reader finishes, the frontend calls `complete_donated_book_reading(request_id)` (Chapter 13 has the full function body). In one `SECURITY DEFINER` transaction: ownership of the `books` row transfers to the reader, its `status` resets to `available` (the new owner can now re-list it, lend it, or donate it onward), `acquired_via_donation` is forced `true` (so it can never quietly become someone's personal lending copy — Chapter 12), the request is marked `returned` (which is what actually fires `trg_increment_read_count` — donation completion and lending return share that one trigger), and the now-stale `book_progress` row is deleted.

> **NOTE**: This is why a book's `read_count` is a genuinely meaningful "how many people has this book reached" metric across both listing types, and why the "Trusted Sharer" badge and admin top-contributor stats (Chapters 9, 13) can treat donations and completed loans as comparable evidence of community contribution — the schema was deliberately designed so one counter serves both paths.

INTERN EXERCISE

Walk this entire lifecycle by hand against your local Docker stack (Chapter 4): register two test accounts, list a `donate` book as account A, request it as account B, accept, hand over, update reading progress, then complete the donation. Confirm in Studio that `books.owner_id` really did change to account B's id, and that `books.read_count` is now 1. This single exercise touches more of this manual — pages, schema, functions, RLS — than any other one thing you could do first.

CHAPTER 17

Wishlists, Matching, Clubs & Events

17.1 Wishlists: asking for a book that doesn't exist yet

A wishlist entry (`wishlist/page.tsx`, Chapter 8) is just a title, author, and genre a user wants but couldn't find on Browse. On its own it does nothing — it becomes useful only through the matching flow triggered from the *other* side of the transaction.

The matching flow

1. User A adds *"The Old Man and the Sea"* to their wishlist. Nothing happens yet — `matched_book_id` stays `NULL`.
2. User B, independently, lists a copy of *"The Old Man and The Sea"* (different capitalisation, doesn't matter) on `my-books`.
3. `my-books/page.tsx` calls `match_wishlists_for_book(title, owner_id)` (Chapter 13) right after the insert.
4. The `pg_trgm` similarity search finds User A's wishlist row (similarity comfortably above the 0.35 threshold despite the capitalisation difference), and the frontend updates that row's `matched_book_id` and fires a notification to User A.

> **NOTE**: Re-read Chapter 13's explanation of *why* this needs to be a `SECURITY DEFINER` RPC rather than a plain query: the `wishlists` RLS policy (Chapter 14) restricts a normal `SELECT` to `user_id = auth.uid()`. User B's own session, running the match search, could never see User A's wishlist row through an ordinary query — the whole feature would be silently non-functional without the RPC's elevated privilege, which is exactly the bug this migration fixed.

The unique index on `(user_id, lower(btrim(title)))` (Chapter 12) exists to stop a second, independent problem: without it, adding the same title twice (perhaps after a page refresh double-submits a form) would previously have produced two wishlist rows, and a later match would notify the user twice for the same book.

17.2 Clubs: geography plus a trust gate

Clubs are the only feature in the schema besides books themselves that carries its own latitude/longitude (Chapter 12) and its own nearby-search RPC, `get_clubs_nearby` (Chapter 13) — the same haversine pattern, applied to a different table, ordered by distance then by `member_count`.

Creating a club is gated on trust, in the UI and now the database

`clubs/create/page.tsx` (Chapter 8) requires 5+ completed exchanges and zero reports against the creator, computed with two client-side count queries before the form will even submit. The `clubs` table's INSERT policy still allows any authenticated user (Chapter 14), but a `BEFORE INSERT` trigger, `check_club_creation_eligibility` (Chapter 13), now re-derives the same eligibility rule at the database layer, so the gate holds even for a request that bypasses the UI.

→ HOW TO CHANGE THIS

Raising the bar to, say, 10 exchanges means editing the number in *two* places, both of which currently hard-code `5`:

1. `src/app/(dashboard)/clubs/create/page.tsx`, line 81 — `setEligible(completedCount >= 5 && !userHasReports)` — plus the `5` inside the "5+ completed exchanges" label around line 132, or the badge will contradict the actual gate.
2. `check_club_creation_eligibility()` in `supabase/schema.sql` — the line `IF completed_count < 5 OR report_count > 0 THEN`. Never hand-edit `schema.sql` directly: run `npx supabase migration new raise_club_threshold`, put the `CREATE OR REPLACE FUNCTION` with `< 10` in the new migration file, then apply it (Chapter 4).

Ship only #1 and the UI merely *feels* stricter — a direct API call still gets through at the old threshold, since the trigger is what actually enforces the rule.

Membership and posts

Joining is a simple `club_members` insert, kept honest by `trg_club_member_count` (Chapter 13) rather than client-side arithmetic. `removeClubMember` in admin actions (Chapter 18) used to redundantly re-count on top of the trigger, introducing an off-by-one on every admin-initiated removal; that manual re-count has been removed.

Posting to a club is creator-only (Chapter 14); a new post fans out one notification per member via a single batched `.insert(notifications[])` call (Chapter 8) — the same N+1-avoidance discipline seen in the messages list (Chapter 8) and the admin broadcast action (Chapter 18).

Deactivation is soft, always

Neither a club's creator nor an admin can hard-delete a club through the app layer — `active = false` is the only operation exposed (`clubs/[id]/page.tsx`'s "delete," and the admin `deactivateClub`/`reactivateClub` actions, Chapter 18). Only a `super_admin`, and only directly via SQL, could actually remove a `clubs` row (Chapter 14's DELETE policy) — there is no UI path to a hard delete at all, by design.

■ INTERN EXERCISE

Add a wishlist entry for a title that doesn't exist yet, then list a book with a *deliberately misspelled* version of that title from a second test account (swap two letters, or use a different capitalisation) and confirm the match still fires. Then try a title that only shares one common word (e.g. "The Old Man" vs. "Old Man River") and confirm it does *not* match — this is the fastest way to build a feel for where the 0.35 similarity threshold actually sits in

practice, beyond just reading the number.

17.3 Events: club-scoped, but publicly discoverable

A club can schedule events (`club_events`, plus one `event_rsvps` row per attendee — Chapter 12): a title, description, start/end time, either an in-person location or an online meeting link, an optional attendee cap, and a per-event `visibility` of `public` or `members_only`.

> **NOTE**: `visibility` **gates the RSVP action, not discovery.** Every event stays listed and viewable by anyone — on the club's own page, on the cross-club `/events` directory, and in the landing page's "Upcoming Events" preview — regardless of the viewer's club membership, mirroring how clubs themselves are already publicly browsable (Chapter 14). A `members_only` event just shows "Join the club to RSVP" instead of an RSVP button for a non-member. Two different features, deliberately not conflated: hiding an event outright would need a second, separate flag.

Creating an event is gated exactly like posting an announcement (Chapter 14) — only the club's creator — rather than the club-creation eligibility rule earlier in this chapter; a club's creator has already cleared that bar once, so there is no second trigger to re-derive it for events. `trg_event_attendee_count` keeps `club_events.attendee_count` in sync with real `event_rsvps` rows, the same atomic-trigger shape as `trg_club_member_count` above (Chapter 13).

> **KNOWN ISSUE**: The RSVP `INSERT` policy's `visibility` check was added alongside a second, independent condition — `capacity IS NULL OR attendee_count < capacity` — but that second condition didn't ship on day one. The "Event full" state was originally enforced only by disabling the RSVP button once `attendee_count >= capacity` in `events/[id]/page.tsx`; a direct `POST /rest/v1/event_rsvps` call could still RSVP past a full event, confirmed by testing exactly that against a capacity-1 event before the policy was corrected. Same lesson as the club-creation gate earlier in this chapter, one feature later: a disabled button is a UX nicety, not a security boundary — only the policy is.

Event notifications reuse the club-announcement broadcast shape: creating an event fans out an `event_created` notification to every current member (Chapter 15), routed through the same `can_notify` guard with a new branch verifying the caller is the event's own creator and the target is a real club member. There is deliberately no per-RSVP notification to the organiser — at any real scale that would be pure noise, so the attendee list on the event and admin pages covers that need instead.

The cross-club directory (`/events`) uses `get_events_nearby`, the same haversine-and-`NULLS LAST` pattern as `get_clubs_nearby` (Chapter 13), falling back to a plain `created_at`-ordered query when the viewer has no location set. A zero-dependency Route Handler, `api/events/[id]/ics/route.ts`, generates an RFC5545 calendar file on demand for "Add to calendar" — no external library, just a template string with the handful of characters (`,`, `;`, `\`) that need escaping inside an ICS field.

INTERN EXERCISE

Create a club event with `capacity` set to 1, RSVP to it yourself, then — from a second test account that is *not* a member of that club — try to RSVP through the UI (should be blocked by the disabled button) and then directly via a `POST` to `/rest/v1/event_rsvps` with that account's own access token (should come back `403`). This is the same "is it a real gate or just a hidden button" habit Chapter 14's task asks you to build for club creation, applied to the newest feature in the schema.

CHAPTER 18

The Admin & Moderation Subsystem, End to End

Chapters 14 and 15 covered the role hierarchy and the guard/act/log/revalidate action pattern in isolation. This chapter follows a single report from submission to resolution through every layer it touches, then surveys the parts of the admin subsystem a report never reaches: content management and platform settings.

18.1 A report's journey

1. Submission

Any user can flag another user or a book from `report-modal.tsx` (Chapter 9) — a reason (from a fixed radio list) plus optional free-text details. This inserts one `reports` row, `status = 'pending'`, `category = 'other'` by default. RLS (Chapter 14) lets the reporter see only their own submitted reports from here on — visibility into the report's fate belongs to moderators.

2. Triage

`admin/reports/page.tsx` (Chapter 8) lists every report, filterable by status. A moderator can re-categorise it (`updateReportCategory`), assign it to a specific admin (`assignReport`), and leave internal notes (`addAdminNote`, written to the separate `admin_notes` table so reporters and reported users never see moderator deliberation) — each of these three, like every admin action, guarded by `guardRole('moderator')` and logged to `admin_audit_log` (Chapter 15).

3. Resolution — the long way or the quick way

`updateReportStatus(reportId, status)` moves a report to `resolved` or `dismissed`, stamping `resolved_at`/`resolved_by`. Separately, a "Quick Actions" panel on the same page can ban, warn, or hide in a single click, auto-resolving the report as a side effect.

> **NOTE**: The Quick Actions panel's ban action used to always call `banUser(userId, reason, false, 7)` — a hardcoded 7-day suspension, even though the underlying `banUser` Server Action fully supports an arbitrary duration (`banUser(userId, reason, isPermanent, durationDays?)`, Chapter 15). The panel now shows a duration (days) field alongside the reason textarea when the quick action is `ban`, defaulting to 7 but editable before confirming.

4. What happens inside `banUser`, concretely

1. Guard: `guardRole('moderator')`.
2. Insert a `user_bans` row (permanent history, Chapter 12).
3. Update `profiles.is_banned`/`ban_reason`/`ban_expires_at` (the summary fields the Proxy layer reads on every request, Chapter 7).
4. Insert a `notifications` row telling the user they've been banned and why.
5. Write to `admin_audit_log`.
6. `revalidatePath('/admin')`.

These six steps are still exactly what happens, but as of a code-quality pass, steps 1, 5, and 6 are no longer written inside `banUser` itself — `banUser` is now a handler passed to a shared `withAdminAction('moderator', 'ban_user', handler)` wrapper that performs the guard *before* calling the handler and the log/revalidate *after* it returns, identically for all 24 admin actions (Chapter 15 has the full code). Steps 2–4 above are the part that's genuinely specific to banning someone, and are all that's left inside `banUser`'s own handler body.

The very next time that banned user makes any request to a protected route, `src/proxy.ts` (Chapter 7) reads `profiles.is_banned`/`ban_expires_at` and redirects them to `/profile` — the one page a banned user can still reach, so they can at least see the ban reason rather than being stuck on an opaque redirect loop.

18.2 The audit log is the whole point

Every path through this chapter — resolve, dismiss, ban, warn, hide, categorise, assign, note — ends at exactly one `admin_audit_log` insert, visible on `admin/settings/page.tsx`'s paginated audit log view (Chapter 8), joined to the acting admin's profile. Because the log has no UPDATE or DELETE policy at all (Chapter 14), it is the one part of the admin subsystem that is genuinely tamper-evident: even a `super_admin` cannot quietly edit history after the fact through the normal application layer.

18.3 Beyond reports: content and settings

Two admin pages never touch a report at all:

- **Content** (`admin/content/page.tsx`) manages `announcements` (with the single-active-banner invariant enforced in `createAnnouncement`, Chapter 12), `genres`, `areas`, and `book_of_month` (same single-active-row pattern via `saveBotm`).
- **Settings** (`admin/settings/page.tsx`) edits `platform_settings` directly — the same key/value table the rate-limiting triggers read from (Chapter 13). Toggling `feature_clubs`, `feature_wishlists`, `feature_messages`, `feature_ratings`, or `maintenance_mode` here now has an observable effect: `src/lib/platform-settings.ts` centralises reading and parsing these flags, `src/proxy.ts` (Chapter 7) redirects away from a disabled feature's routes (and everyone but admins to `/maintenance` when maintenance mode is on), the sidebar nav hides the corresponding link, and the requests page hides its Rate/Message buttons accordingly. `updatePlatformSetting` is also the action guarded by `is_super_admin()` rather than

`is_admin_or_mod()` (Chapter 14) — the one meaningfully higher bar in the whole admin action set.

NOTE: `updatePlatformSetting` used to write its value with `JSON.parse(`"${value}"`)` — wrapping the raw incoming string in literal quote characters to coerce it into a JSON string before storing it in the `jsonb` column. Any value containing an unescaped `"` or `\` would throw a `SyntaxError` inside the (caught) `try` block, so the setting silently failed to save with only a generic error message. It now uses `JSON.stringify(value)`, the correct one-line fix.

INTERN EXERCISE

As a `moderator`-role test account (not `super_admin`), attempt to change a value on the Settings page and confirm it is rejected — then check whether the rejection happens because the button/form is hidden by the frontend, or because the underlying `updatePlatformSetting` call fails `guardRole('super_admin')` even if you force the request through. This distinguishes a UI-only gate from a real one, the same distinction Chapter 17 raised for club creation — a habit worth building for every admin feature you touch.

PART VI

Operating in Production

CHAPTER 19

Deploying to Vercel

19.1 The mechanical part

OLK is a stock Next.js App Router project, which is exactly what Vercel is built to deploy with zero custom configuration:

1. Push the repository to GitHub (or GitLab/Bitbucket).
2. In the Vercel dashboard, "Add New Project," import the repository. Vercel auto-detects Next.js and sets the build command (`next build`) and output directory automatically — nothing in `next.config.ts` needs to change for this.
3. Add every environment variable from Chapter 20 in the project's *Settings* → *Environment Variables* screen, pointed at your **production/hosted** Supabase project, not the local Docker stack.
4. Deploy. Every subsequent push to the connected branch redeploys automatically; pull requests get their own preview deployment with the same environment variables.

> **WARNING**: Never put the **hosted** project's `SUPABASE_SERVICE_ROLE_KEY` into a preview-deployment-shared environment variable scope if preview deployments are ever made public or shared with anyone outside the core team — that key bypasses RLS entirely (Chapter 14). Scope service-role-key-dependent variables to Production only unless you have a specific reason to test the digest/email path from a preview deploy.

19.2 Running the database migrations against production

Vercel builds the *frontend*; it does not know anything about your Supabase project's schema. Before (or as part of) your first deploy, the hosted database needs every migration in `supabase/migrations` applied, in order:

Listing 19.1: Linking to and migrating the hosted project (run once per new migration batch)

```
npx supabase login
npx supabase link --project-ref <your-project-ref>
npx supabase db push
```

NOTE: `supabase db push` applies migrations that exist locally but haven't been applied to the linked remote project yet — it does not touch `schema.sql`, and does not care whether that dump file is stale (Chapter 6). This is the deploy-time equivalent of `supabase db reset` in local development (Chapter 4), except it applies forward-only against a database that already has real user data, so there is no reset-and-replay safety net — review any new migration carefully before pushing it against production.

19.3 The digest cron: wired up

`src/app/api/digest/route.ts` is fully implemented (Chapter 15) and is now actually triggered in production by a `vercel.json` at the repository root:

Listing 19.2: vercel.json — schedules the weekly digest

```
{
  "crons": [
    { "path": "/api/digest", "schedule": "0 8 * * 1" }
  ]
}
```

NOTE: Vercel's built-in Cron Jobs feature calls the path with `GET`, not `POST`, while `route.ts` used to only export a `POST` handler — a mismatch that would have fired the schedule every week and silently 405'd forever. The route now exports both `GET` and `POST`, delegating to the same digest logic, so it responds correctly regardless of which method triggers it.

→ HOW TO CHANGE THIS

Moving the digest from Monday 8am to, say, Friday 5pm is one line in `vercel.json` at the repository root: `"schedule": "0 8 * * 1"` → `"0 17 * * 5"` (standard 5-field cron — minute, hour, day-of-month, month, day-of-week). Vercel evaluates the schedule in UTC, not your local timezone, so convert first. This file is read only at deploy time, not per-request, so a new schedule takes effect on the *next* deploy — don't expect editing it to change a run that's already scheduled for today.

`CRON_SECRET` itself is just a long random string you generate once and set as an environment variable in both Vercel's dashboard and wherever else the digest is triggered from — there is no Supabase-side concept of this secret, it exists purely as a shared value the route handler compares against the incoming `Authorization` header.

19.4 What Vercel does *not* give you here

There is no server to SSH into, no long-running process, and — relevant to Chapter 1's longer-term goal — no way to run Supabase itself on Vercel. Vercel hosts the Next.js frontend only; the database remains wherever you point `NEXT_PUBLIC_SUPABASE_URL`, whether that's Supabase's hosted cloud today or, eventually, a self-hosted instance on a server in Kashmir. Moving the database off Supabase's cloud in the future would change exactly one thing from Vercel's perspective: the

environment variables in Chapter 20 would point at a different host. The frontend deployment story described in this chapter does not change at all.

CHAPTER 20

Environment Variables Reference

Every environment variable actually read anywhere in `src/`, in one place. `NEXT_PUBLIC_*`-prefixed variables are inlined into the browser bundle at build time by Next.js convention — never put a secret behind that prefix.

Variable	Required?	Purpose & where it's read
`NEXT_PUBLIC_SUPABASE_URL`	Yes	The Supabase API gateway URL. Read in `src/lib/supabase/client.ts`, `server.ts`, `proxy.ts`, and both service-role clients. Points at `http://127.0.0.1:54321` locally (Chapter 4) or your hosted project's URL in production.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	The public, RLS-constrained anon key. Safe to expose in the browser bundle by design — it grants nothing RLS doesn't independently allow (Chapter 14).
`SUPABASE_SERVICE_ROLE_KEY`	Only for email & digest	Bypasses RLS entirely. Read only in `src/lib/send-notification-email.ts` and `src/app/api/digest/route.ts`, both server-only files, to resolve a user's email address by id via `auth.admin.getUserById`. **Never** prefix this with `NEXT_PUBLIC_`.
`RESEND_API_KEY`	No	If unset, both `send-notification-email.ts` and the digest route silently no-op (Chapter 15) — this is the default, safe state in local development. Set it to enable real transactional email.
`RESEND_FROM_EMAIL`	No	Defaults to `"OLK<notifications@olkashmir.com>"` if unset. Must be a domain verified in your Resend account before real sends will succeed.
`NEXT_PUBLIC_SITE_URL`	No	Defaults to `https://olkashmir.com` if unset. Used to build the "View on OLK" link inside notification and digest emails — set this explicitly for any non-production deployment (a staging URL, a preview deploy) so emails sent from that environment link back to the right place instead of the production domain.
`CRON_SECRET`	Only for the digest	The shared-secret bearer token `/api/digest` checks (Chapter 15). Generate a long random string once; it has no relationship to Supabase at all.

20.1 Local vs. production, side by side

Variable	Local (`.env.local`)	Production (Vercel dashboard)
`NEXT_PUBLIC_SUPABASE_URL`	`http://127.0.0.1:54321`	Your hosted project's URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	The demo key `supabase start` prints	Your hosted project's anon key
`SUPABASE_SERVICE_ROLE_KEY`	Usually omitted (email/digest just no-op)	Your hosted project's service role key
`RESEND_API_KEY`	Usually omitted	Set, for real email
`CRON_SECRET`	Any placeholder string, for manual `curl` testing (Chapter 15)	A real, long random secret

PRO TIP: Chapter 4 recommended keeping a second, gitignored `.env.remote` file with the hosted project's `URL`/`ANON_KEY` pair, precisely so switching `.env.local` between local and hosted development is a two-line copy-paste rather than a trip back to the Supabase dashboard every time.

→ HOW TO CHANGE THIS

An unset `NEXT_PUBLIC_SITE_URL` silently falls back to the production domain — the line `const siteUrl = process.env.NEXT_PUBLIC_SITE_URL || 'https://olkashmir.com'` appears in *both* `src/lib/send-notification-email.ts` and `src/app/api/digest/route.ts`. Deploying a staging environment without setting the variable doesn't error — it just quietly links every email's "View on OLK" button back to production. Set `NEXT_PUBLIC_SITE_URL` explicitly for that environment rather than changing the fallback in code, which would flip the default for production too.

INTERN EXERCISE

Grep the entire `src/` tree for `process.env` (`grep -rn "process.env" src/`) and confirm the result set matches exactly the seven variables in this chapter — no more, no fewer. If you ever add an eighth, this chapter needs a matching new row in the same pull request (Chapter 6's rule about keeping this manual in sync applies here too).

CHAPTER 21

The Maintenance Playbook

A cookbook, organised as "if you need to do X, start here." Each entry names the exact files to touch and the chapter with the full background.

21.1 "I need to add a new page"

1. Create `src/app/(dashboard)/your-route/page.tsx` (or outside the group if it shouldn't share the dashboard shell — Chapter 7).
2. Decide Server vs. Client Component *first*, based on whether it needs interactivity (Chapter 3) — don't default to `"use client"` out of habit; most new read-only pages should be Server Components even though most existing ones aren't (Chapter 8).
3. If it should be reachable only when logged in, add its path prefix to `PROTECTED_ROUTES` in `src/proxy.ts` (Chapter 7) — this is a manually maintained array, nothing enforces the addition for you.
4. Add a link in `dashboard-sidebar.tsx` if it belongs in primary nav (Chapter 9).

21.2 "I need to change the database schema"

1. `npx supabase migration new descriptive_name` (Chapter 6 — never hand-create the file).
2. Write the migration with a comment block explaining *why*, following the house style in `20260701182526_...` and `20260701182527_...` (Chapter 6).
3. `npx supabase db reset` locally, and manually re-exercise every page that touches the changed table (Chapter 6 — the test suite is minimal and does not cover page-level flows yet).
4. If the change touches `books` or `clubs`, check whether `get_books_nearby`/`get_clubs_nearby` need their `RETURNS TABLE` column list updated too (Chapter 13 — this has bitten this codebase once already).
5. Update Chapter 12 of *this manual* in the same pull request.

21.3 "I need to add a new admin action"

Add a function to `src/lib/admin-actions.ts` following the guard/act/log/revalidate shape exactly (Chapter 15): `guardRole(minRole)` first, the mutation(s), an `auditLog(...)` call, `revalidatePath('/admin')`, and a `try/catch` returning `{success, error?}`. Pick `minRole` deliberately — `'moderator'` for anything reversible, `'super_admin'` for anything destructive or that changes another admin's privileges (Chapter 14).

21.4 "I need to add a new notification type"

1. Call `createNotification` (Chapter 15) with a new `type` string from wherever the triggering event happens.
2. If it should also send an email, add an entry to `NOTIFICATION_SUBJECTS` in `send-notification-email.ts` (Chapter 15) — falls back to a generic subject if you skip this, so it degrades gracefully, but do it anyway.
3. Add an icon/label for it in *both* places that maintain that mapping independently: `notifications/page.tsx` and `notification-bell.tsx` (Chapter 8 flags these as already-drifted duplicates — adding a third divergence makes this worse, so consider extracting a shared constant while you're in there).
4. Ensure any user-influenced text passed into the notification's `title` goes through `escapeHtml` before it can reach the email HTML template (Chapter 15).

21.5 "I need to change a rate limit or feature flag"

Edit the row in `platform_settings` — either through `admin/settings/page.tsx` directly, or a migration's `INSERT ... ON CONFLICT (key) DO UPDATE` if it should ship as a new default (Chapter 13). Remember `get_platform_setting_int` expects the stored JSON to be either a JSON string or JSON number — writing raw unquoted text into the `value` column outside the app's own read/write path can break the `#>> '{}'` unwrap (Chapter 13).

21.6 "I need to test something as a banned user / admin / moderator"

Register a fresh account locally (Chapter 4), then flip the relevant `profiles` columns directly in Studio's table editor or via SQL — `is_banned = true`, or `is_admin = true, admin_role = 'moderator'` (both fields, together — Chapter 14 explains why one alone grants nothing). This is faster and more reliable than trying to reach the same state through the UI.

21.7 "I need to regenerate the schema dump"

`npx supabase db dump -local -schema public > supabase/schema.sql` (Chapter 6), after your local database reflects every migration you want captured. Do this deliberately, as its own commit, not as an incidental side effect of an unrelated change.

21.8 "I need to run a security & code-quality audit"

This isn't a one-time event described elsewhere in this manual as history — it's a practice worth repeating periodically (monthly is a reasonable cadence for a project this size), and the process is the same each time:

1. **Survey before fixing.** Look across at least three angles, since they catch different things: leaked secrets/credentials (grep the tree and git history, check `.gitignore` covers every `.env*` file, check client-vs-server secret boundaries per Chapter 15); code quality (god files, duplicated patterns, dead code — Chapter 6's lint-debt cleanup is exactly this category); and RLS/database security, which means reading every migration *in order* rather than any single one in isolation (Chapter 14) — a policy can look correct in the migration that added it and still be wrong in the schema's current, cumulative state, the way `books`' legacy delete policy was.
2. **Verify a finding yourself before trusting it, especially a severe one.** Don't just accept a claimed vulnerability — reproduce it. The profile self-escalation issue (Chapter 14) was confirmed by reading the actual `GRANT`/policy statements in the migration files before writing a single line of fix, not by taking a description of the bug at face value.
3. **Fix in severity order, and get explicit sign-off before anything touches the linked cloud project.** Write migrations locally, run `supabase db reset` (Chapter 6), and prove the fix with SQL that simulates *both* the attack and the legitimate flow it must not break — `SET ROLE authenticated; SET request.jwt.claims = '...'` to impersonate a specific test user's session in the local Studio SQL editor (Chapter 14's intern exercise shows the `anon`-role version of this same technique) is how every fix in this pass was actually proven, not assumed. Only run `supabase db push` once local proof exists, and only against the local stack's own linked project — never experiment with attack payloads against the hosted database.
4. **Verify code-quality fixes by driving the real app, not just by reading the diff.** `tsc -noEmit`, `npm run lint`, and `npm run build` catch a lot, but none of them prove a refactored page still works end to end. Point the dev server at the *local* Supabase stack specifically for this (Chapter 4) — a headless browser script that logs in as a real local test account and exercises the actual flow (add a book, accept a request, whatever the diff touched) is the only thing that catches a prop wired to the wrong callback after a component split.
5. **Update this manual in the same pass**, per the entry below — not as an afterthought once the code is merged.

21.9 "Something changed in this codebase and this manual is now wrong"

Fix the manual in the same pull request as the code change, in the relevant `Guide/chapters/*.tex` file. This is not a nice-to-have: Chapter 12's note about `schema.sql` having gone five migrations stale is a direct, cautionary illustration of what happens to any documentation artifact that isn't updated as part of the change it describes. Don't let this manual become the next example.

INTERN EXERCISE

Pick any one row in this playbook you haven't personally done yet, and do it against your local stack, start to finish, before reading further into this manual. Muscle memory beats reading every time.

CHAPTER 22

Known Issues & Technical Debt

This chapter used to list 21 items — every **KNOWN ISSUE** flagged earlier in this manual, collected in one place so a maintainer could find "what's currently wrong with this codebase" without reading 21 chapters first. All 21 have since been fixed (correctness bugs, security gaps, and most of the design/consistency debt), each in the chapter that originally flagged it, per this chapter's own house rule below. A pre-existing lint-debt problem discovered along the way (Chapter 6) has since been fixed too — and so has the 37-warning `react-hooks/exhaustive-deps`/`@next/next/no-img-element` debt that this chapter went on to flag once the error-level debt was clear, and the stock-Tailwind-to-`brand-*` token migration (Chapter 10) this chapter flagged next. The remaining `alert()` consistency item below has since been closed out too, by a full security & code-quality audit (Chapter 21) that also fixed several RLS gaps (Chapter 14) and a notification-spoofing hole (Chapter 15) that this chapter had never flagged in the first place — this manual only documents debt once someone finds it, not debt that exists but hasn't been noticed yet. What's left is a short, honest list of what's still actually open.

22.1 Still open

1. `about-modal.tsx` **still has a placeholder bio for one founder.** Owais Saleem's bio is real; Malik Umair's is still "Biography coming soon." — waiting on him to provide the text, not a code task. Chapter 9.
2. **Automated test coverage is minimal.** Vitest is configured and covers a handful of pure functions (Chapter 6), but there is no component, integration, or end-to-end coverage. Growing this is ongoing, incremental work, not a single fixable bug.

> **NOTE**: If you fix one of the items above, delete its entry from this chapter in the same pull request — an item that's been fixed but still listed here is its own small instance of documentation drift, which is exactly the failure mode this chapter exists to avoid repeating.

CHAPTER 23

Troubleshooting Guide

Symptom-first, ordered roughly by how often each actually comes up.

"I changed `.env.local` but the app still hits the old backend"

Next.js bakes `NEXT_PUBLIC_*` variables into the server process at startup — it does not hot-reload them. Kill `npm run dev` and restart it. Chapter 4 flags this exact trap.

"`supabase start` says containers are already running, or something looks stale"

Check what's actually running and whether it matches your migrations before trusting it:

Listing 23.1: Diagnosing a possibly-stale local stack

```
docker ps --filter "name=_olk"
docker exec supabase_db_olk psql -U postgres -d postgres -c "\dt public.*"
```

Count the tables against Chapter 12 (24, as of this edition). Fewer than that means the volume predates recent migrations — run `npx supabase db reset` to bring it current. Chapter 4 documents a real instance of this happening.

"A query that should work according to the schema returns zero rows"

This is almost always RLS (Chapter 14), not a bug in the query itself. Ask: is this table's SELECT policy restricted to `auth.uid() = user_id` (or similar)? If you need to see *other users'* rows for a legitimate reason, you need a `SECURITY DEFINER` RPC (Chapter 13), not a wider client-side query — widening a table's RLS policy to work around one screen is usually the wrong fix and a security regression.

"A migration fails to apply"

→ HOW TO CHANGE THIS

Read the actual Postgres error — `supabase db reset` prints it inline, referencing the specific migration file and statement. Common causes in this codebase's style: a `CREATE POLICY` without an `IF NOT EXISTS` guard re-running against a database that already has it (wrap in the `DO $$ ... EXCEPTION WHEN duplicate_object THEN NULL; END $$;` pattern used throughout `20260627095459_admin_comprehensive.sql`, Chapter 12), or a `DROP FUNCTION` missing before a `CREATE OR REPLACE` that changes a return type (Postgres disallows changing a function's return type via `CREATE OR REPLACE` alone — Chapter 13's note on `get_books_nearby` explains this exact situation).

"Realtime (chat / notification bell) isn't updating live"

Check, in order: (1) is the local Supabase Realtime container actually healthy (`docker ps -filter name=_olk`, Chapter 4)? (2) does the channel's `filter` string exactly match the row you expect (`request_id=eq.${requestId}`, Chapter 8) — a mismatched id silently subscribes to nothing? (3) in dev, did Fast Refresh leave a duplicate/stale subscription — check for the instance-unique channel-naming pattern in Chapter 9 and confirm your own new code follows it if you're adding a new realtime subscription elsewhere.

"Email isn't sending, even in production"

Check, in order: is `RESEND_API_KEY` actually set in Vercel's environment variables (Chapter 20)? Is `RESEND_FROM_EMAIL`'s domain verified in the Resend dashboard? For the digest specifically: is anything actually calling `/api/digest` at all — recall Chapter 19's finding that no scheduler is currently wired up, so "email isn't sending" may simply mean "nothing is triggering the endpoint."

"I get a 401 testing the digest endpoint myself"

Your `Authorization` header must be exactly `Bearer <CRON_SECRET value>` (Chapter 15) — check for a stray trailing newline or space if you pasted the secret from somewhere, and confirm you're comparing against the same environment the server process actually loaded (local `.env.local` vs. Vercel's dashboard value are independent).

"Every `npm run dev` boot prints a Proxy/Middleware deprecation warning"

Expected, for now — see Chapter 7 for the full explanation and the migration task to make it go away permanently.

"Port 3000 (or 54321–54327) is already in use"

Something — often a dev server or Supabase stack from a previous session — is still running. `ss -ltnp | grep <port>` or `lsof -i :<port>` to identify the process; for a stray `next dev`, killing it and restarting is safe and stateless. For Supabase container ports, prefer `npx supabase stop` over killing Docker containers directly, so the CLI's own bookkeeping stays consistent.

"I'm not sure if something is really blocked by RLS or just by the UI"

This distinction matters enough that it recurs as its own exercise in Chapters 8, 17, and 18.

→ HOW TO CHANGE THIS

The reliable test: open the browser devtools network tab (or Studio's SQL editor with `SET ROLE authenticated;` / `SET ROLE anon;`), and try the operation directly against the REST API or SQL editor rather than through the app's UI. If it succeeds there despite the UI normally preventing it, the gate is UI-only.

INTERN EXERCISE

Deliberately break something small and practice diagnosing it with this chapter alone: temporarily rename a column referenced by `get_books_nearby` in a scratch migration, run `supabase db reset`, and work through the resulting error using only the "a migration fails to apply" entry above. Then revert the scratch migration. This builds the single most useful skill for maintaining this codebase — reading a real Postgres error and knowing where in the six-migration, twenty-four-table schema it's actually pointing.

PART VII

Appendices

APPENDIX A

Full Table Reference

A single-page-per-eye-scan version of Chapter 12, for quick lookup. Types are abbreviated (`tz` = `timestamptz`). **FK** column names the table it references.

Table	Columns	Key FKs
`profiles`	id, display_name, area_name, latitude, longitude, bio, email_digest, is_admin, admin_role, is_banned, ban_reason, ban_expires_at, warning_count, last_active_at, created_at, updated_at	id → auth.users
`books`	id, owner_id, title, author, condition, listing_type, status, genre, cover_url, lending_duration_months, acquired_via_donation, read_count, description, publication_year, hidden_by_admin, admin_hide_reason, hidden_at, created_at	owner_id → auth.users
`book_requests`	id, book_id, requester_id, status, created_at, handed_over_at, completed_at	book_id → books; requester_id → profiles
`book_progress`	id, book_id, request_id, reader_id, progress_pct, updated_at	book_id → books; request_id → book_requests; reader_id → profiles
`messages`	id, request_id, sender_id, content, created_at	request_id → book_requests; sender_id → profiles
`bookmarks`	id, user_id, book_id, created_at	user_id → profiles; book_id → books
`ratings`	id, request_id, rater_id, rated_user_id, score, comment, created_at	request_id → book_requests; rater_id/rated_user_id → profiles
`wishlists`	id, user_id, title, author, genre, active, matched_book_id, created_at	user_id → profiles; matched_book_id → books
`book_notes`	id, book_id, user_id, note, created_at	book_id → books; user_id → profiles
`clubs`	id, name, description, interest, area_name, latitude, longitude, cover_url, creator_id, member_count, active, created_at	creator_id → profiles
`club_members`	id, club_id, user_id, joined_at	club_id → clubs; user_id → profiles

Table	Columns	Key FKs
club_posts	id, club_id, author_id, content, created_at	club_id → clubs; author_id → profiles
club_events	id, club_id, creator_id, title, description, cover_url, starts_at, ends_at, is_online, location_name, meeting_url, latitude, longitude, visibility, capacity, attendee_count, active, created_at	club_id → clubs; creator_id → profiles
event_rsvps	id, event_id, user_id, created_at	event_id → club_events; user_id → profiles
notifications	id, user_id, type, title, body, link, read, created_at	user_id → profiles
notification_templates	id, name, title, body, type, created_at	—
announcements	id, admin_id, title, body, type, is_banner, active, starts_at, ends_at, created_at	admin_id → profiles
genres	id, name, display_order, active, created_at	—
areas	id, name, district, active, created_at	—
book_of_month	id, title, author, description, cover_url, month_label, active, created_at	—
platform_settings	key (PK), value (jsonb), description, updated_at, updated_by	updated_by → profiles
reports	id, reporter_id, reported_user_id, reported_book_id, reason, details, status, category, assigned_to, resolved_at, resolved_by, created_at	reporter_id/reported_user_id/ass → profiles; reported_book_id → books
user_bans	id, user_id, admin_id, reason, is_permanent, expires_at, unbanned_at, unbanned_by, created_at	user_id/admin_id/unbanned_by → profiles
user_warnings	id, user_id, admin_id, reason, created_at	user_id/admin_id → profiles
admin_audit_log	id, admin_id, action, target_type, target_id, details (jsonb), created_at	admin_id → profiles
admin_notes	id, report_id, admin_id, content, created_at	report_id → reports; admin_id → profiles

APPENDIX B

Command Cheat-Sheet

Every command this manual actually uses, grouped by tool.

B.1 Node / Next.js

```
npm install          # install dependencies
npm run dev          # start the dev server (localhost:3000)
npm run build        # production build
npm run start        # run a production build locally
npm run lint         # ESLint
```

B.2 Supabase CLI (via npx — nothing installed globally)

```
npx supabase init                       # one-time: create supabase/config.toml
npx supabase start                      # start the local Docker stack
npx supabase stop                       # stop it (data persists in a volume)
npx supabase status                     # reprint local credentials
npx supabase db reset                   # wipe + replay every migration, local only
npx supabase migration new <name>       # scaffold a correctly-timestamped migration
npx supabase db dump --local --schema public > supabase/schema.sql
                                         # regenerate the schema.sql snapshot

# Hosted project (production) only:
npx supabase login
npx supabase link --project-ref <ref>
npx supabase db push                    # apply pending migrations to the hosted project
```

B.3 Docker (rarely needed directly — the Supabase CLI drives this for you)

```
docker ps --filter "name=_olk"          # see which OLK containers are running
docker exec supabase_db_olk psql -U postgres -d postgres -c "\dt public.*"
                                         # list tables directly against the local DB
docker info                             # confirm the daemon is reachable at all
```

B.4 Verifying the environment (Chapter 4)

```
docker --version
docker info           # confirms the daemon is reachable, not just the CLI
groups                # confirm your user is in the 'docker' group on Linux
```

B.5 Compiling this manual

```
cd Guide
pdflatex main.tex && pdflatex main.tex   # run twice: once for content, once for
                                          # the table of contents and cross-references
# or, if latexmk is available:
latexmk -pdf main.tex
```

PRO TIP: Every `Chapter ??` you might see in a single-pass compile is not an error — it means the cross-reference hasn't been resolved yet because LaTeX needs a second pass to know where every labelled chapter landed. Always compile twice (or use `latexmk`, which does this automatically) before judging the output.

APPENDIX C

Glossary

Term	Meaning in this codebase
App Router	Next.js's file-system-based routing convention rooted at `src/app`, used exclusively in this project (Chapter 7).
Client Component	A React component marked `"use client"`; renders once on the server, then hydrates and re-renders in the browser (Chapter 3).
Server Component	The App Router default; renders only on the server, never ships its own code to the browser (Chapter 3).
Server Action	A `'use server'`-marked function, callable directly from a Client Component as if local, actually executing on the server (Chapter 15).
Route Handler	A `route.ts` file exporting HTTP verbs (`GET`/`POST`/...) — a genuine API endpoint, used exactly once in this codebase, for the digest cron (Chapter 15).
Proxy	Next.js 16's renaming of Middleware — code that runs before every matched request, in this project's `src/proxy.ts` (Chapter 7).
RLS (Row-Level Security)	Postgres's built-in per-row access control; the entire authorization model for ordinary data access in this app (Chapter 14).
RPC	A call to a Postgres function via `supabase.rpc(...)`, as opposed to a query-builder `.from().select()` call (Chapter 11).
`SECURITY DEFINER`	A Postgres function attribute making it run with its owner's privileges rather than the caller's — the mechanism used throughout this schema to deliberately bypass RLS for narrow, specific purposes (Chapter 11).
PostgREST	The Supabase component that auto-generates a REST API from the Postgres schema — what `supabase.from(...)` actually talks to (Chapter 11).
GoTrue	Supabase's authentication service; owns the `auth.users` table (Chapter 11).
Realtime	The Supabase service that streams Postgres row changes to subscribed clients over WebSockets; powers chat and the notification bell (Chapters 8, 9).
JWT	JSON Web Token; what GoTrue issues on login and what RLS policies evaluate via `auth.uid()`/`auth.role()` (Chapter 14).

Term	Meaning in this codebase
Foreign-key embed	PostgREST's ability to nest a related table's columns inside a query's result by walking an actual foreign-key constraint, e.g. `.select('*, profiles(display_name)')` — only works across FK edges the schema cache actually knows about (Chapters 8, 11).
Migration	A single, chronologically-timestamped SQL file in `supabase/migrations`, applied in order; the only correct way to change the schema (Chapter 6).
Trigger	A Postgres function automatically invoked on a table event (`INSERT`/`UPDATE`/`DELETE`); used in this schema for profile provisioning, read-count increments, club member counting, and rate limiting (Chapter 13).
Haversine formula	The great-circle-distance calculation (`6371 * acos(...)`) used by `get_books_nearby`/`get_clubs_nearby` to sort by real-world distance (Chapter 13).
`pg_trgm`	A Postgres extension providing trigram-based text similarity (`similarity()`); backs the wishlist fuzzy-matching RPC (Chapter 13).
`jsonb`	Postgres's binary JSON column type; used for `platform_settings.value` and the two audit/detail `details` columns (Chapter 12).
Service role key	A Supabase API key that bypasses RLS entirely; used only in two server-only files in this codebase to resolve email addresses for sending mail (Chapters 2, 15).
Anon key	The public, RLS-constrained Supabase API key shipped to the browser; grants nothing RLS doesn't independently allow (Chapter 20).
Admin role hierarchy	`viewer` < `moderator` < `super_admin`, the `admin_role` enum gating everything under `/admin` (Chapter 14).
N+1 avoidance	Fetching related data for many rows in one batched query (`.in('id', allIds)`) instead of one query per row in a loop — a recurring, deliberate pattern across this codebase's frontend (Chapters 8, 17).